AWS CLOUD | HANDS-ON IMPLEMENTATION GUIDE

Deploy a Three-Tier Web Application on AWS

A secure, highly available and scalable three-tier architecture, built step by step with a custom VPC, EC2, RDS for MySQL, Application Load Balancers and Auto Scaling in the Asia Pacific (Sydney) Region.

VPC | EC2 | RDS MySQL | ALB | Auto Scaling | S3 | IAM | NAT | SSM

REGION	VERSION	DATE	SCOPE
Asia Pacific (Sydney) ap-southeast-2	1.0	November 2026	15 steps | 3 tiers 6 subnets | 15 services

CREATED BY



Mohit Gadilohar

Cloud & DevOps Project | Implementation Documentation

LinkedIn
linkedin.com/in/mohit-gadilohar-devops

GitHub
github.com/mohitlohar1555-oss

Document at a Glance

This document records, step by step, how a three-tier web application was built and deployed on AWS in the Asia Pacific (Sydney) Region: a public web tier, a private application tier and a private MySQL database tier inside one custom VPC. Every step lists the exact settings used, the commands run and screenshots taken from the AWS console while doing the work, followed by a checkpoint so you always know you are on track.

15	3	6	15
GUIDED STEPS	APPLICATION TIERS	SUBNETS	AWS SERVICES

Document Control

ITEM	DETAILS
Title	Deploy a Three-Tier Web Application on AWS
Created by	Mohit Gadilohar
LinkedIn	linkedin.com/in/mohit-gadilohar-devops
GitHub	github.com/mohitlohar1555-oss
Version / Date	1.0 | November 2026 (deployed 5 November 2026)
AWS Region	Asia Pacific (Sydney) `ap-southeast-2`; subnets in ap-southeast-2a, 2b and 2c
Technology stack	Nginx + React (web tier), Node.js + PM2 (app tier), Amazon RDS for MySQL 8.4 (database tier)
Access method	AWS Systems Manager Session Manager: no SSH keys, no bastion host, no public IPs needed on private servers
Evidence	Screenshots are captured from the author's own AWS account during the deployment. Figures marked "Reference screenshot" or "Illustrative" are based on the source guide.

Revision History

VERSION	DATE	AUTHOR	DESCRIPTION
1.0	November 2026	Mohit Gadilohar	Initial release: complete 15-step implementation guide with screenshots, troubleshooting guide, resource inventory and hardening notes.

How to Read This Document

Each step opens with a short explanation, followed by numbered actions, settings tables, commands and screenshots. Coloured boxes call out the information that matters most:

WHY THIS STEP?
Explains the purpose of the step before you start.

IMPORTANT
A setting that must be exact, or something that affects your bill.

NOTE
Extra context or an alternative way to do the same thing.

PRO TIP
A shortcut or good habit that saves time.

WARNING
A mistake that can break the deployment or expose data.

CHECKPOINT
A quick test to confirm the step worked before moving on.

Text in a blue code chip, like `Three-Tier-Project-VPC`, marks a value you will reuse in later steps: names, CIDR blocks and ports.

Your Journey at a Glance

Four phases, fifteen steps. Every step ends with a quick check, so a mistake is caught early instead of three steps later.

PHASE 1
Network foundation
Steps 1 to 3
VPC, subnets, internet and NAT gateways, route tables, security groups

PHASE 2
Data and access
Steps 4 to 6
RDS MySQL database, IAM role, S3 bucket with the source code

PHASE 3
Servers and traffic
Steps 7 to 12
App and web servers, AMIs, launch templates, target groups, load balancers

PHASE 4
Scale and launch
Steps 13 to 15
Auto Scaling groups, open the application, clean up

Contents

Document at a Glance 3
- Document Control 3
- Revision History 3
- How to Read This Document 4
- Your Journey at a Glance 4

1. Introduction 7

2. Project Overview 8
- AWS Components Used 8
- Key Terms in Plain English 9

3. Architecture 10
- Network Plan 10
- Security Group Rules (Traffic Flow) 11
- Prerequisites 11

Step 1: Create the VPC and Subnets 12
- Create the VPC 12
- Create the subnets 13
- Enable auto-assign public IP on the web subnets 18

Step 2: Create the Internet Gateway, NAT Gateways and Route Tables 20
- Create and attach the internet gateway 20
- Create the NAT gateways 21
- Create the route tables 22
- Edit routes and subnet associations 24
- Verify with the resource map 28

Step 3: Create the Security Groups 30

Step 4: Create the DB Subnet Group and the RDS Database 34
- Create the DB subnet group 34
- Create the database 36

Step 5: Create the IAM Role 41

Step 6: Create an S3 Bucket and Upload the Source Code 45
- Create the bucket 45
- Upload the application code 48

Step 7: Create the Test App Server and Set It Up 50
- Launch the test app server 50
- Put your database details in the code 53
- Open a terminal with Session Manager 55
- Configure the app tier 55

Step 8: Create an AMI of the App Server 61

Step 9: Target Group, Launch Template and Internal Load Balancer (App Tier) 63
- A. Create the target group 63
- B. Create the launch template 67
- C. Create the internal Application Load Balancer 70

Step 10: Create the Test Web Server and Set It Up 74
- Launch the test web server 74
- Point nginx.conf at the internal load balancer 76
- Configure the web server 77

Step 11: Create an AMI of the Web Server 80

Step 12: Target Group, Launch Template and External Load Balancer (Web Tier) 82

A. Create the target group (web tier) 82

B. Create the launch template (web tier) 85

C. Create the external (internet-facing) load balancer 87

Step 13: Create the Auto Scaling Groups 91

A. Auto Scaling group for the app tier 91

B. Auto Scaling group for the web tier 91

Step 14: Access the Application 93

Prove the fault tolerance 94

Step 15: Clean Up the Infrastructure 95

Troubleshooting Quick Guide 96

Appendix A: Resource Inventory 97

Appendix B: Review Notes and Hardening 98

Ideas to harden for production 98

References and Credits 99

1. Introduction

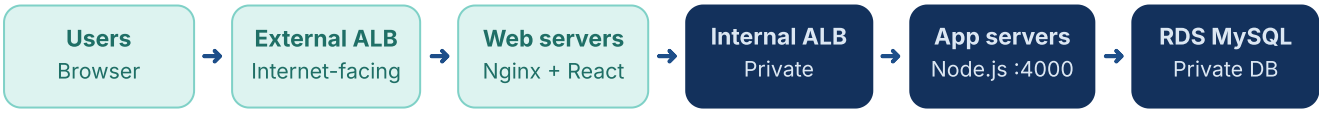
Most real applications must keep working when a single server fails. This project shows how to design for that from the first click. It builds a small, production-style application in which the website, the business logic and the data each live in their own protected layer, while AWS keeps every layer healthy with load balancing and Auto Scaling.

A three-tier architecture splits an application into three layers, each with one job. That separation improves **security** (a layer can only be reached through the one in front of it), **manageability** (you can change one layer without touching the others) and **flexibility** (each layer scales on its own).

TIER	RESPONSIBILITY	TECHNOLOGY IN THIS PROJECT
1. Web tier (presentation)	What the user sees and clicks. Serves pages and forwards API calls.	Nginx serving a React application, on EC2 in public subnets behind an internet-facing load balancer.
2. App tier (logic)	Processes requests, applies business rules, talks to the database.	Node.js API kept alive by PM2 on port 4000, on EC2 in private subnets behind an internal load balancer.
3. Database tier (data)	Stores and returns the application data.	Amazon RDS for MySQL 8.4 in private DB subnets, not publicly accessible.

A user talks only to the web tier. The web tier forwards API calls to the app tier, the app tier reads and writes the database, and the answer travels back along the same path.

How a request travels



Cyan = reachable from the internet (public subnets). Navy = private, never directly reachable.

2. Project Overview

The goal was a highly available, secure, scalable and fault-tolerant application on AWS. In practice that translated into four design rules:

- **High availability.** Subnets and load balancers span multiple Availability Zones, so losing one zone does not stop the application.
- **Security.** Only the external load balancer faces the internet. App servers and the database sit in private subnets and accept traffic only from the tier in front of them.
- **Scalability.** Auto Scaling groups build servers from launch templates, and load balancers spread requests across whatever is running.
- **Fault tolerance.** Failed health checks make Auto Scaling replace a server automatically, and each tier has at least two servers.

AWS Components Used

#	AWS SERVICE	ROLE IN THIS PROJECT
1	Amazon VPC	The isolated network (`10.0.0.0/16`) that holds every resource.
2	Amazon EC2	Runs the web and app servers (Amazon Linux 2023).
3	Amazon RDS for MySQL	Managed MySQL 8.4 database for the application data.
4	Amazon S3	Holds the application source code so any server can fetch it.
5	AWS IAM	A role that lets the servers read from S3 without stored credentials.
6	AWS Systems Manager	Session Manager gives a terminal on private servers without SSH keys.
7	Application Load Balancer	One internet-facing ALB for the web tier and one internal ALB for the app tier.
8	Target groups	The list of servers (and their health checks) behind each load balancer.
9	Auto Scaling groups	Keep the wanted number of healthy web and app servers running.
10	Launch templates	The recipe Auto Scaling uses to create each new server.
11	Amazon Machine Images	Saved copies of the configured app server and web server.
12	Internet gateway	The door between the VPC and the internet.
13	NAT gateways	Let private servers reach the internet for downloads while staying unreachable.
14	Route tables	Rules that decide where each subnet sends its traffic.
15	Security groups	Stateful firewalls around every resource.

Key Terms in Plain English

TERM	WHAT IT MEANS
VPC	Your own private network inside AWS, with an IP range you choose.
Subnet	A slice of the VPC tied to one Availability Zone. Public subnets have a route to the internet gateway; private subnets do not.
Internet gateway / NAT gateway	The internet gateway lets public subnets talk to the internet in both directions. A NAT gateway lets private servers start outbound connections only.
Route table	A list of rules saying where traffic for each destination should go.
Security group	A virtual firewall around a resource. Rules can name another security group as the source.
IAM role	Permissions a server can borrow, so no passwords or keys live on the machine.
AMI and launch template	An AMI is a saved copy of a configured server. A launch template is the recipe (AMI, size, network, role) used to build new ones.
Target group, load balancer, Auto Scaling group	A target group lists the servers a load balancer sends traffic to. The Auto Scaling group keeps the right number of healthy servers registered there.

3. Architecture

The diagram shows how the components connect and how the tiers are laid out inside the VPC.

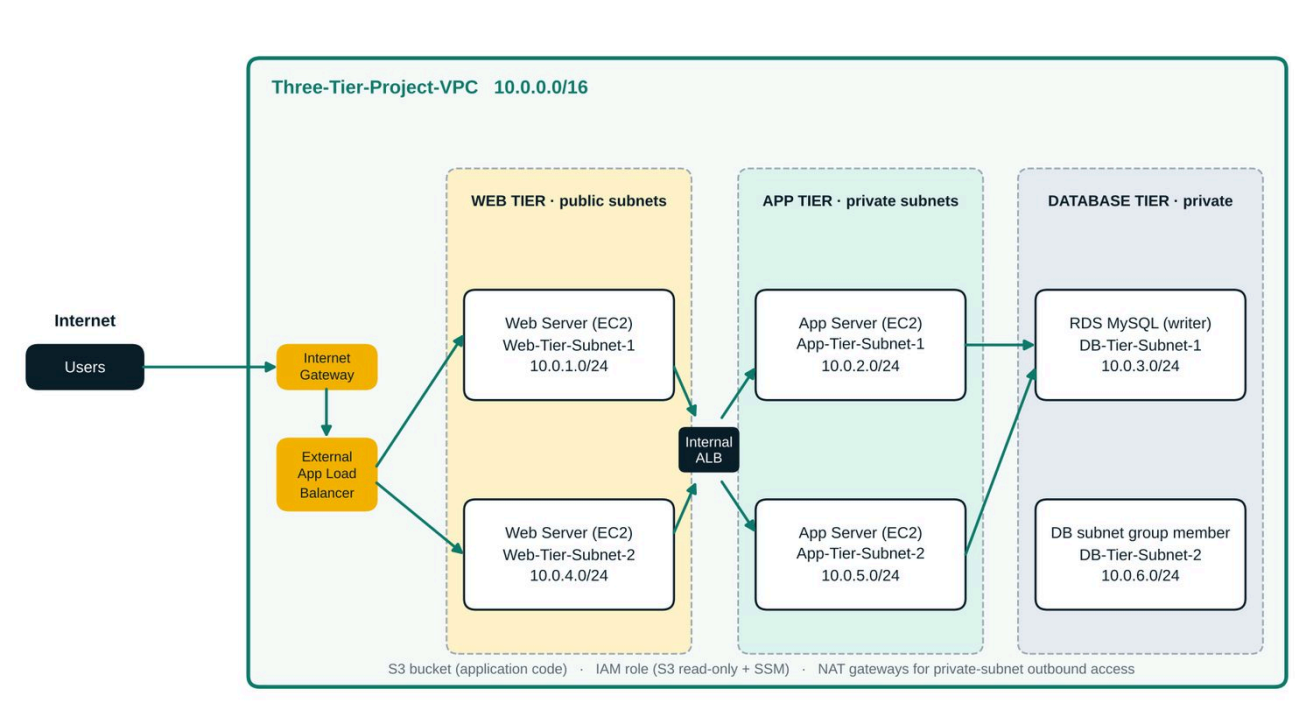


Figure A. VPC 10.0.0.0/16 with public web subnets, private app subnets and private database subnets, load balancers in front of the web and app tiers.

The web tier lives in public subnets because users must reach it. The app and database tiers are private. All of it sits in a custom VPC, and an S3 bucket carries the source code so that configuring a server is a single copy command.

Network Plan

SUBNET	AZ	TYPE	CIDR	HOSTS
Web-Tier-Subnet-1-Public	ap-southeast-2a	Public	10.0.1.0/24	Web servers, NAT-GW-1
App-Tier-Subnet-1-Private	ap-southeast-2c	Private	10.0.2.0/24	App servers
DB-Tier-Subnet-1-Private	ap-southeast-2c	Private	10.0.3.0/24	MySQL database
Web-Tier-Subnet-2-Public	ap-southeast-2b	Public	10.0.4.0/24	Web servers, NAT-GW-2
App-Tier-Subnet-2-Private	ap-southeast-2b	Private	10.0.5.0/24	App servers

SUBNET	AZ	TYPE	CIDR	HOSTS
DB-Tier-Subnet-2-Private	ap-southeast-2b	Private	10.0.6.0/24	Second DB subnet (DB subnet group)

The Availability Zones are listed exactly as they were selected in this deployment.

Security Group Rules (Traffic Flow)

SECURITY GROUP	INBOUND PORT	ALLOWED SOURCE
External-Load-Balancer-SG	HTTP (80)	0.0.0.0/0 (anyone on the internet)
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL/Aurora (3306)	App-Tier-SG

WHY THIS STEP?
Each tier accepts traffic only from the tier in front of it. That chain of security-group references is why the database can never be reached straight from the internet.

Prerequisites

- An AWS account that can create VPC, EC2, RDS, IAM, S3 and load balancer resources.
- A web browser, plus a code editor on your PC (VS Code was used) to edit two configuration files.
- One Region for everything. This project uses Asia Pacific (Sydney) `ap-southeast-2`; resources in different Regions cannot see each other.
- The application source code (a React web tier, a Node.js app tier and an `nginx.conf`) from the public sample three-tier project listed in References.

COST WARNING
Several services bill by the hour: two NAT gateways, two load balancers, the RDS database and the EC2 servers. Finish the project, capture your screenshots, then follow Step 15 to delete everything.

NOTE
You can name resources however you like as long as you stay consistent. This document uses the names visible in the screenshots.

STEP 01 OF 15 NETWORK FOUNDATION

Create the VPC and Subnets

Mark out your own private corner of AWS

WHY THIS STEP?

Everything else lives inside this private network, so it is built first. The VPC and its six subnets decide which resources can ever talk to each other.

Sign in to the AWS Console, choose **Asia Pacific (Sydney)** in the Region selector and keep it there for the whole project. Use the console search bar to open each service.

Create the VPC

1. Search for **VPC** and open **Your VPCs > Create VPC**.

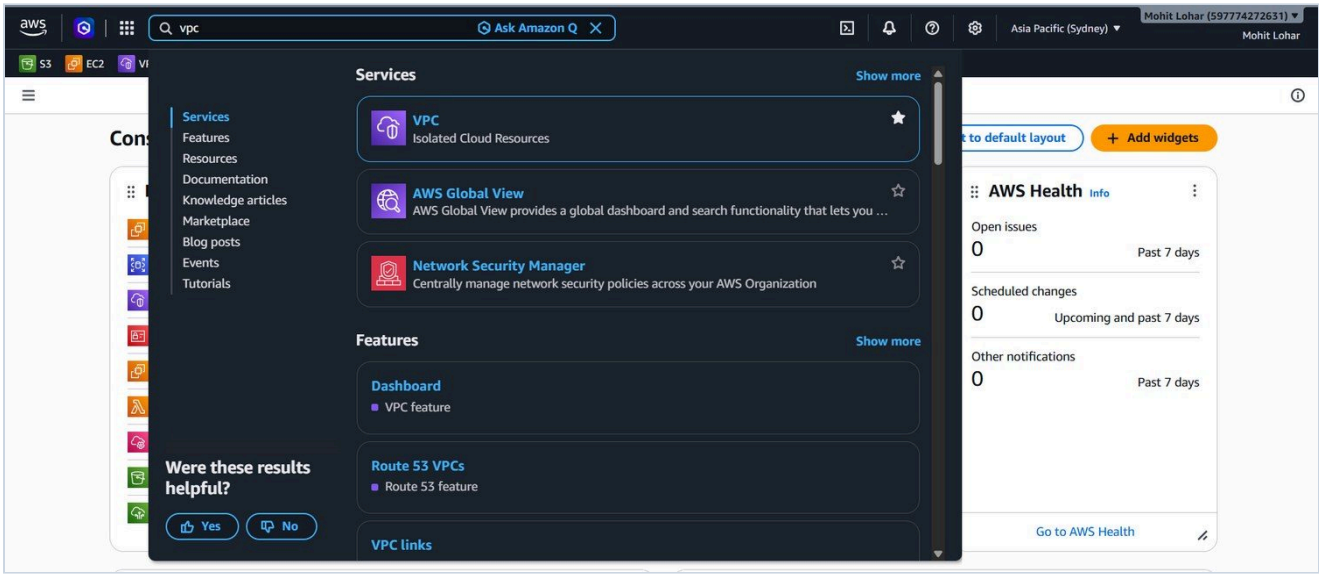


Figure 1. Searching for the VPC service in the console search bar.

2. Choose **VPC only** and set the name tag to `Three-Tier-Project-VPC`.
3. Select **IPv4 CIDR manual input** and enter `10.0.0.0/16`. Keep tenancy **Default**.

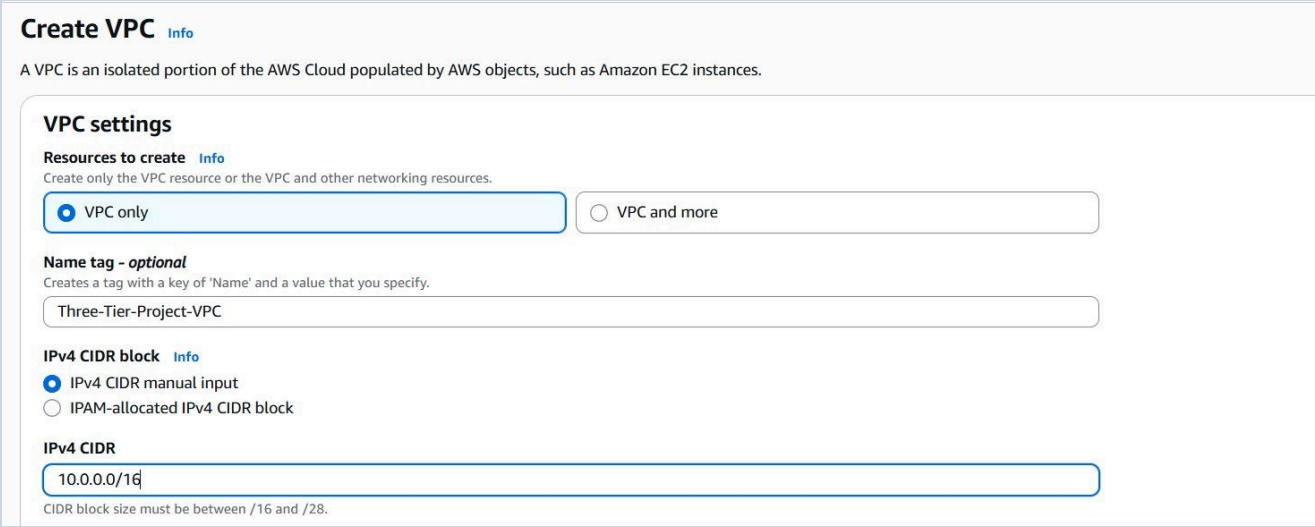


Figure 2. Create VPC: VPC only, name tag Three-Tier-Project-VPC, IPv4 CIDR 10.0.0.0/16.

4. Confirm the Name tag and click **Create VPC**.

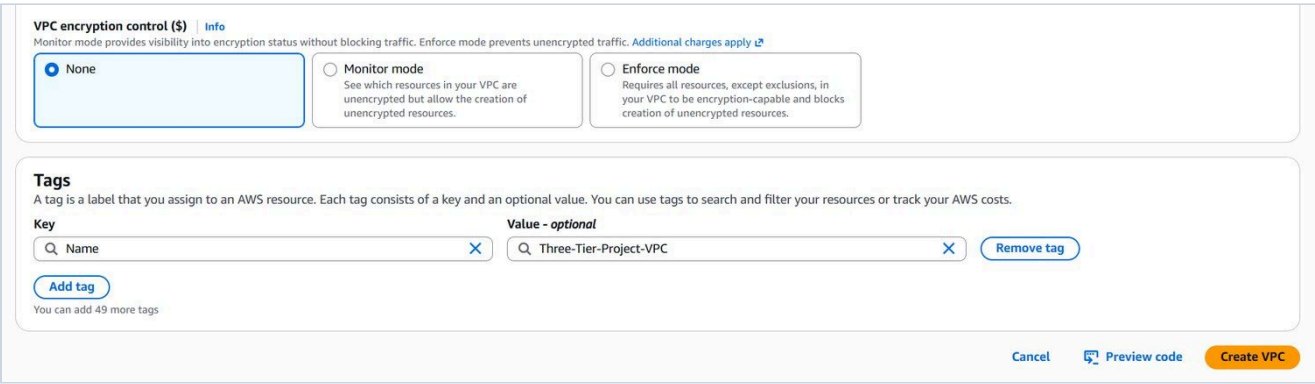


Figure 3. Scroll down, confirm the Name tag and click Create VPC.

Create the subnets

Create six subnets in the new VPC. Go to **VPC > Subnets > Create subnet**, choose `Three-Tier-Project-VPC` and enter each subnet as below. Each subnet is created from the same form: name, Availability Zone, IPv4 CIDR block, then **Create subnet**.

SUBNET NAME	AVAILABILITY ZONE	TYPE	IPV4 CIDR BLOCK
Web-Tier-Subnet-1-Public	ap-southeast-2a	Public	10.0.1.0/24
App-Tier-Subnet-1-Private	ap-southeast-2c	Private	10.0.2.0/24
DB-Tier-Subnet-1-Private	ap-southeast-2c	Private	10.0.3.0/24
Web-Tier-Subnet-2-Public	ap-southeast-2b	Public	10.0.4.0/24
App-Tier-Subnet-2-Private	ap-southeast-2b	Private	10.0.5.0/24

SUBNET NAME	AVAILABILITY ZONE	TYPE	IPV4 CIDR BLOCK
DB-Tier-Subnet-2-Private	ap-southeast-2b	Private	10.0.6.0/24

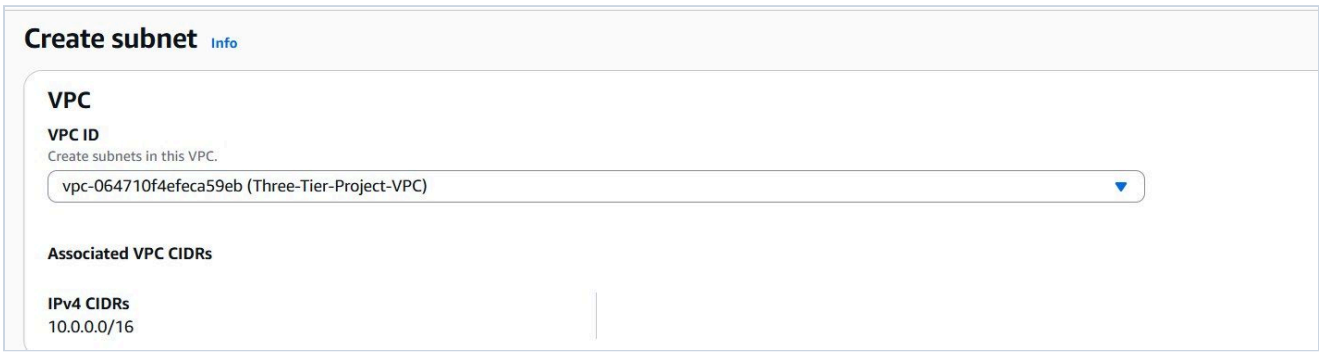


Figure 4. Choose Three-Tier-Project-VPC (VPC CIDR 10.0.0.0/16) for the new subnets.

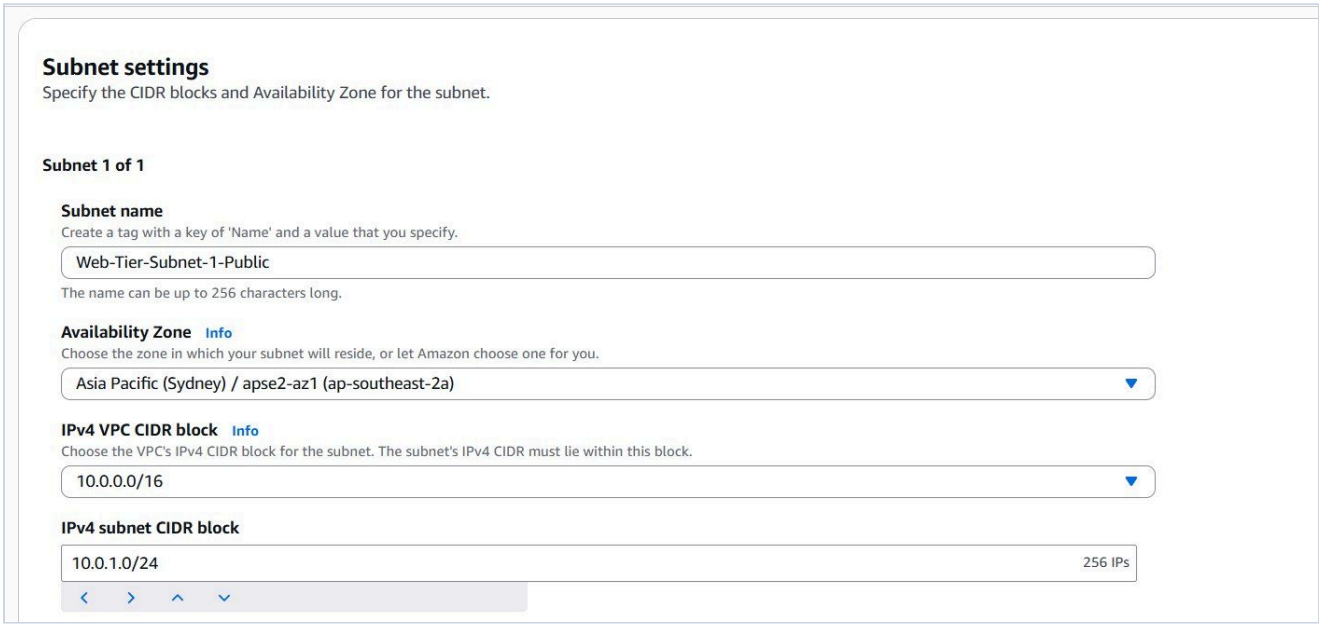


Figure 5. Subnet 1: Web-Tier-Subnet-1-Public in ap-southeast-2a (10.0.1.0/24).



Figure 6. Name tag for Web-Tier-Subnet-1-Public, then click Create subnet.

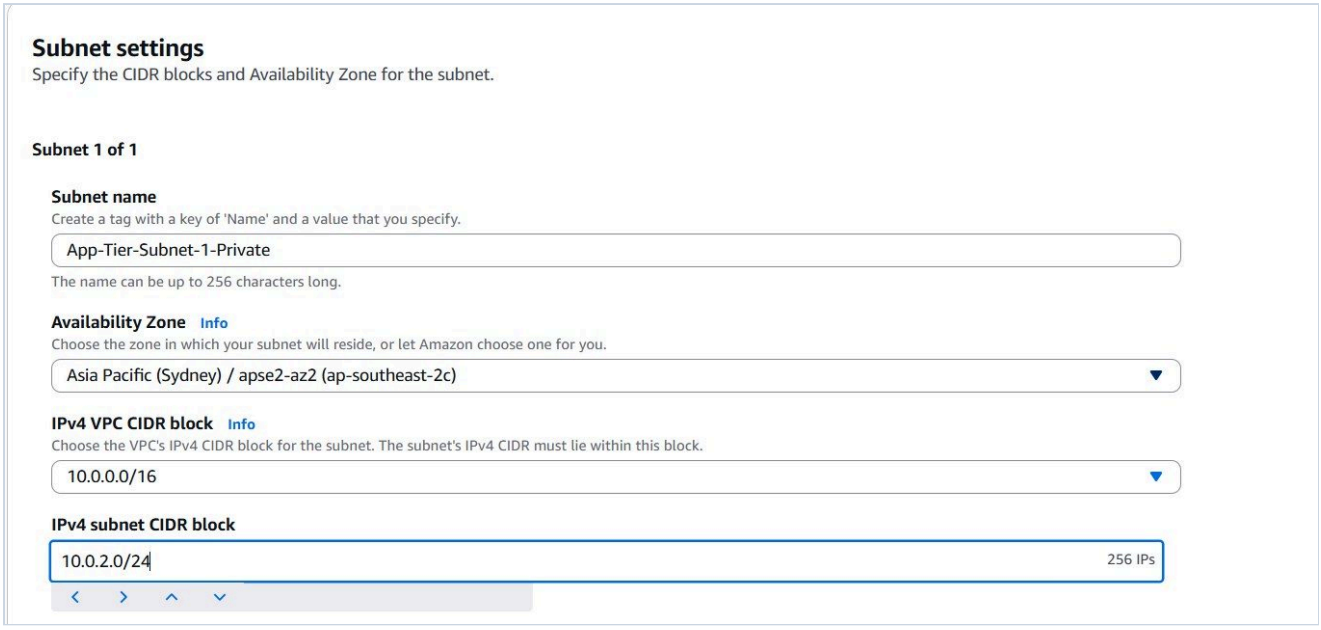


Figure 7. Subnet 2: App-Tier-Subnet-1-Private in ap-southeast-2c (10.0.2.0/24).

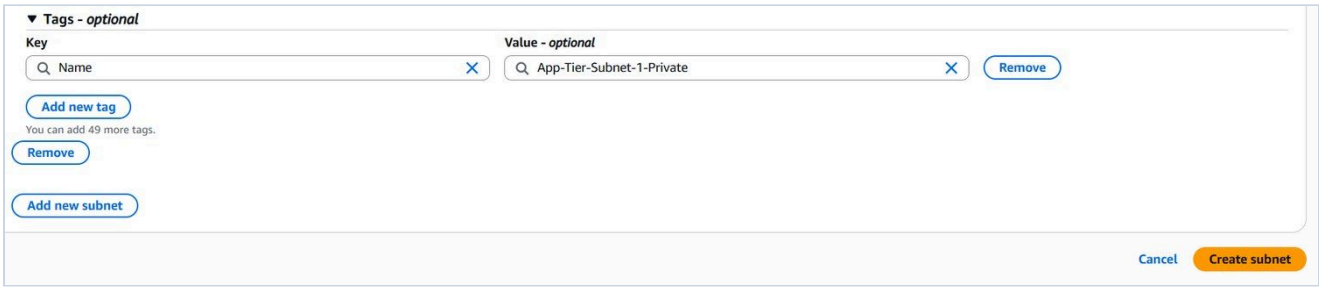


Figure 8. Name tag for App-Tier-Subnet-1-Private, then click Create subnet.

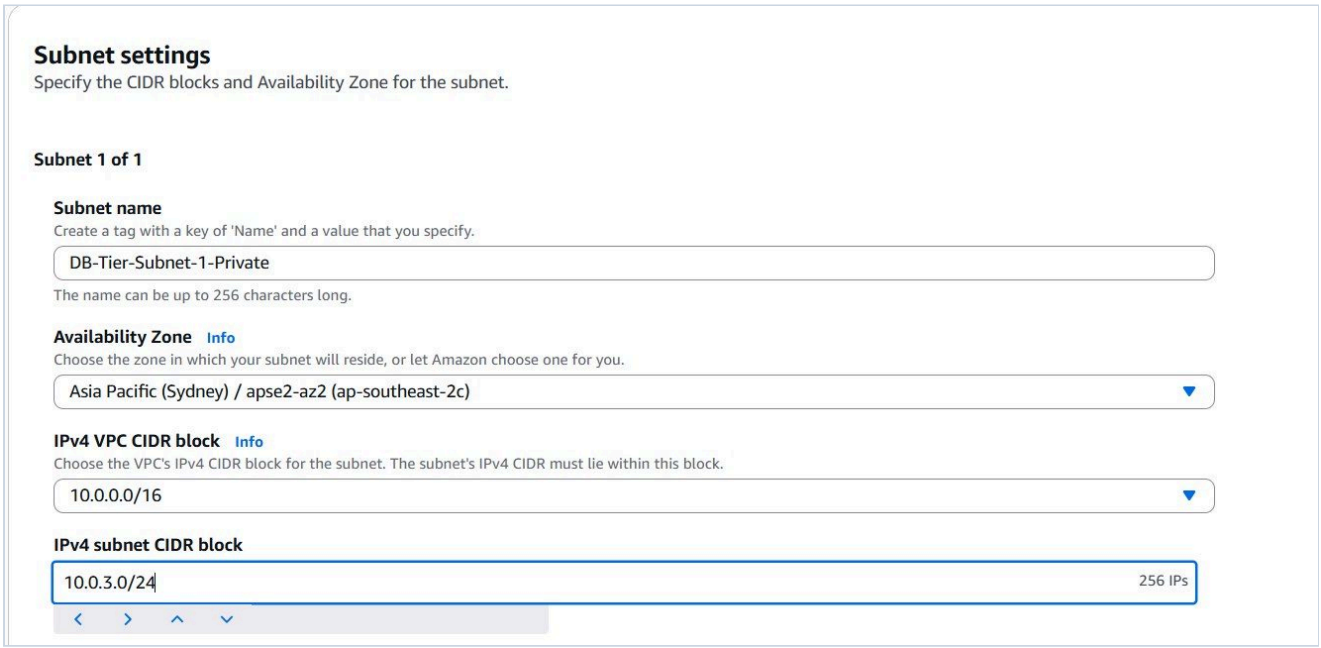


Figure 9. Subnet 3: DB-Tier-Subnet-1-Private in ap-southeast-2c (10.0.3.0/24).



Figure 10. Name tag for DB-Tier-Subnet-1-Private, then click Create subnet.

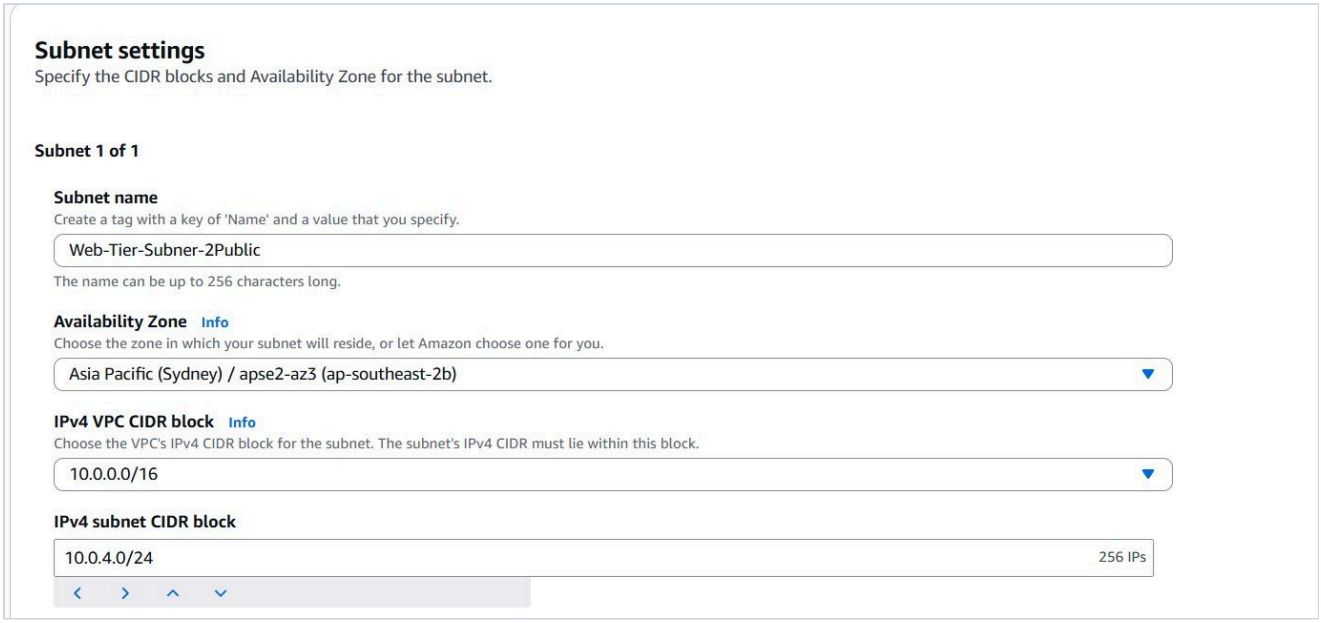


Figure 11. Subnet 4: Web-Tier-Subnet-2-Public in ap-southeast-2b (10.0.4.0/24).



Figure 12. Name tag for Web-Tier-Subnet-2-Public, then click Create subnet.

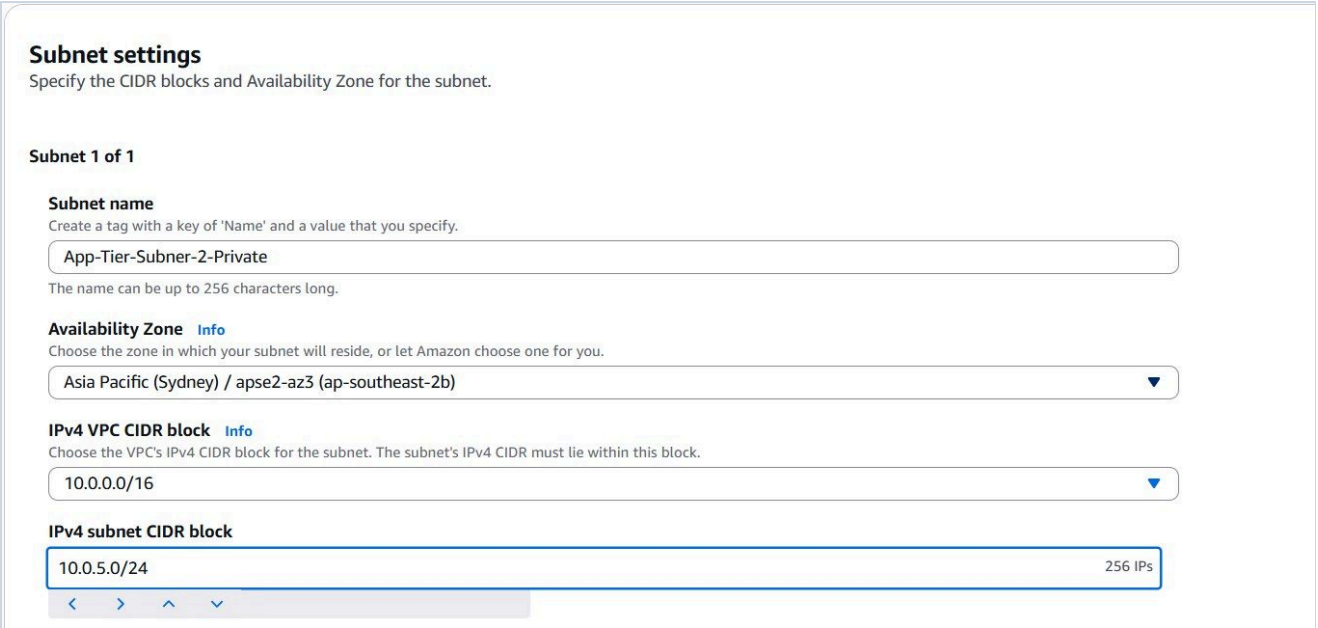


Figure 13. Subnet 5: App-Tier-Subnet-2-Private in ap-southeast-2b (10.0.5.0/24).

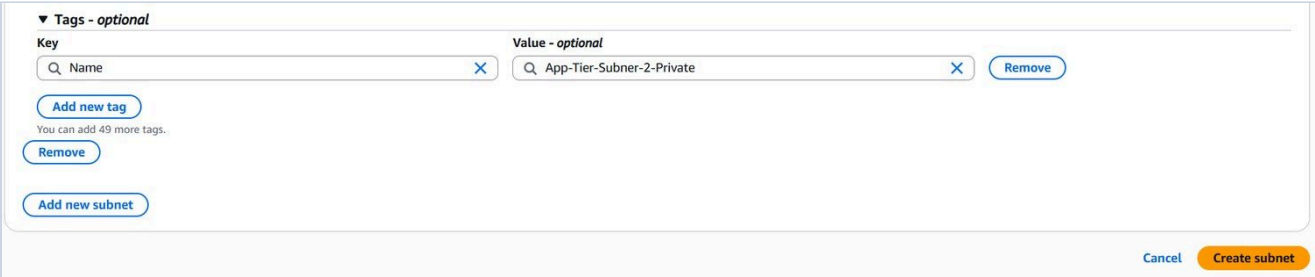


Figure 14. Name tag for App-Tier-Subnet-2-Private, then click Create subnet.

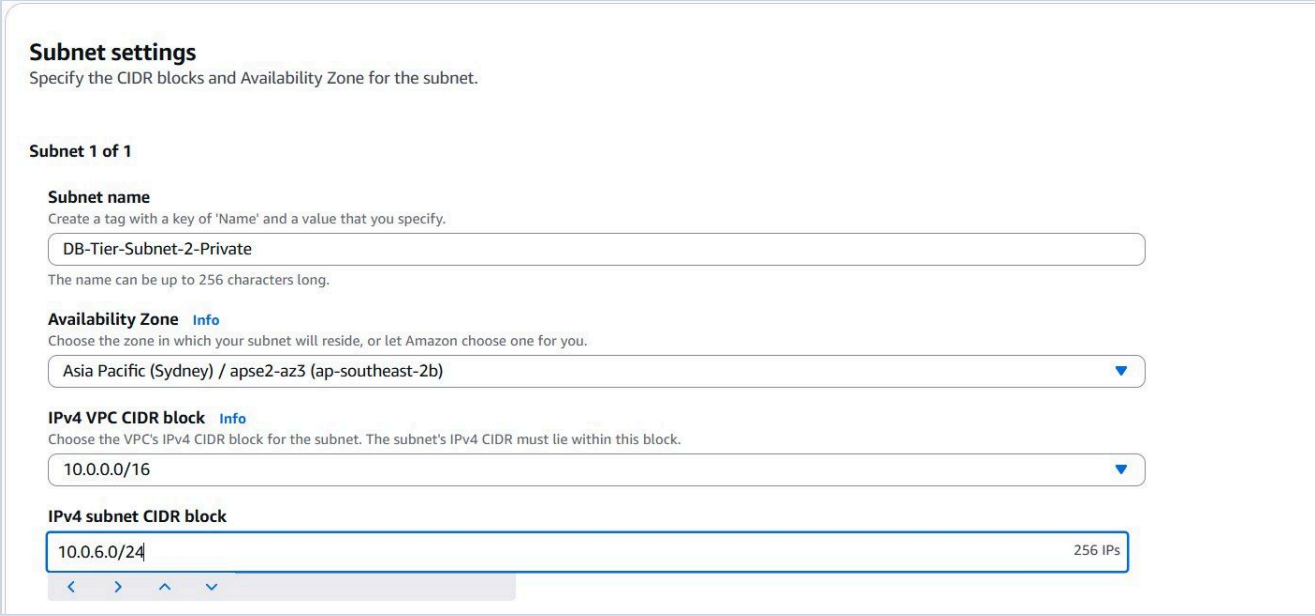


Figure 15. Subnet 6: DB-Tier-Subnet-2-Private in ap-southeast-2b (10.0.6.0/24).



Figure 16. Name tag for DB-Tier-Subnet-2-Private, then click Create subnet.

After the last subnet is created, the Subnets list shows all six subnets in the **Available** state.

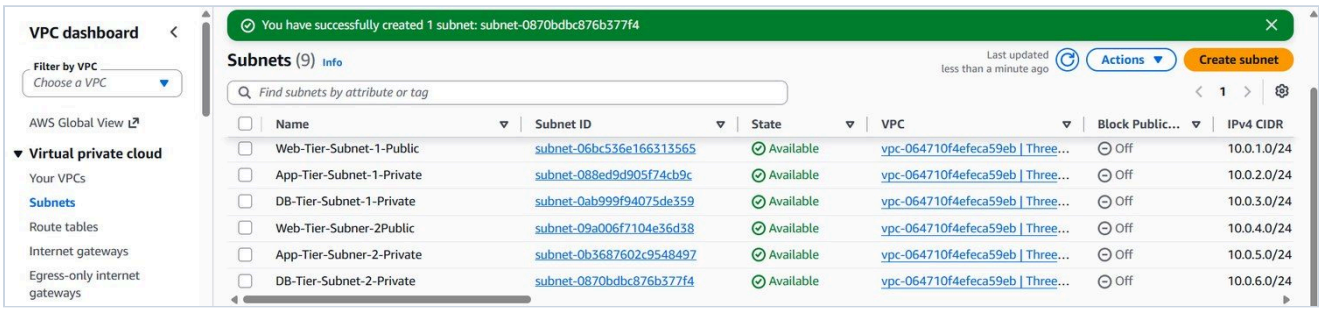


Figure 17. The Subnets list after creation: all subnets Available.

> **NOTE**
> In the screenshots the second web and app subnets were typed as "Web-Tier-Subner-2Public" and "App-Tier-Subner-2-Private" (spelling slips). The Name tag can be corrected under **Actions > Manage tags**; this document uses the intended names.

Enable auto-assign public IP on the web subnets

1. Select `Web-Tier-Subnet-1-Public` and choose **Actions > Edit subnet settings**.

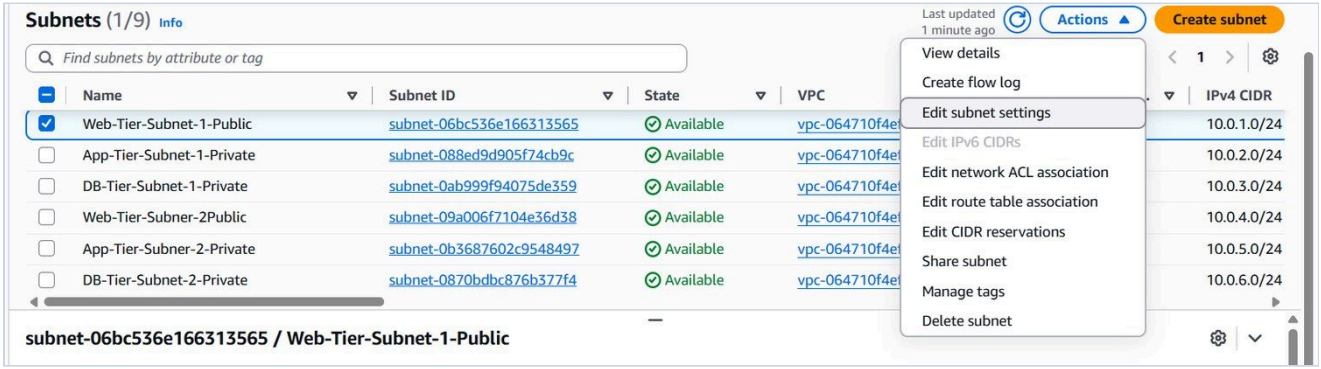


Figure 18. Actions > Edit subnet settings for Web-Tier-Subnet-1-Public.

2 Under **Auto-assign IP settings** tick **Enable auto-assign public IPv4 address** and click **Save**.

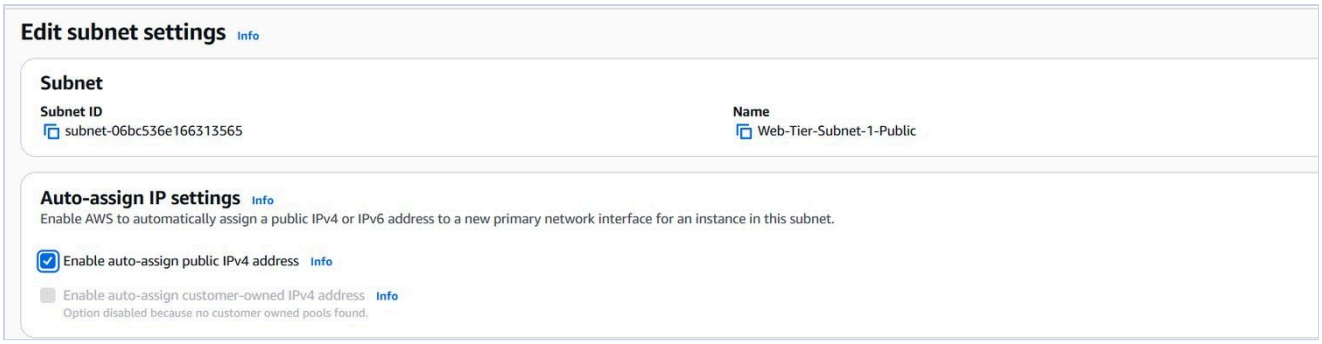


Figure 19. Auto-assign public IPv4 enabled for Web-Tier-Subnet-1-Public.

3 Repeat both actions for `Web-Tier-Subnet-2-Public`.

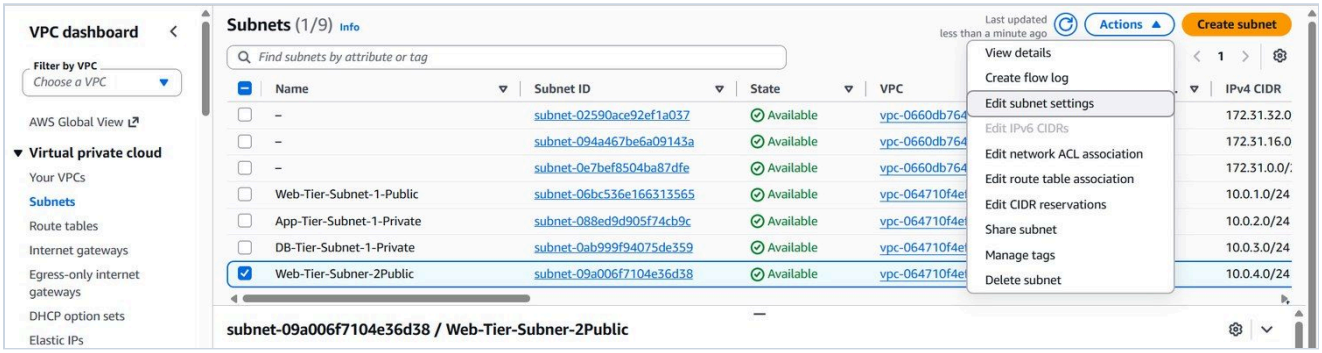


Figure 20. Edit subnet settings for the second public subnet.

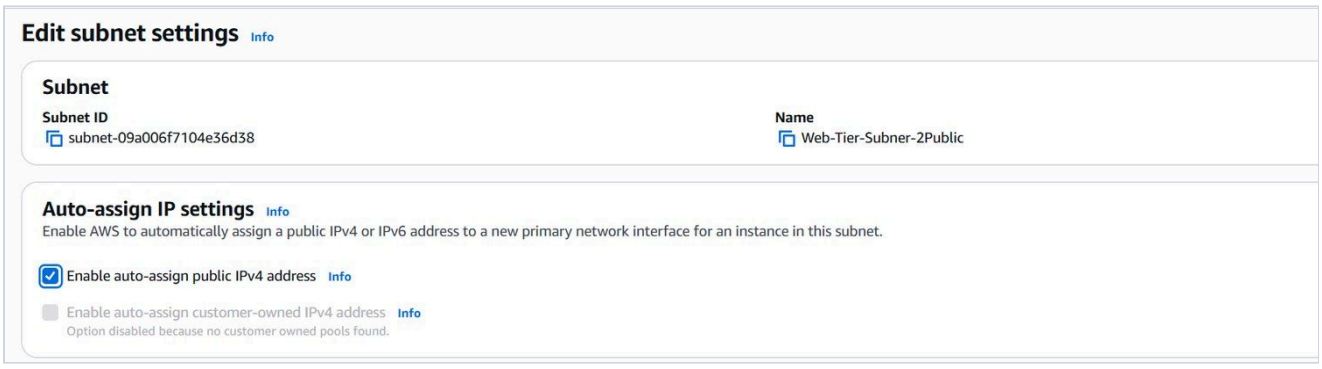


Figure 21. Auto-assign public IPv4 enabled for Web-Tier-Subnet-2-Public.

CHECKPOINT

The Subnets list shows six subnets in the Available state and both Web subnets have auto-assign public IPv4 turned on.

STEP 02 OF 15 NETWORK FOUNDATION

Create the Internet Gateway, NAT Gateways and Route Tables

Open the doors and lay the roads

WHY THIS STEP?

Subnets are only labels until a route table says where their traffic goes. The internet gateway serves the public web subnets; NAT gateways give the private app subnets outbound-only access.

Create and attach the internet gateway

1. Open **VPC > Internet gateways > Create internet gateway**. Enter the name `Internet-GW-Three-Tier-App` and click **Create internet gateway**.

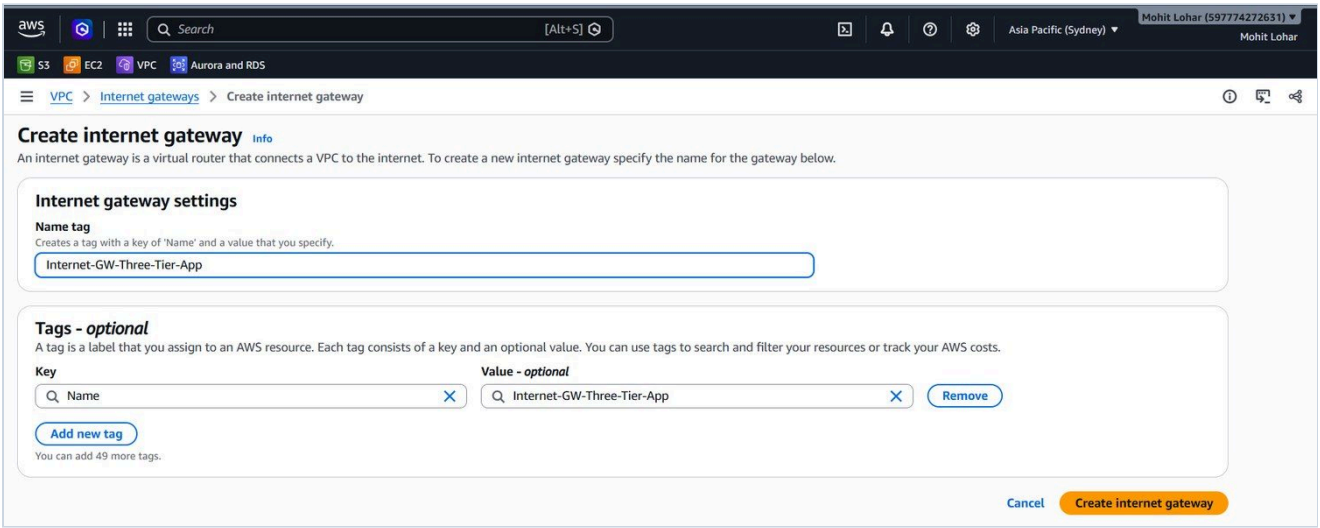


Figure 22. Creating the internet gateway.

2. The gateway starts in the **Detached** state. Choose **Actions > Attach to VPC**.

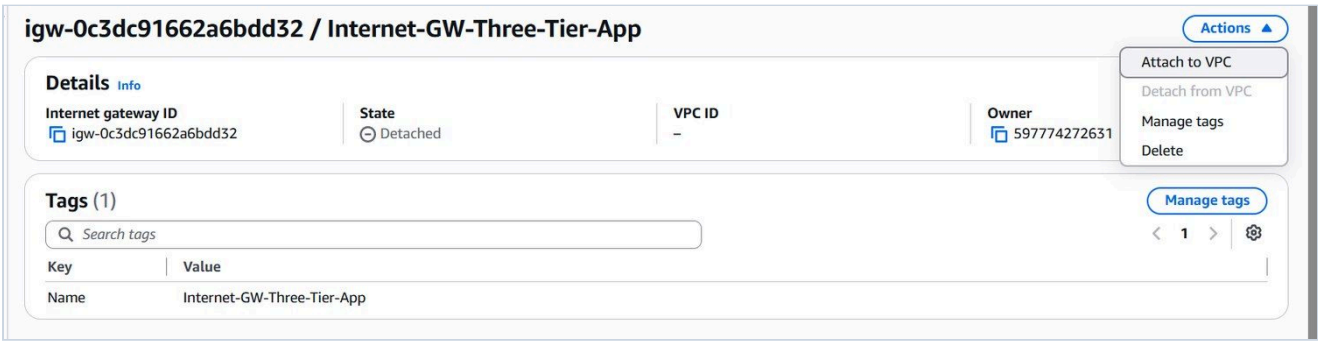


Figure 23. Internet gateway in the Detached state: Actions > Attach to VPC.

3 Select `Three-Tier-Project-VPC` and click **Attach internet gateway**.

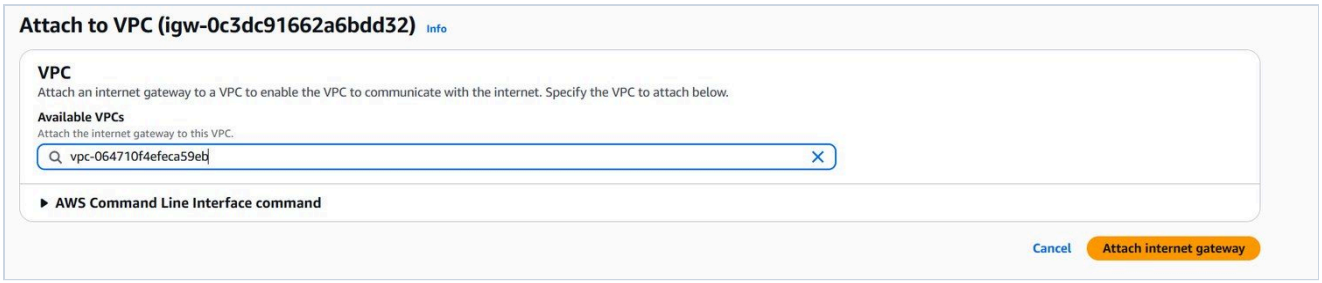


Figure 24. Attaching the internet gateway to Three-Tier-Project-VPC.

Create the NAT gateways

One NAT gateway is created in each public subnet so the private app servers can download packages without being reachable from the internet. Open **VPC > NAT gateways > Create NAT gateway**.

SETTING	NAT-GW-1	NAT-GW-2
Name	NAT-GW-1	NAT-GW-2
Availability mode	Zonal	Zonal
Subnet	Web-Tier-Subnet-1-Public	Web-Tier-Subnet-2-Public
Connectivity type	Public	Public
Elastic IP	Allocate Elastic IP	Allocate Elastic IP

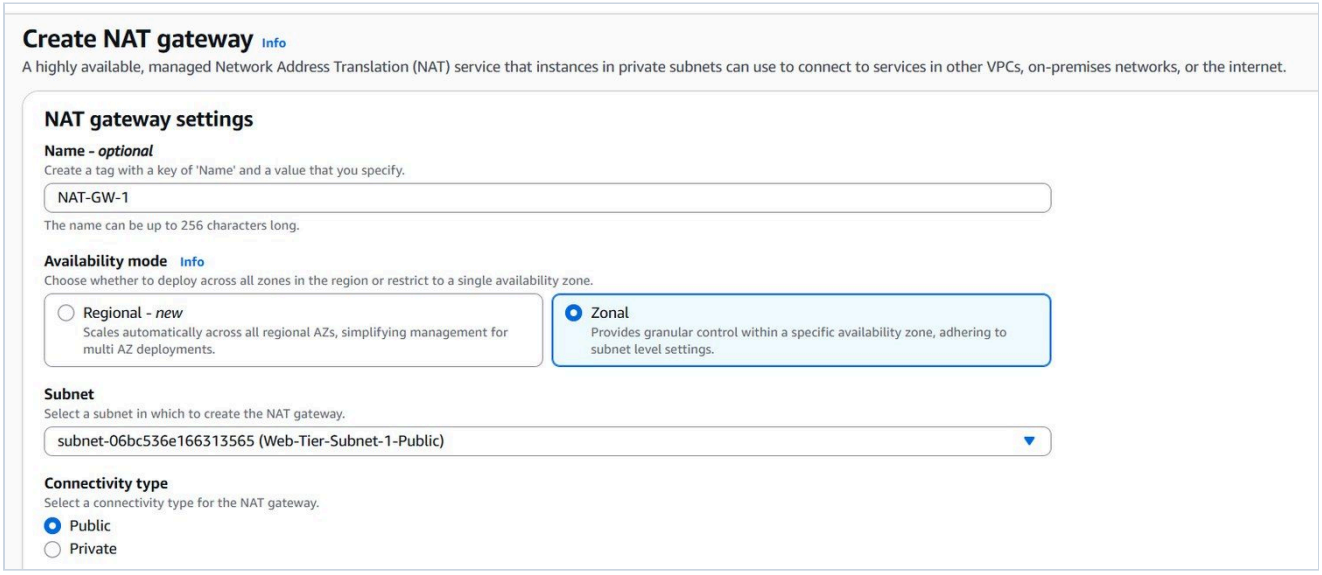


Figure 25. NAT gateway settings for NAT-GW-1.

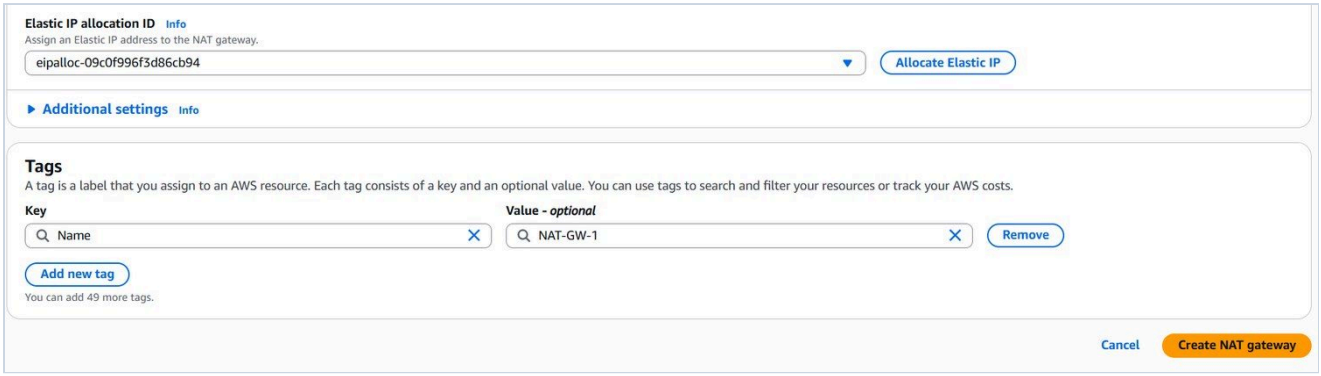


Figure 26. Allocate an Elastic IP, then click Create NAT gateway (NAT-GW-1).

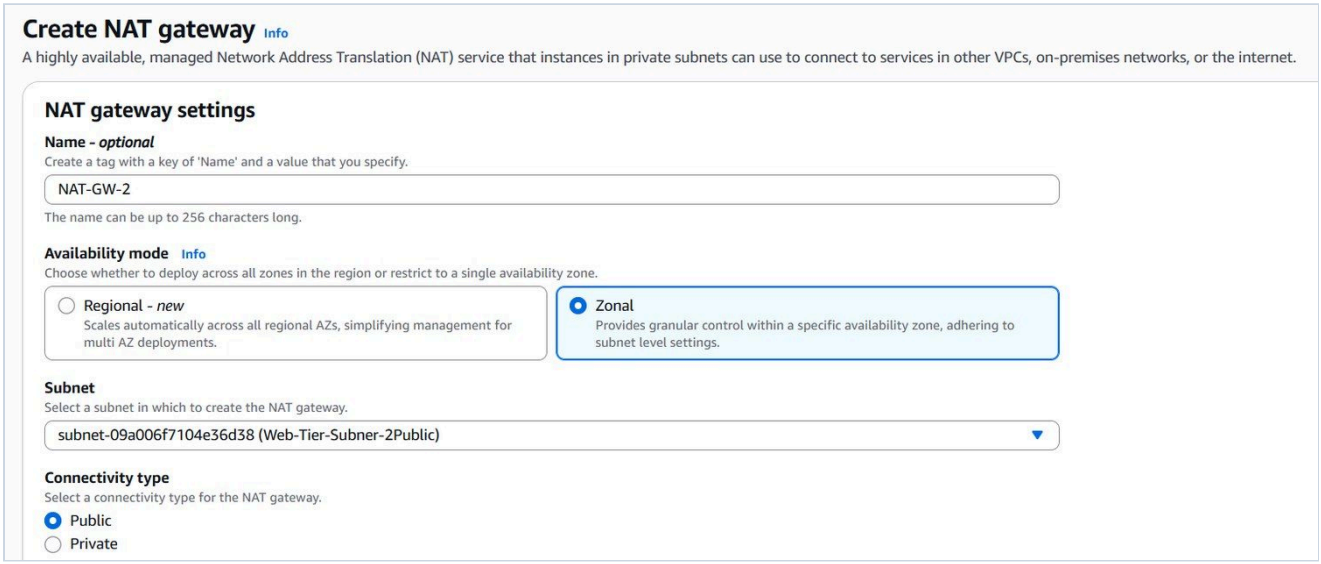


Figure 27. NAT gateway settings for NAT-GW-2.

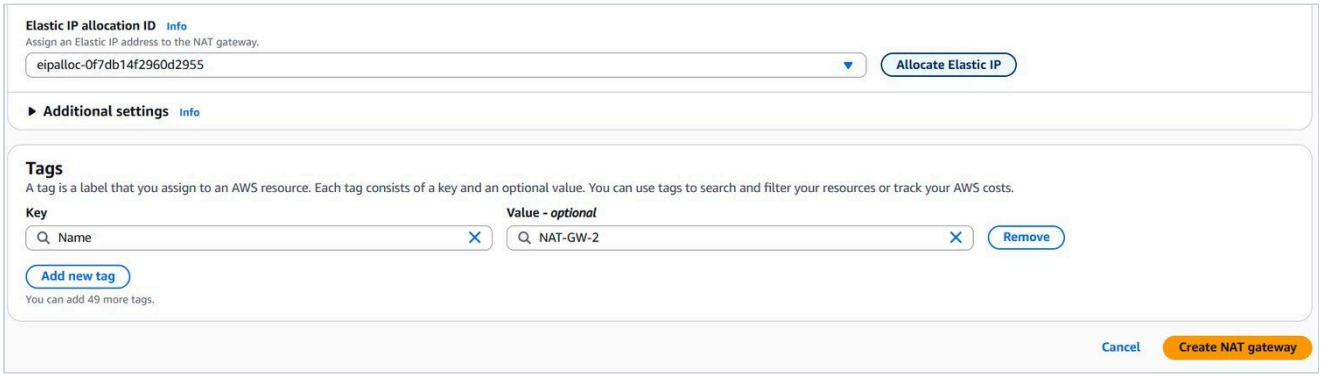


Figure 28. Allocate an Elastic IP, then click Create NAT gateway (NAT-GW-2).

IMPORTANT

NAT gateways and their Elastic IPs bill every hour they exist. Delete them in Step 15 as soon as you are done.

Create the route tables

Three route tables are needed: one for the web tier and one for each app subnet. Go to **VPC > Route tables > Create route table**, choose `Three-Tier-Project-VPC` and create:

- `App-Tier-RT-1`
- `App-Tier-RT-2`
- The web-tier route table (created as `Web-Tier-RT-2` in this deployment)

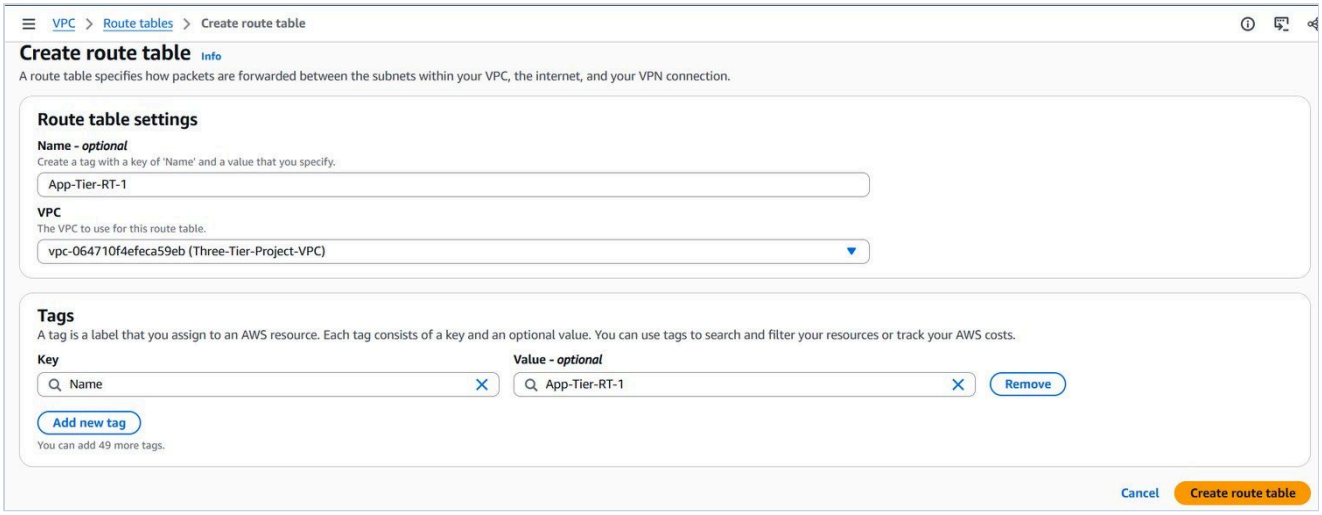


Figure 29. Creating route table App-Tier-RT-1.

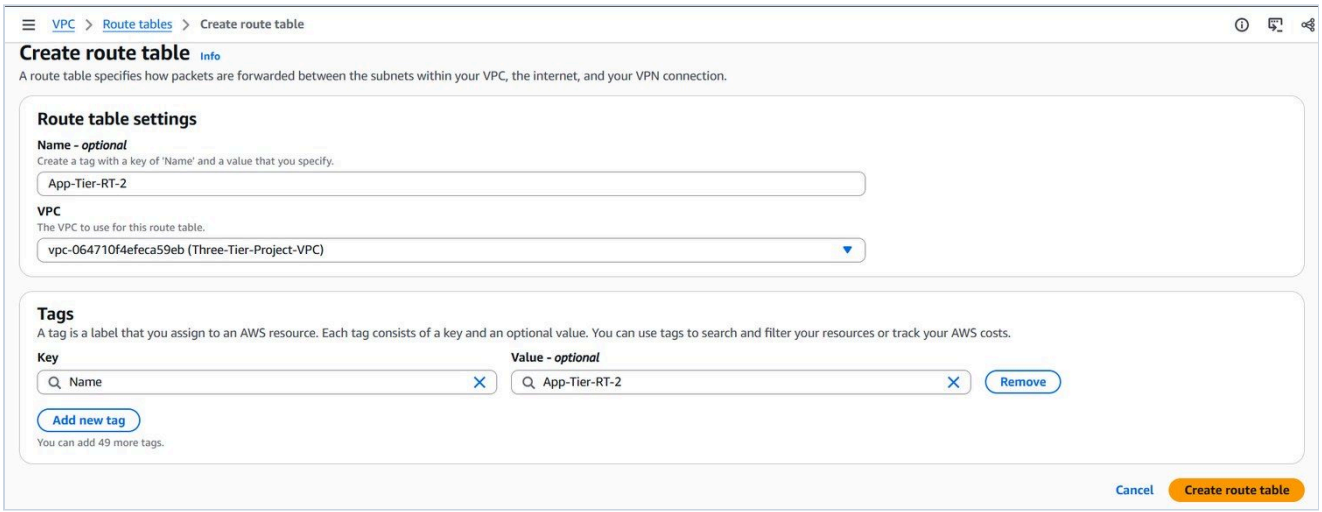


Figure 30. Creating route table App-Tier-RT-2.

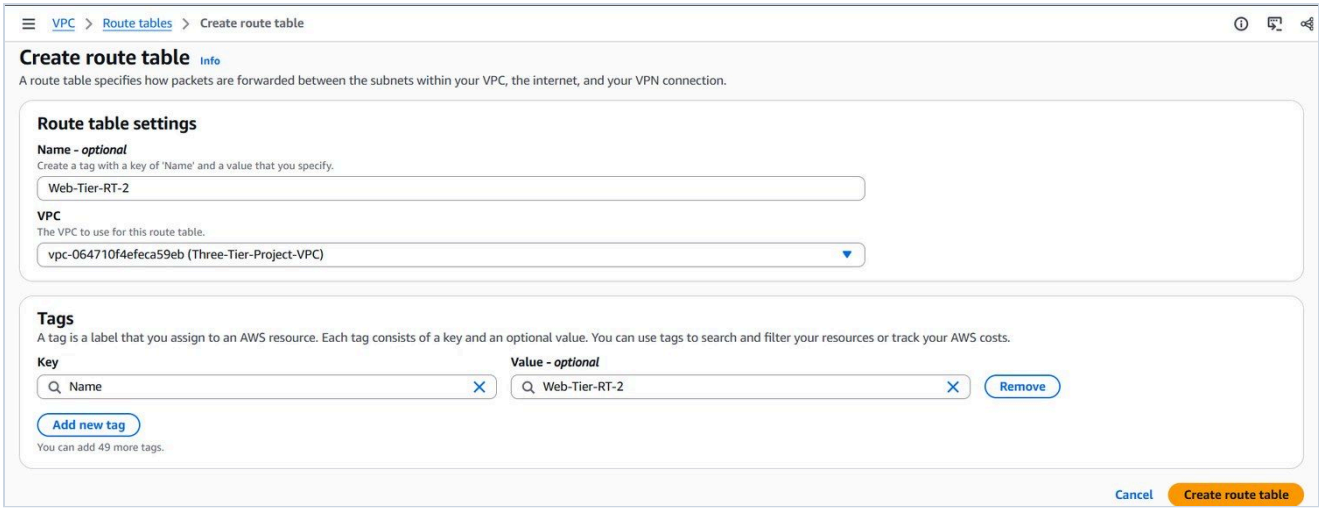


Figure 31. Creating the web-tier route table.

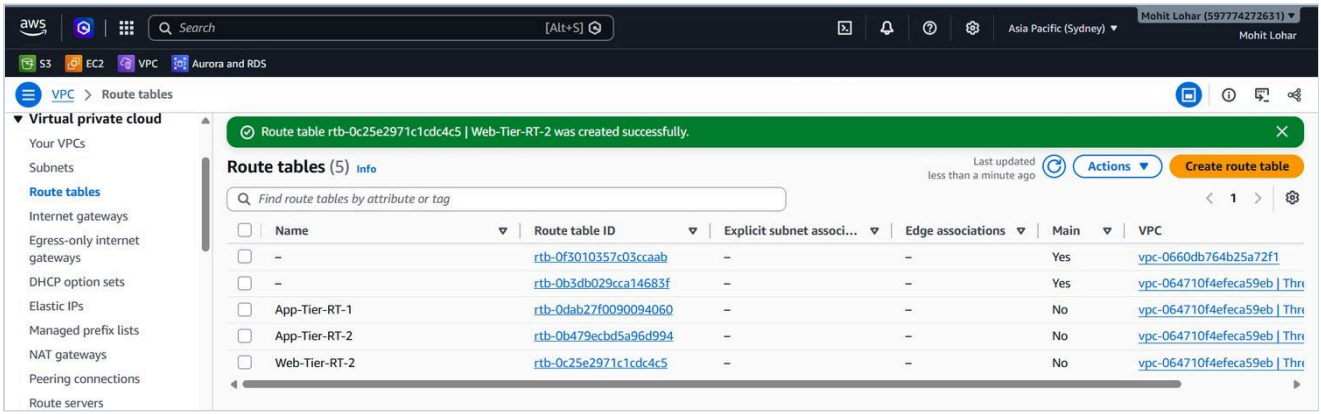


Figure 32. The Route tables list after creation.

Edit routes and subnet associations

ROUTE TABLE	ROUTE (0.0.0.0/0 TARGET)	ASSOCIATED SUBNET(S)
Web-tier route table	Internet-GW-Three-Tier-App	Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public
App-Tier-RT-1	NAT-GW-1	App-Tier-Subnet-1-Private
App-Tier-RT-2	NAT-GW-2	App-Tier-Subnet-2-Private

Web-tier route table to the internet gateway

1. Select the web-tier route table and choose **Actions > Edit routes**.

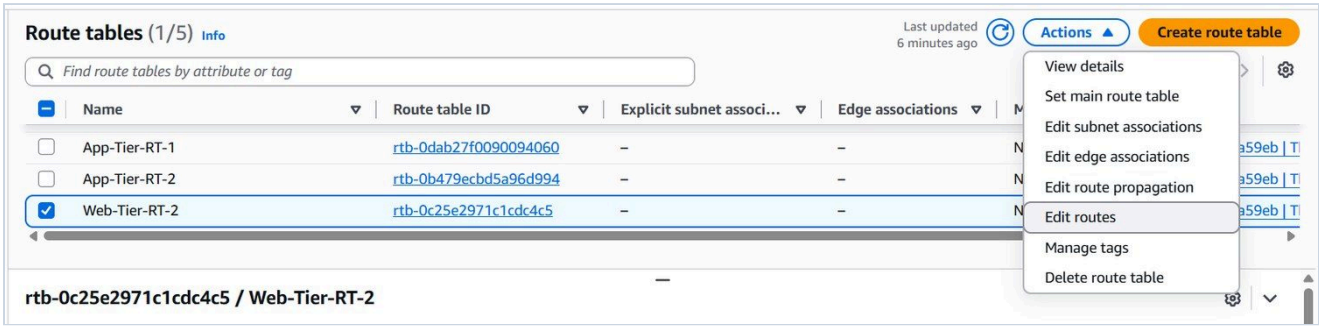


Figure 33. Actions > Edit routes for the web-tier route table.

2. Click **Add route**, destination `0.0.0.0/0`, target **Internet Gateway > Internet-GW-Three-Tier-App**. Click **Save changes**.

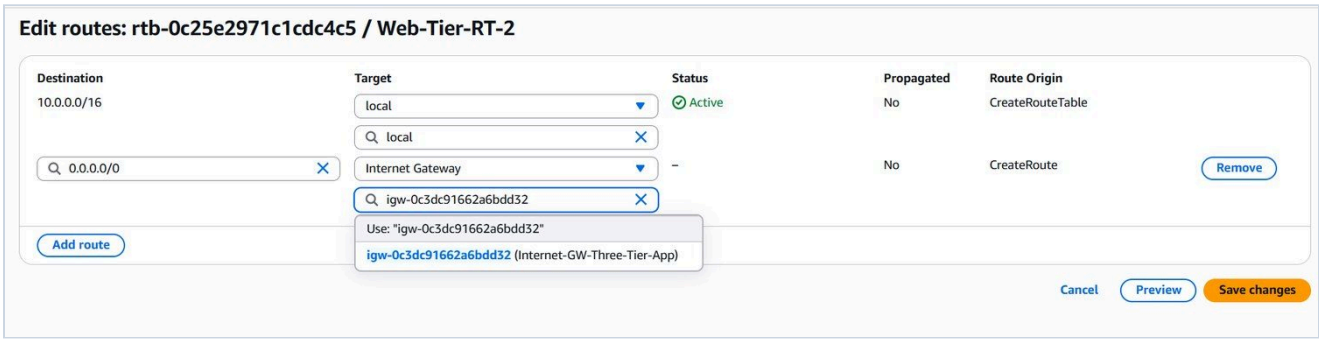


Figure 34. Default route 0.0.0.0/0 to the internet gateway.

3. Select the same route table again and choose **Actions > Edit subnet associations**.

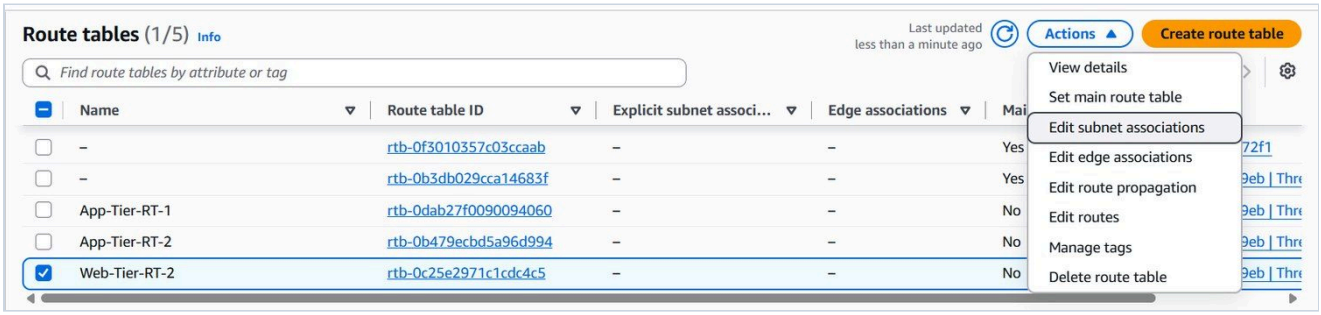


Figure 35. Actions > Edit subnet associations.

4 Tick `Web-Tier-Subnet-1-Public` and `Web-Tier-Subnet-2-Public` and click **Save associations**.

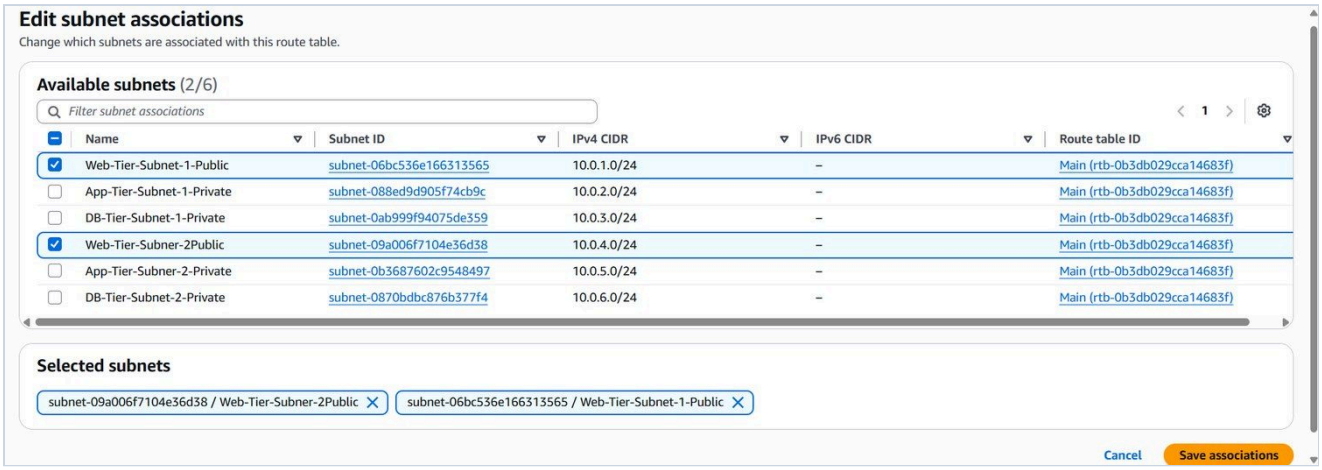


Figure 36. Associating both public web subnets.

App-Tier-RT-1 to NAT-GW-1

1 Select `App-Tier-RT-1` and choose **Actions > Edit routes**.

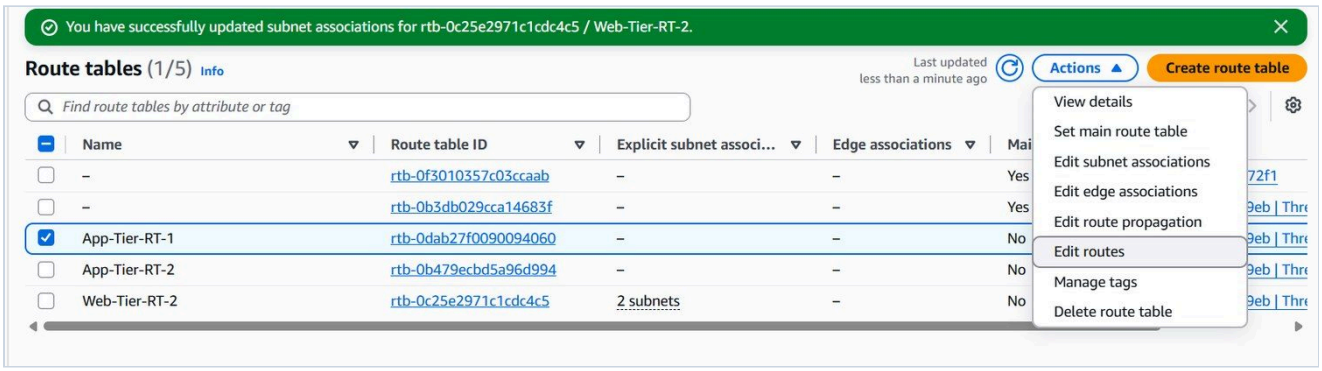


Figure 37. Success banner for the web route table; Actions > Edit routes on App-Tier-RT-1.

2 Add route `0.0.0.0/0`, target **NAT Gateway > NAT-GW-1**, then **Save changes**.

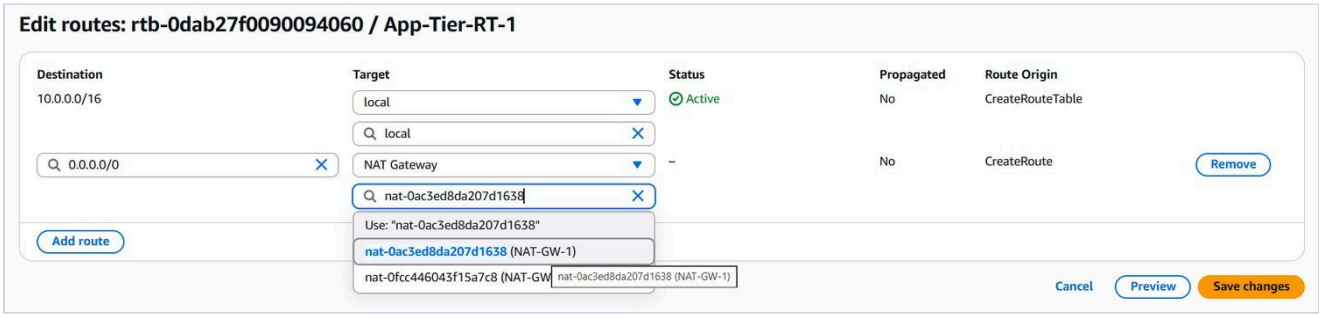


Figure 38. Default route 0.0.0.0/0 to NAT-GW-1.

3. Choose **Actions > Edit subnet associations**, tick `App-Tier-Subnet-1-Private` and click **Save associations**.

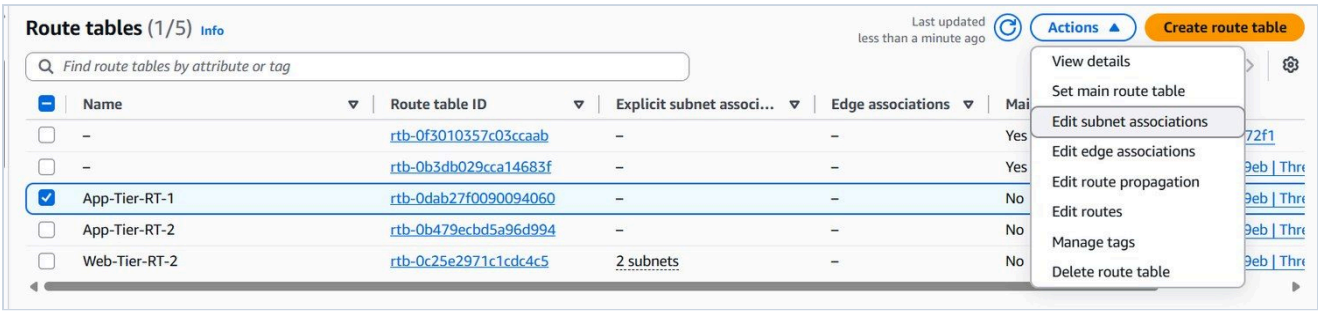


Figure 39. Edit subnet associations for App-Tier-RT-1.

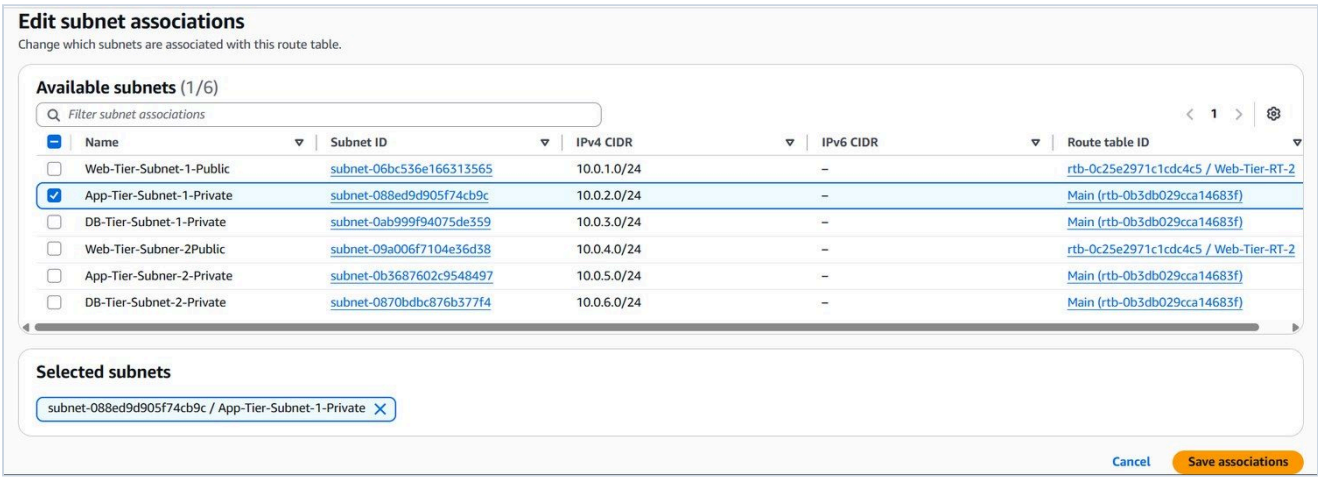


Figure 40. Associating App-Tier-Subnet-1-Private.

App-Tier-RT-2 to NAT-GW-2

1. Select `App-Tier-RT-2` and choose **Actions > Edit routes**.

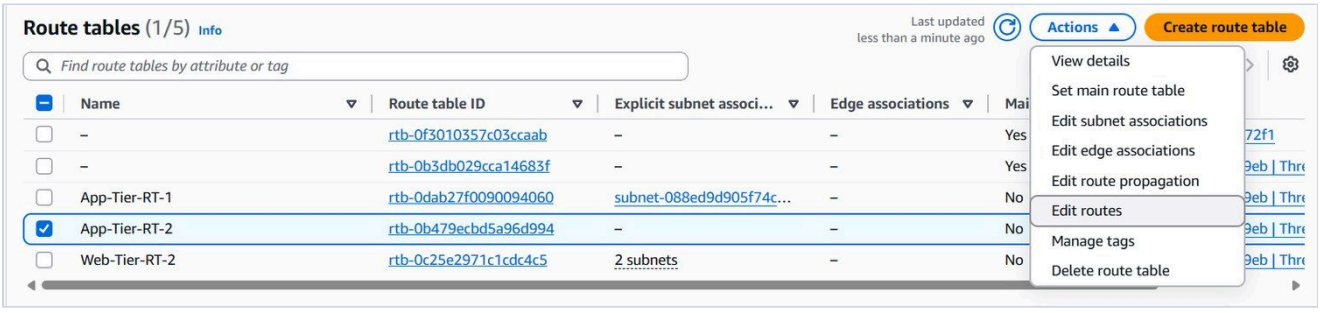


Figure 41. Actions > Edit routes on App-Tier-RT-2.

2 Add route `0.0.0.0/0`, target **NAT Gateway > NAT-GW-2**, then **Save changes**.

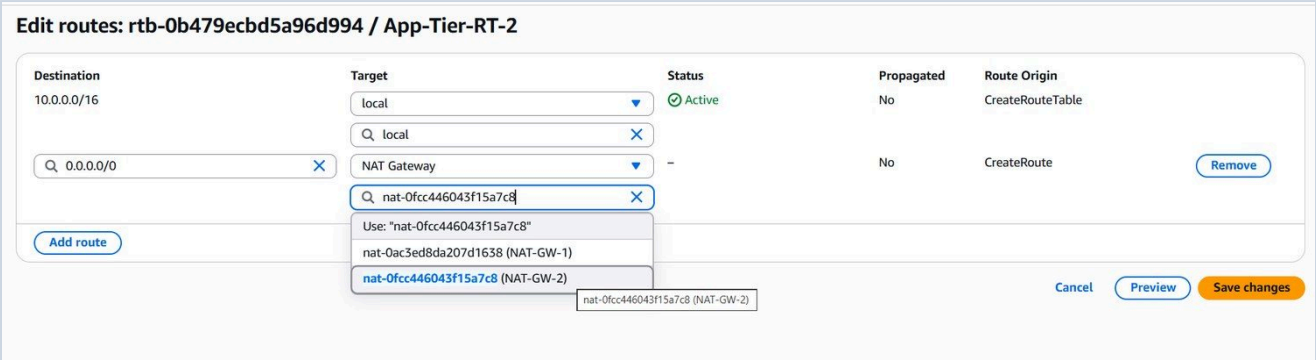


Figure 42. Default route 0.0.0.0/0 to NAT-GW-2.

3 Choose **Actions > Edit subnet associations**, tick `App-Tier-Subnet-2-Private` and click **Save associations**.

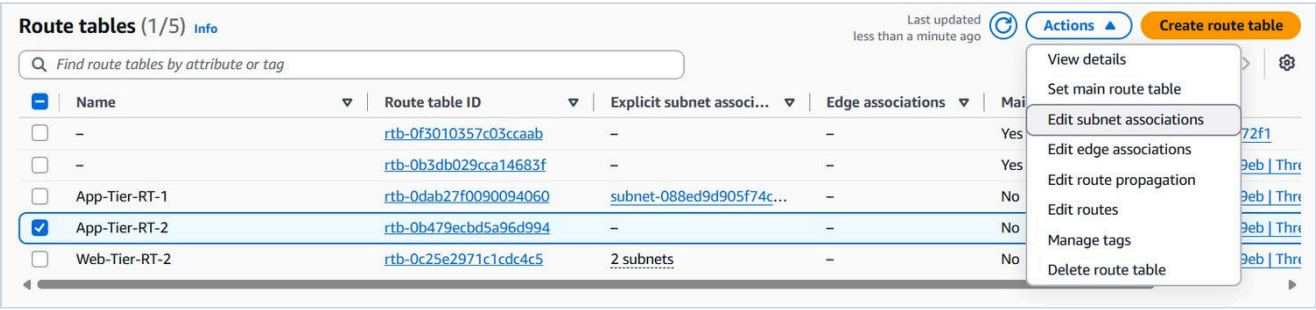


Figure 43. Edit subnet associations for App-Tier-RT-2.

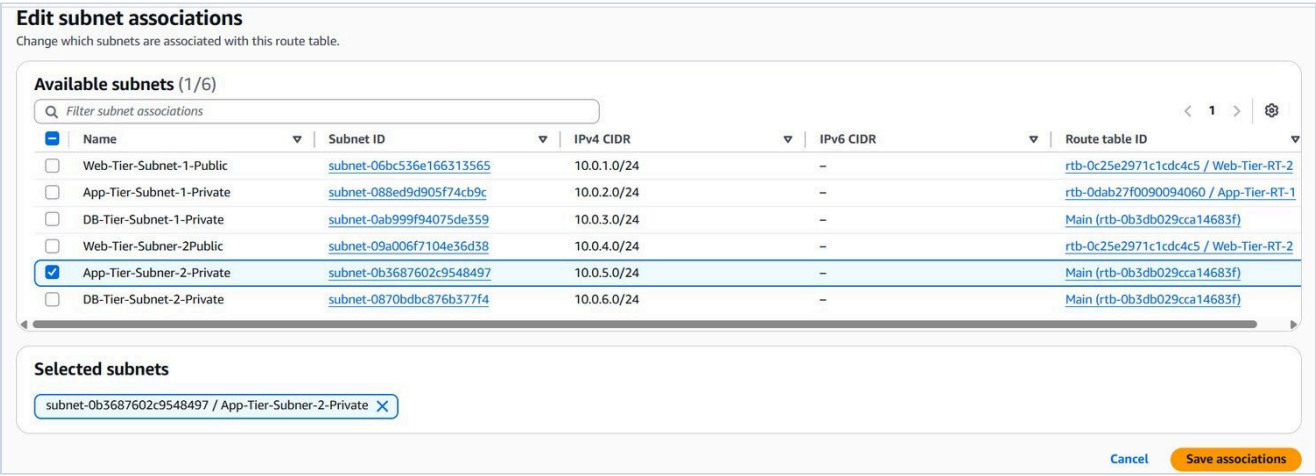


Figure 44. Associating App-Tier-Subnet-2-Private.

Verify with the resource map

Open **Your VPCs > Three-Tier-Project-VPC > Resource map** and hover over each element to confirm that every subnet is connected to the intended route table, the internet gateway or a NAT gateway.

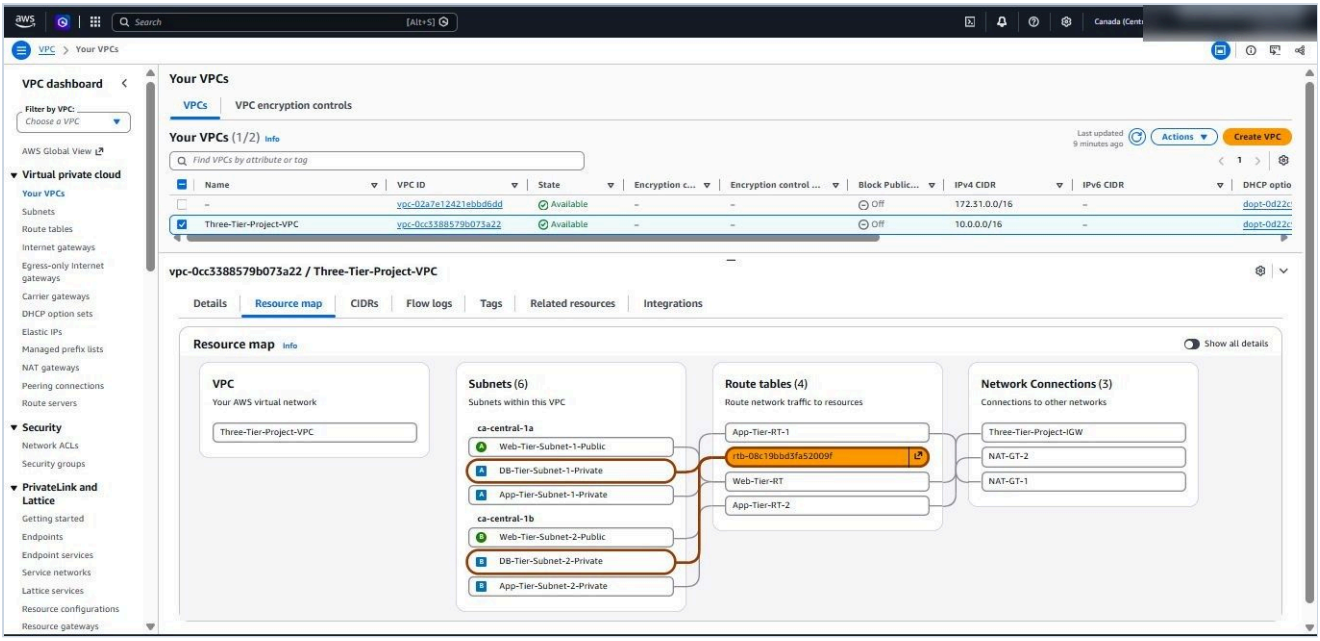


Figure 45. Reference screenshot from the source guide (Canada region, resource names differ slightly): the VPC resource map with subnets, route tables and network connections. Your own map should look the same.

CHECKPOINT

Public subnets route to the internet gateway, each private app subnet routes to its own NAT gateway, and the resource map matches the architecture diagram.

STEP 03 OF 15 NETWORK FOUNDATION

Create the Security Groups

Build a firewall chain, tier by tier

WHY THIS STEP?

Security groups are the firewalls between tiers. By letting each group accept traffic only from the group in front of it, the database can only ever be reached through the app tier.

Create five security groups in **VPC > Security groups > Create security group**. Choose `Three-Tier-Project-VPC` for each one and add a single inbound rule from the table below. Use the same text for the name and description.

SECURITY GROUP	TYPE (PORT)	SOURCE
External-Load-Balancer-SG	HTTP (80)	Anywhere-IPv4 0.0.0.0/0
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL/Aurora (3306)	App-Tier-SG

External-Load-Balancer-SG

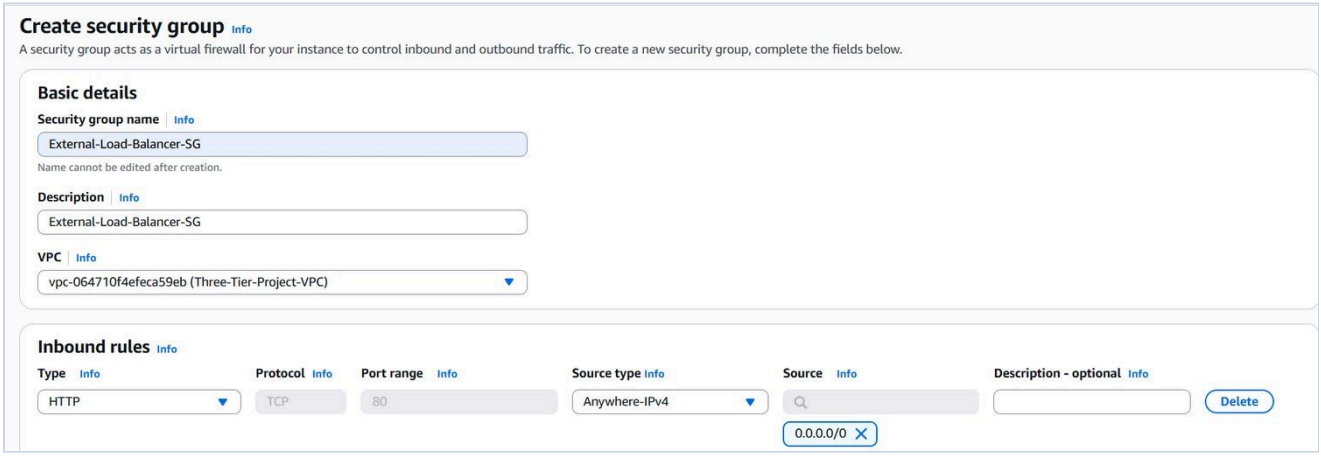


Figure 46. External-Load-Balancer-SG: HTTP 80 from anywhere.

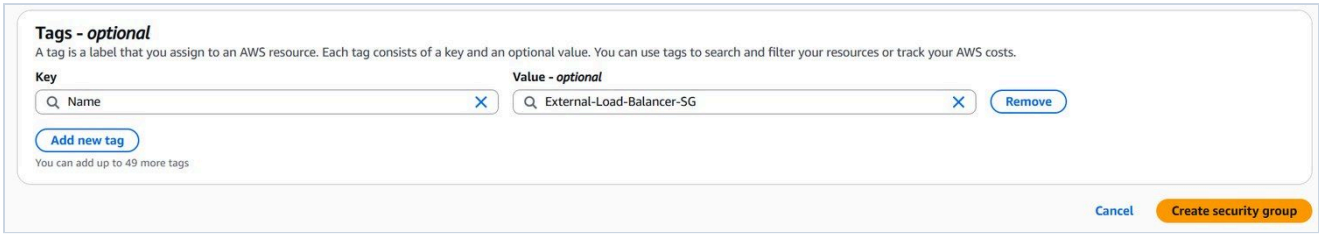


Figure 47. Name tag for External-Load-Balancer-SG, then click Create security group.

Web-Tier-SG

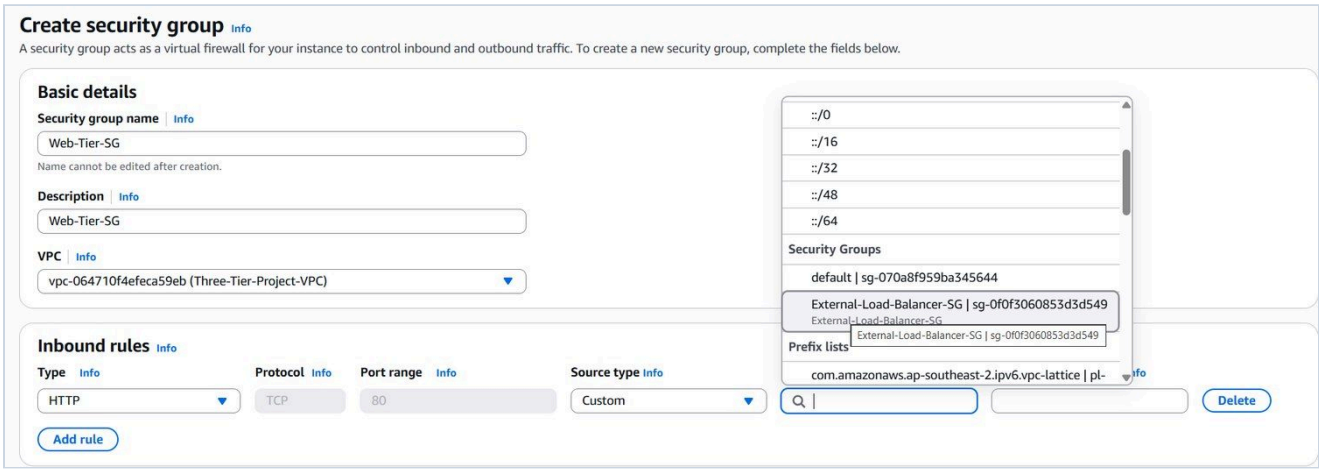


Figure 48. Web-Tier-SG: source is External-Load-Balancer-SG.

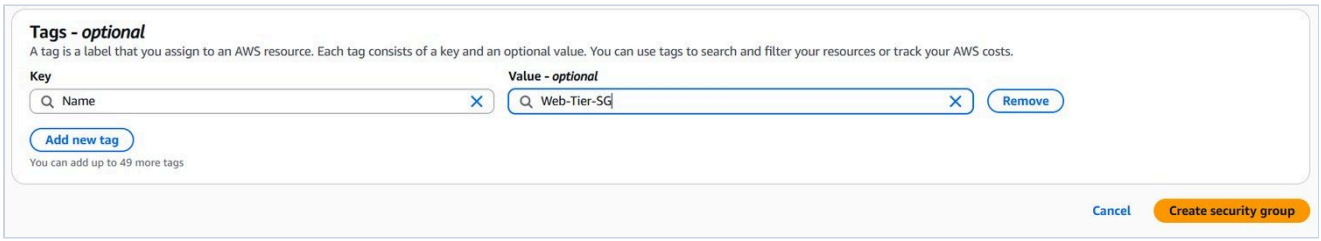


Figure 49. Name tag for Web-Tier-SG, then click Create security group.

Internal-Load-Balancer-SG

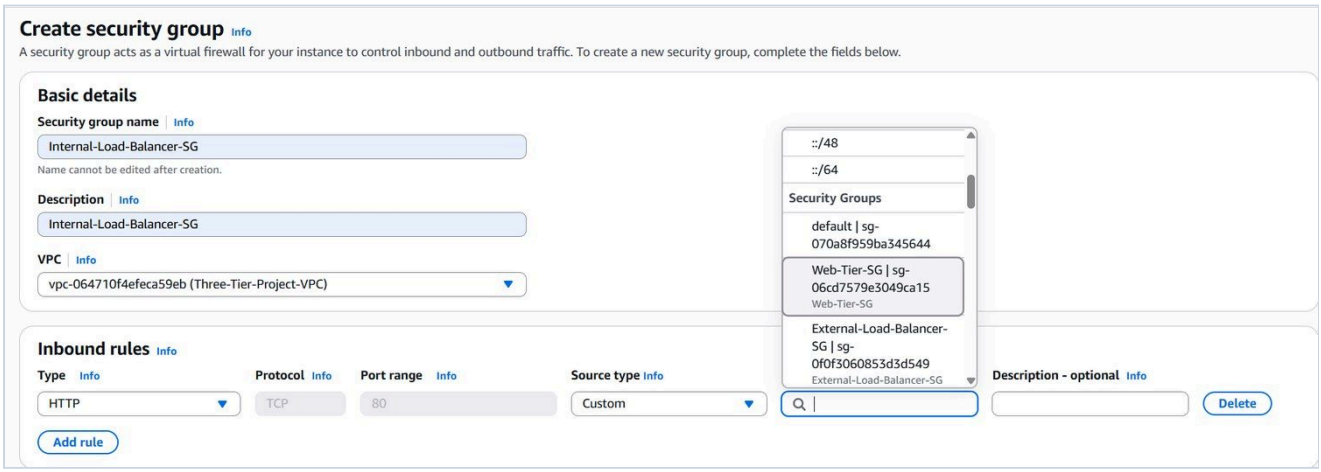


Figure 50. Internal-Load-Balancer-SG: source is Web-Tier-SG.

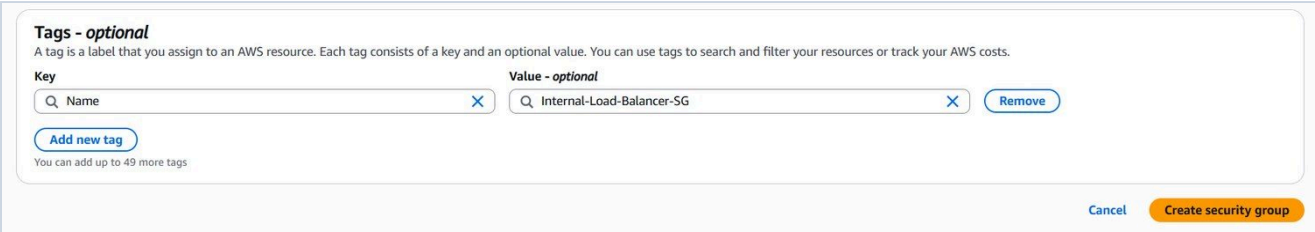


Figure 51. Name tag for Internal-Load-Balancer-SG, then click Create security group.

App-Tier-SG

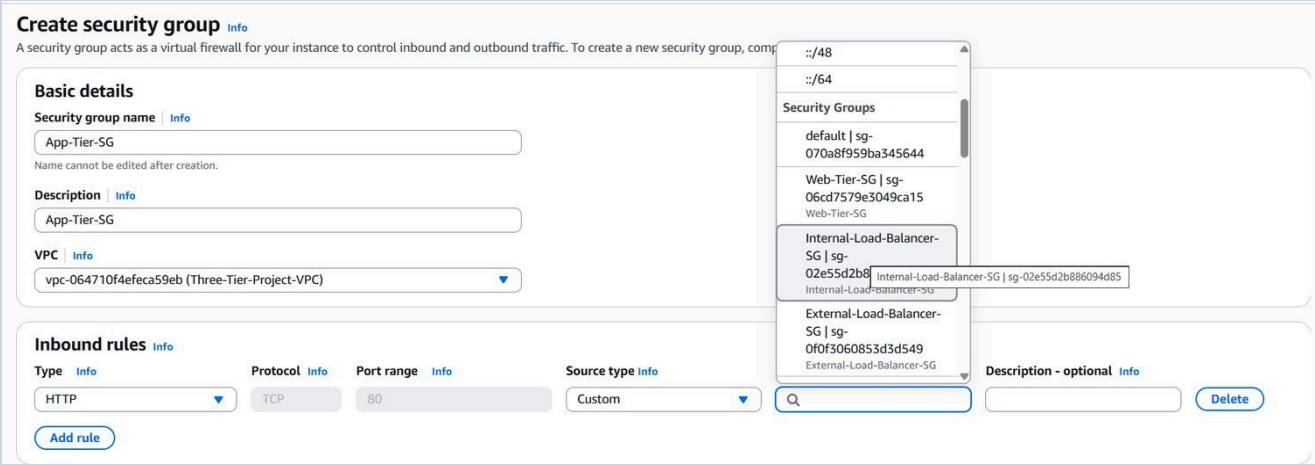


Figure 52. App-Tier-SG: source is Internal-Load-Balancer-SG; set the type to Custom TCP 4000.

IMPORTANT

The Node.js backend listens on port 4000 and the app-tier health check uses that port, so the inbound rule type must be **Custom TCP 4000**. The captured form still shows the default type HTTP (80) at the moment of the screenshot.

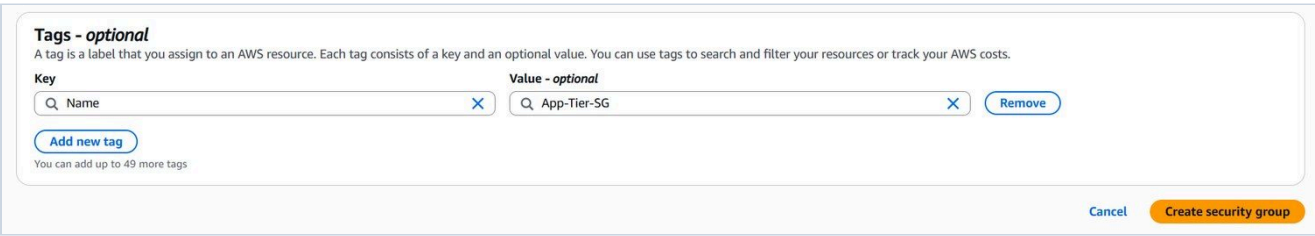


Figure 53. Name tag for App-Tier-SG, then click Create security group.

DB-Tier-SG

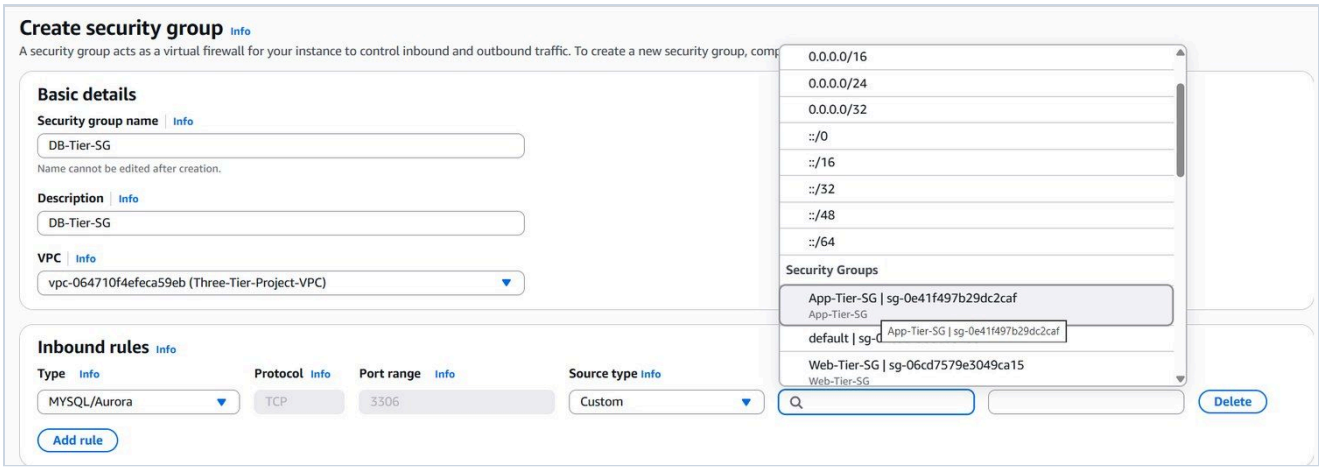


Figure 54. DB-Tier-SG: MYSQL/Aurora 3306 from App-Tier-SG.

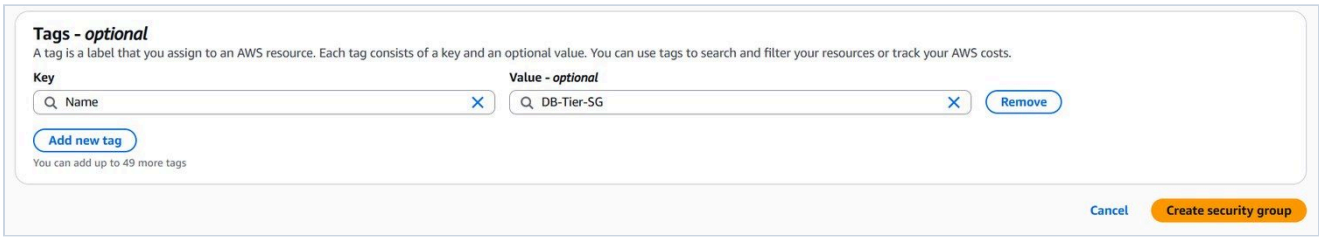


Figure 55. Name tag for DB-Tier-SG, then click Create security group.

After the last group is created, the Security groups list shows all five custom groups together with the VPC default group.

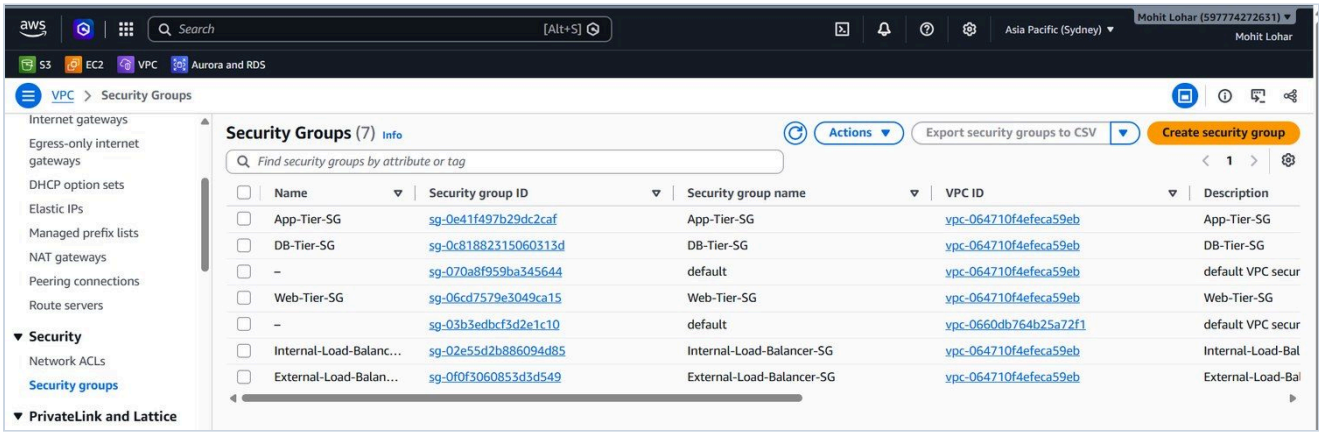


Figure 56. The Security groups list.

CHECKPOINT

Five new security groups exist, each pointing to the group in front of it. DB-Tier-SG allows 3306 only from App-Tier-SG.

STEP 04 OF 15 DATA AND ACCESS

Create the DB Subnet Group and the RDS Database

Give the data a protected home

WHY THIS STEP?

The database holds the only copy of your data, so it goes in private subnets that only the app tier can reach. RDS needs a DB subnet group to know which subnets it may use.

Create the DB subnet group

A DB subnet group tells RDS which private subnets it may use. Its subnets must sit in different Availability Zones so the database can be placed, and replicated, across zones.

1. Search for **RDS** and open **Aurora and RDS**.

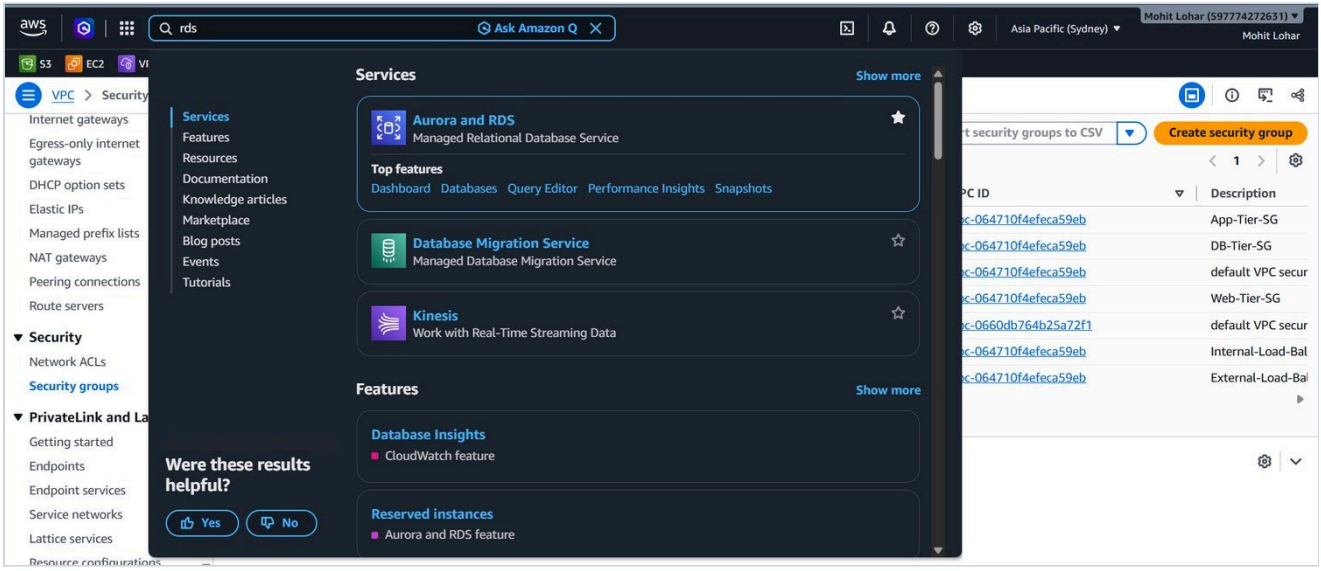


Figure 57. Opening the RDS service.

2. Choose **Subnet groups > Create DB subnet group**.

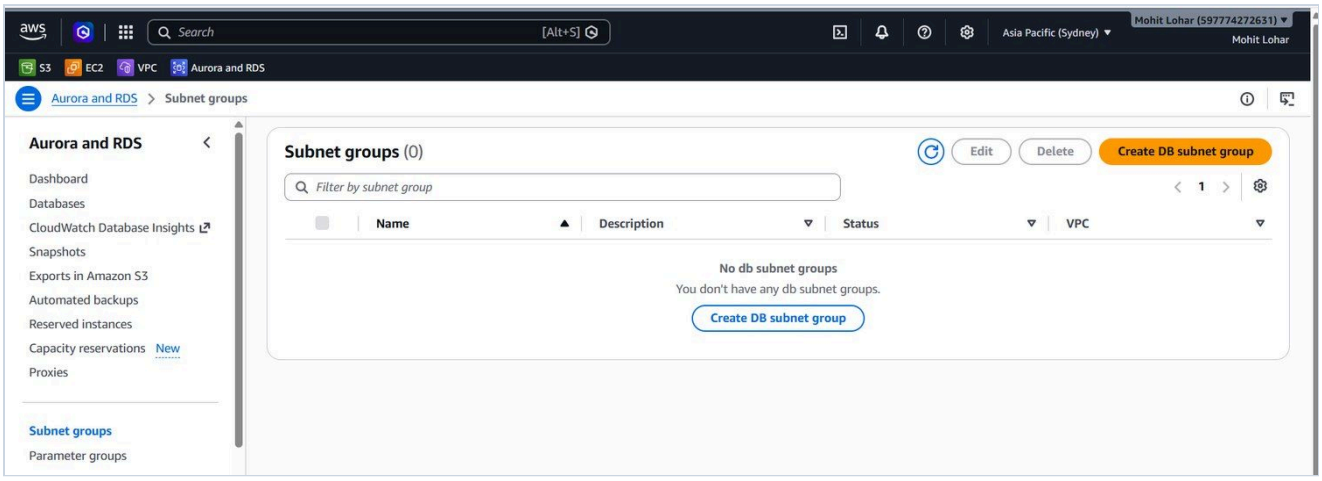


Figure 58. The Subnet groups page (empty).

3. Enter the name `DB-Subnet-Group-Three-Tier-Application`, the same text as description, and choose `Three-Tier-Project-VPC`.

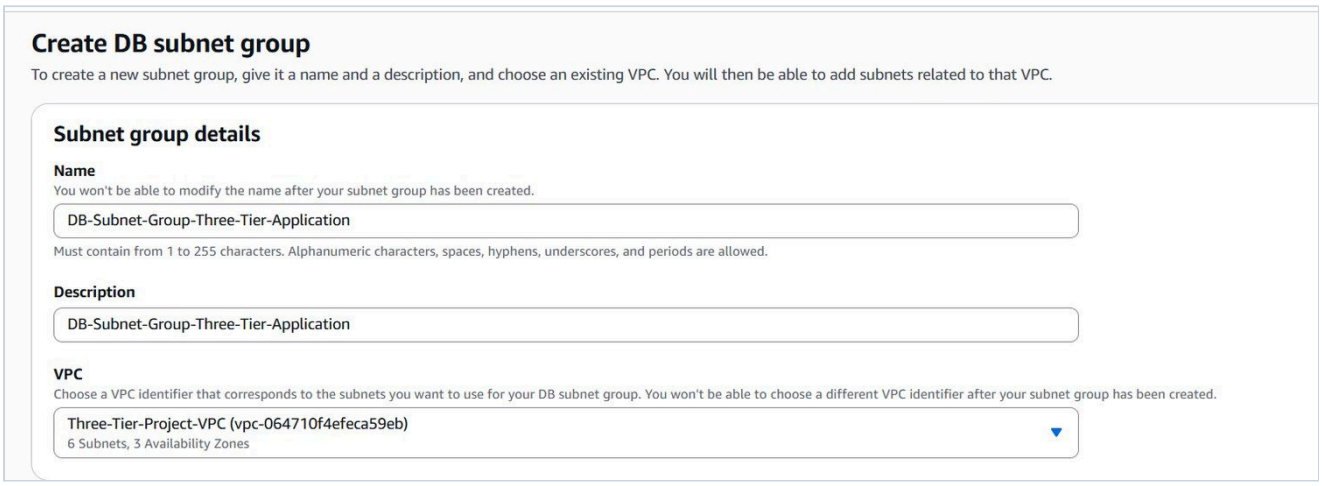


Figure 59. DB subnet group details.

4. Select the Availability Zones `ap-southeast-2b` and `ap-southeast-2c`, then the subnets `DB-Tier-Subnet-2-Private` (10.0.6.0/24) and `DB-Tier-Subnet-1-Private` (10.0.3.0/24).

5. Review the subnet summary and click **Create**.

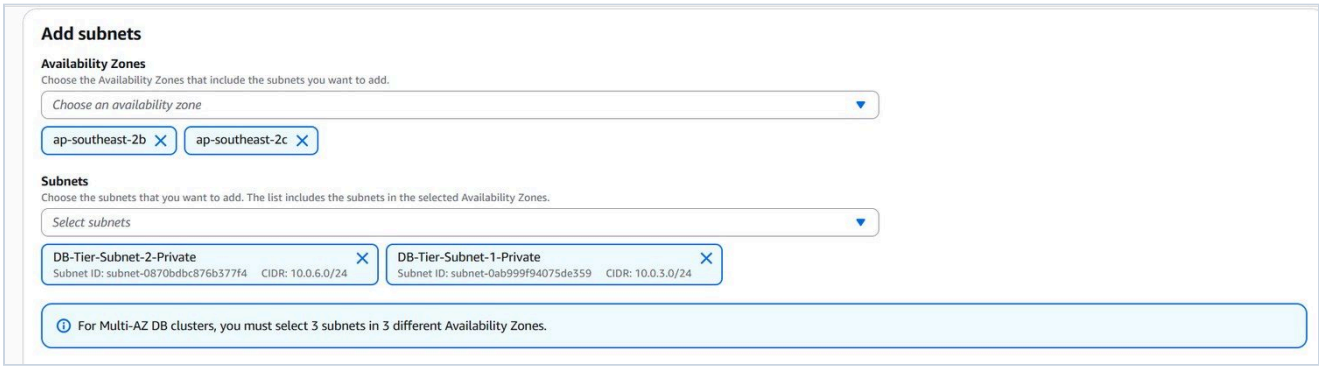


Figure 60. Selecting the Availability Zones and the two database subnets.

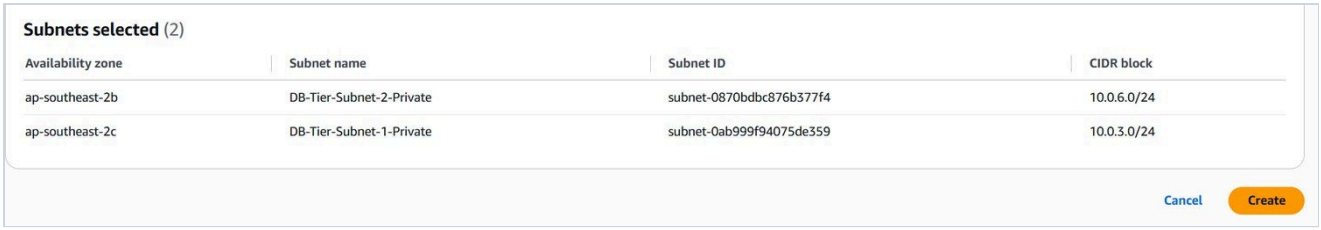


Figure 61. Subnets included in the DB subnet group, then Create.

Create the database

1. Go to **Databases > Create database**. Use the arrow beside the button and choose **Full configuration**.

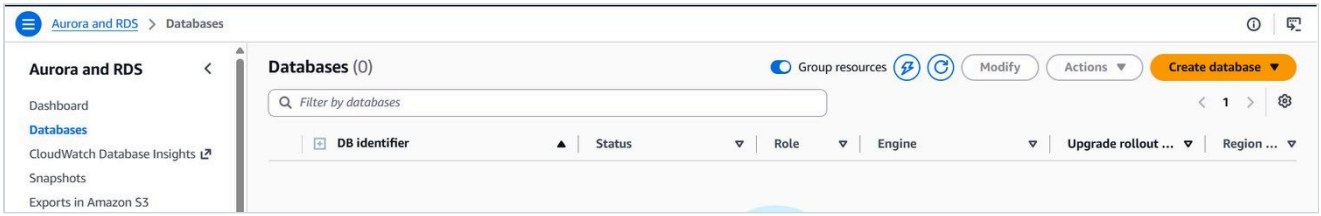


Figure 62. The RDS Databases page (empty).

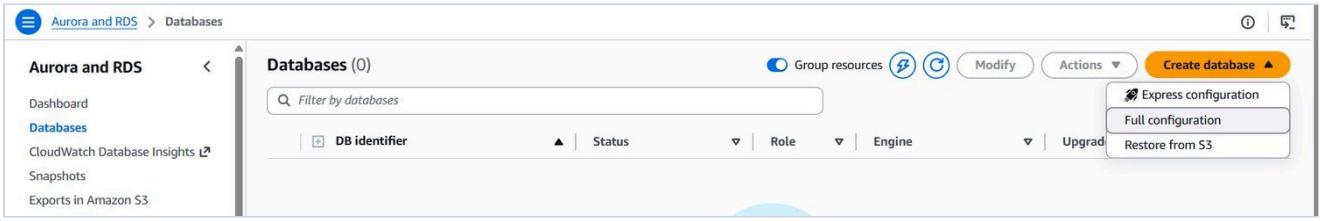


Figure 63. Create database > Full configuration.

SETTING	VALUE
Engine type	MySQL
Creation method / template	Full configuration / Free tier (use Production for a real workload)
Availability	Single-AZ DB instance deployment (1 instance)
DB instance identifier	`database-1`
Master username	`admin`
Credentials management	Self managed (enter and confirm a strong password)
Compute connection	Don’t connect to an EC2 compute resource
VPC / DB subnet group	Three-Tier-Project-VPC / DB-Subnet-Group-Three-Tier-Application
VPC security group	Choose existing: `DB-Tier-SG`
Public access	**No**

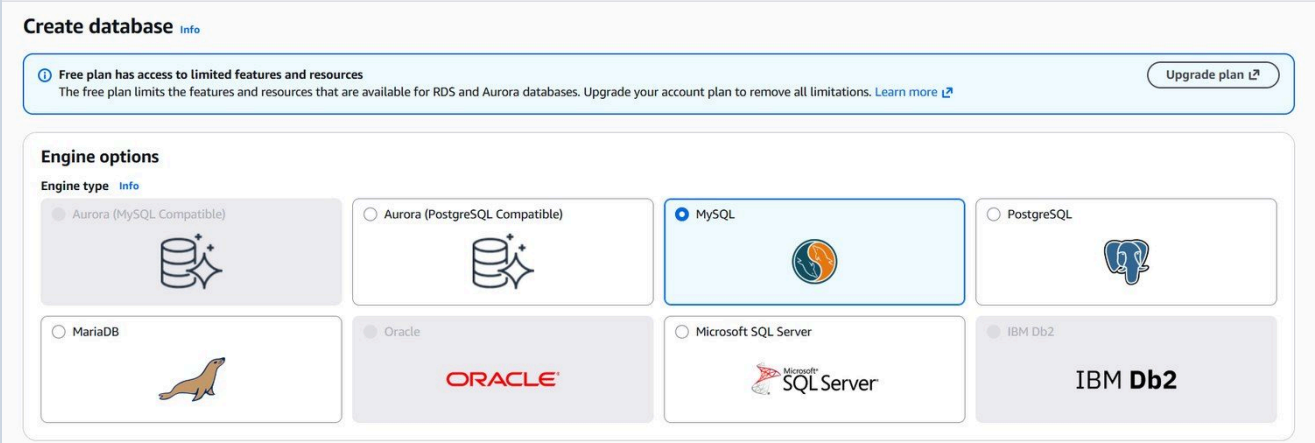


Figure 64. Engine option: MySQL.

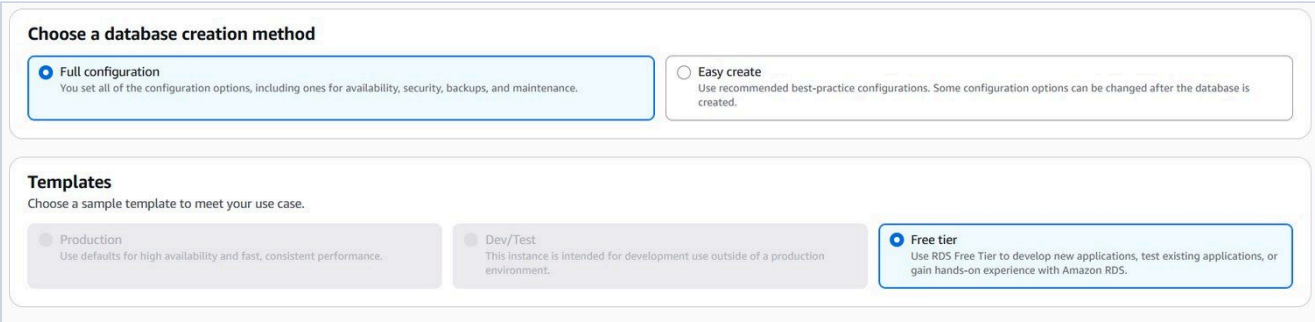


Figure 65. Creation method and template.

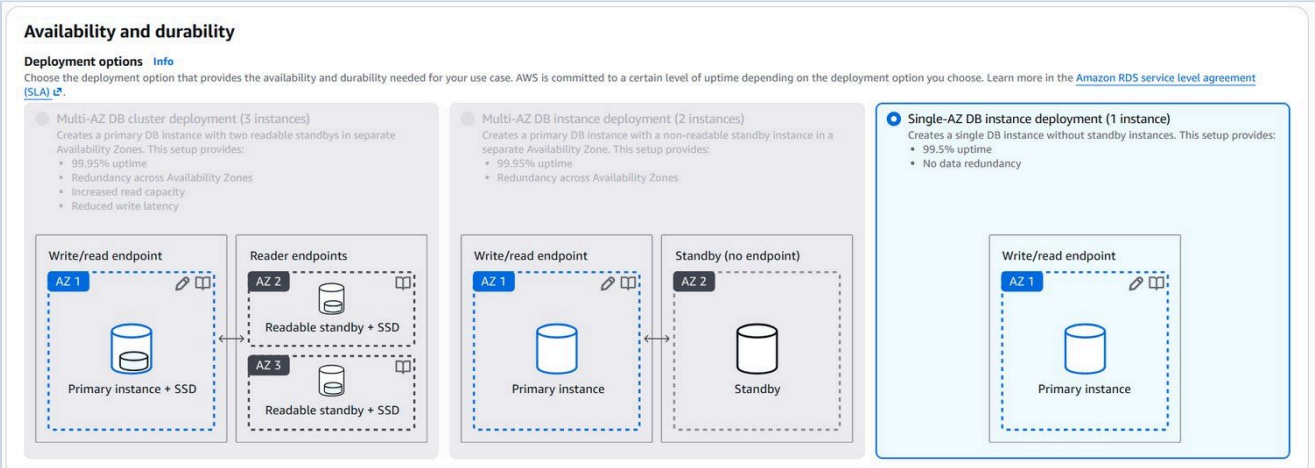


Figure 66. Availability and durability: Single-AZ DB instance.

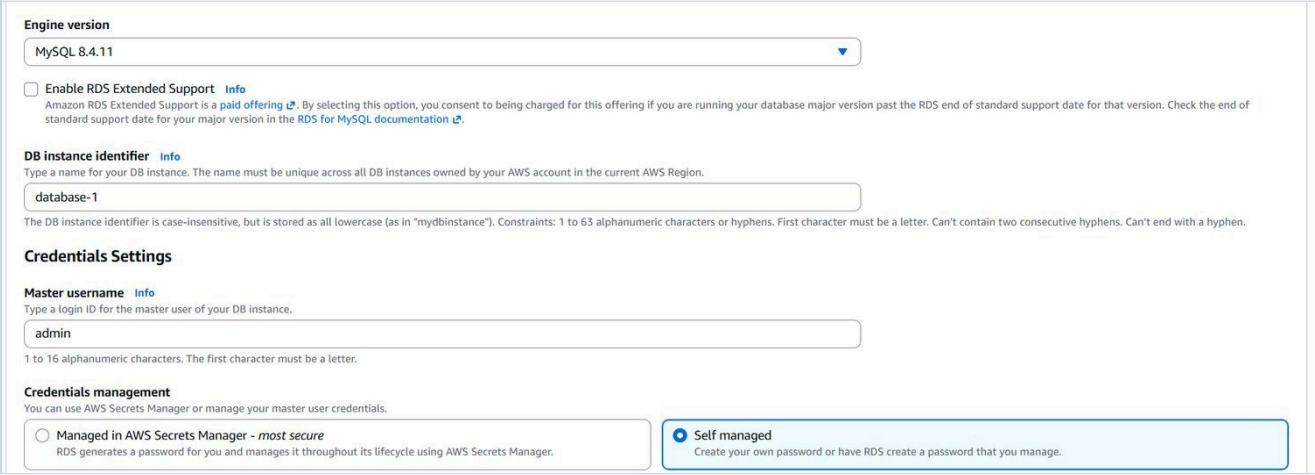


Figure 67. DB instance identifier and credentials settings.



Figure 68. Master password and confirmation (value masked).

IMPORTANT

Remember the master username and password. You need them in Step 7 to connect to the database and fill in `DbConfig.js`.

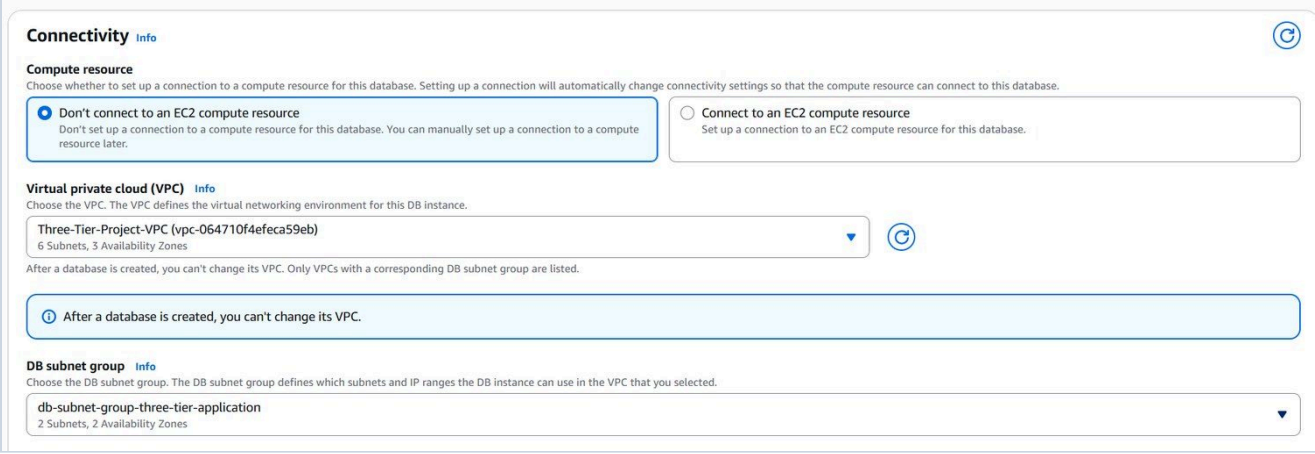


Figure 69. Connectivity: VPC and DB subnet group.

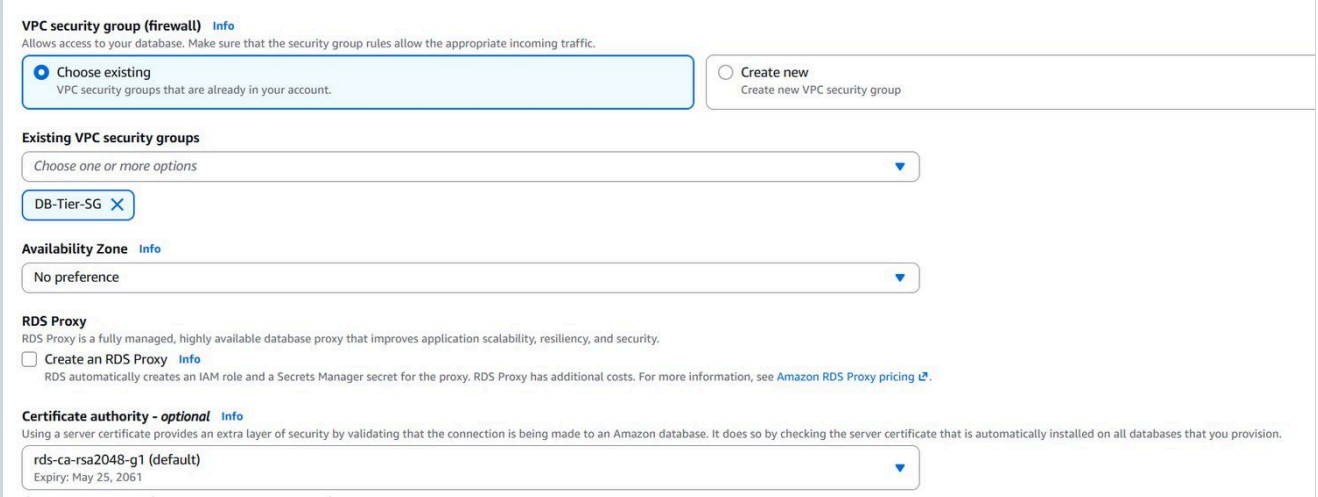


Figure 70. Existing VPC security group DB-Tier-SG.

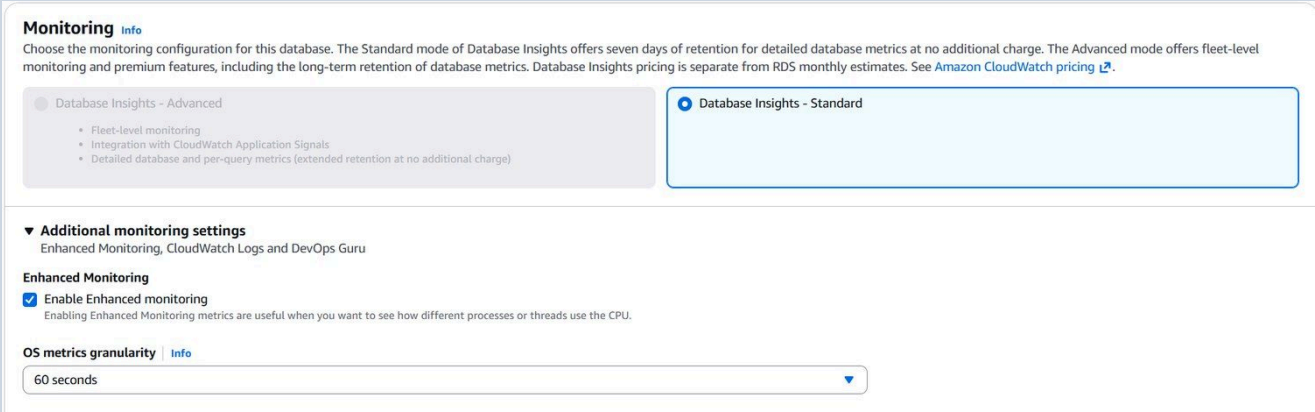


Figure 71. Monitoring settings (Database Insights Standard, Enhanced Monitoring).

2 Review *Additional configuration* and click **Create database**. Creation takes several minutes.

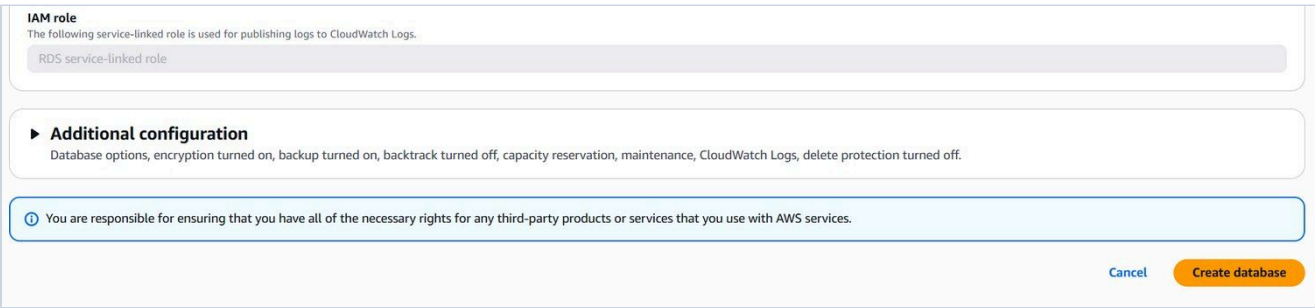


Figure 72. Additional configuration and the Create database button.

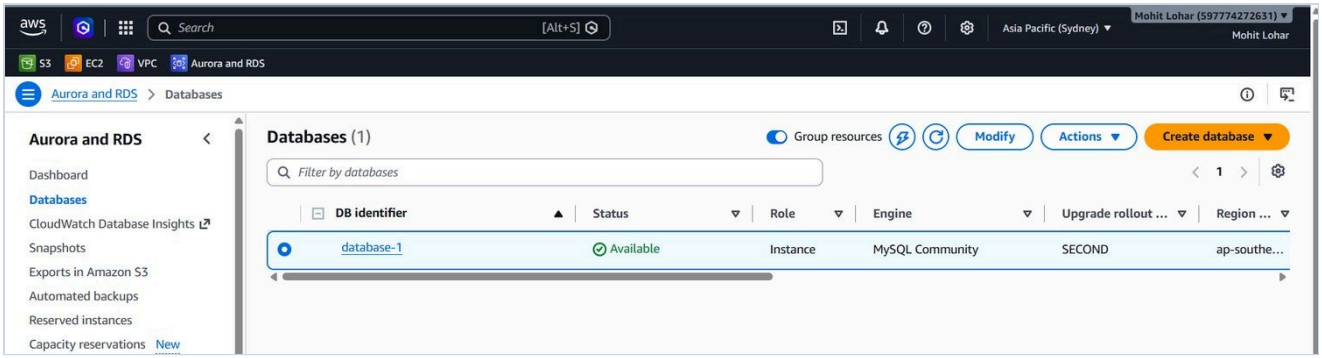


Figure 73. The database database-1 reaches the Available state.

NOTE

The captured deployment used RDS for MySQL (Free tier, Single-AZ, db.t4g.micro) to keep the cost low. For production, use Multi-AZ or Amazon Aurora (MySQL-compatible) so a standby exists in a second Availability Zone.

CHECKPOINT

The database status is Available, public access is No and the security group is DB-Tier-SG.

STEP 05 OF 15

DATA AND ACCESS

Create the IAM Role

Permissions without passwords

WHY THIS STEP?

Servers need to read the code from S3 and be reachable through Session Manager. An IAM role gives them exactly that, so no keys or passwords are stored on the machines.

The role carries two managed policies: `AmazonS3ReadOnlyAccess` (download the code from S3) and `AmazonSSMManagedInstanceCore` (let Session Manager connect without SSH or a public IP).

1. Search for **IAM** and open it.

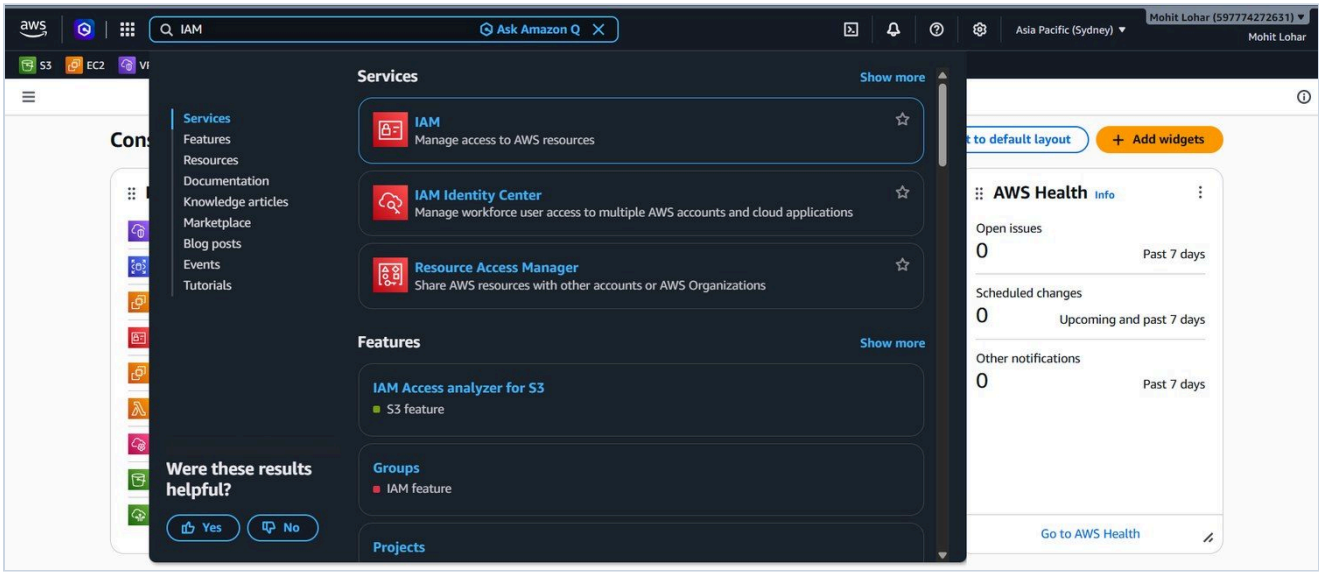


Figure 74. Opening the IAM service.

2. Go to **Roles > Create role**.

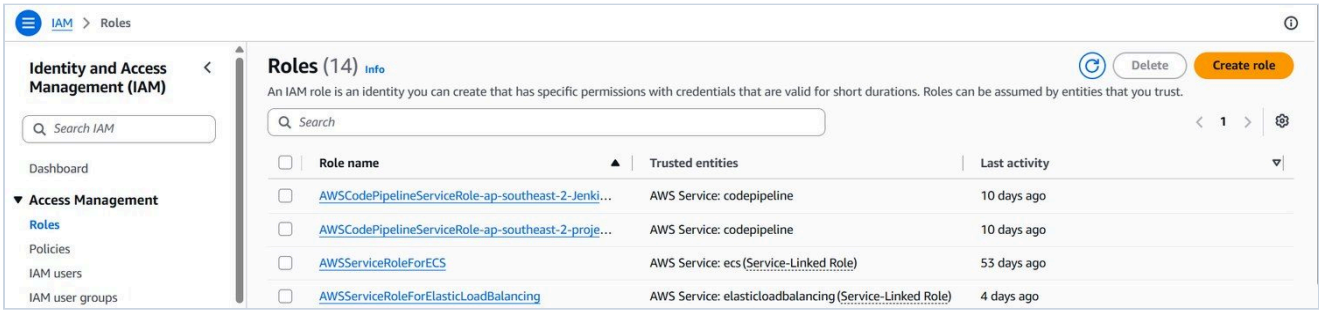


Figure 75. The IAM roles list.

3 Trusted entity type **AWS service**, use case **EC2**. Click **Next**.

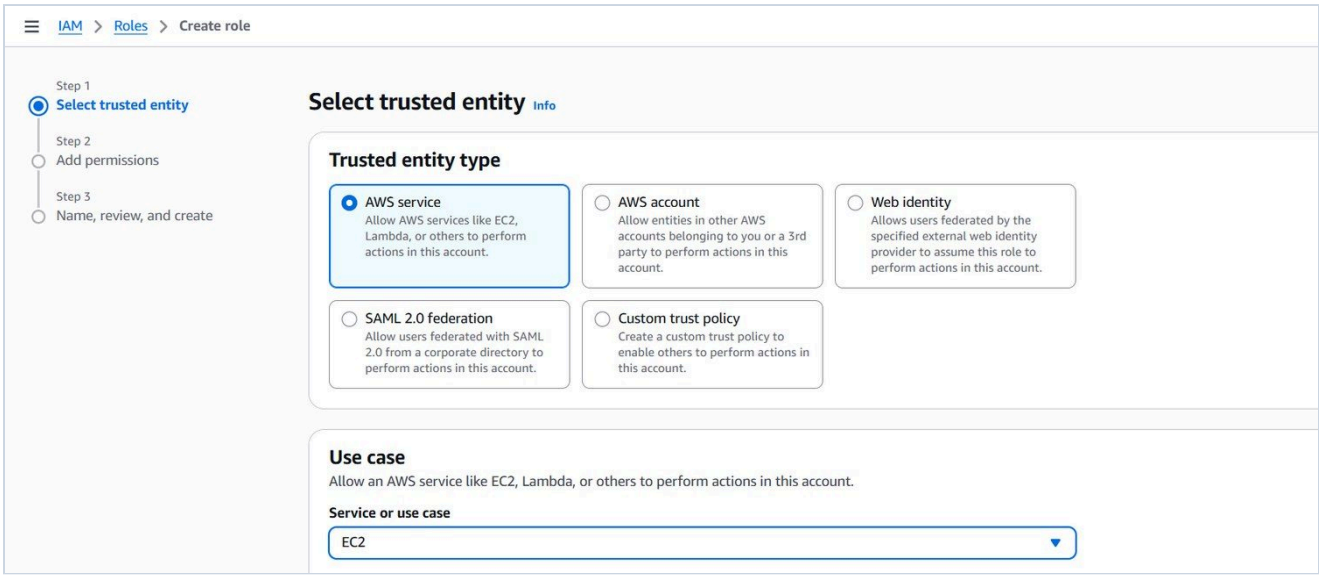


Figure 76. Select trusted entity: AWS service, EC2.

4 Search for `AmazonS3ReadOnlyAccess` and tick it.

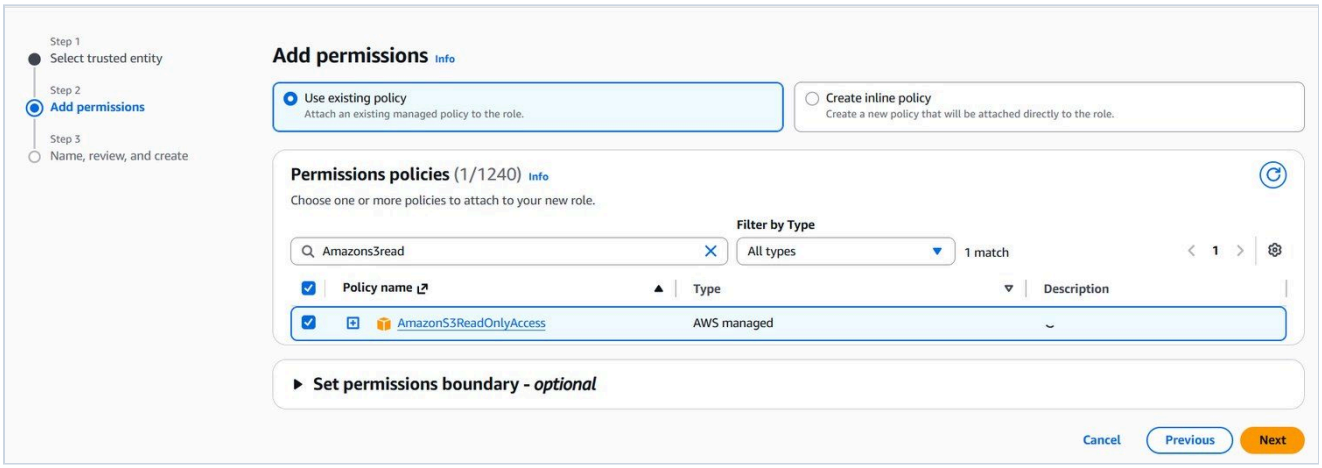


Figure 77. Attach AmazonS3ReadOnlyAccess.

5 Search for `AmazonSSMManagedInstanceCore` and tick it too. Click **Next**.

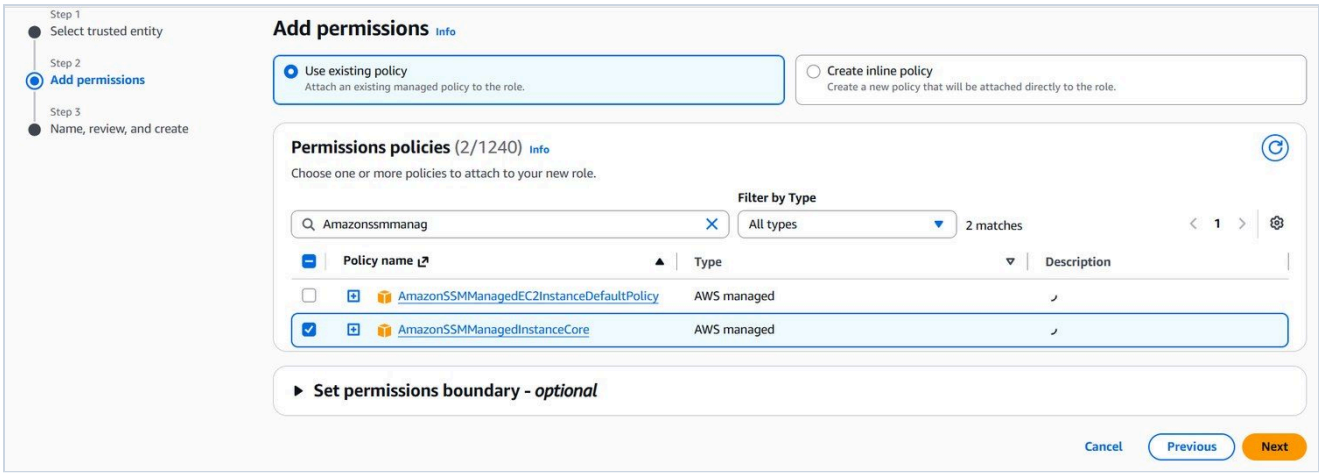


Figure 78. Attach AmazonSSMManagedInstanceCore.

6 Name the role `IAM-role-three-tier-app` and add a description such as "Allows EC2 instances to call AWS services on your behalf".

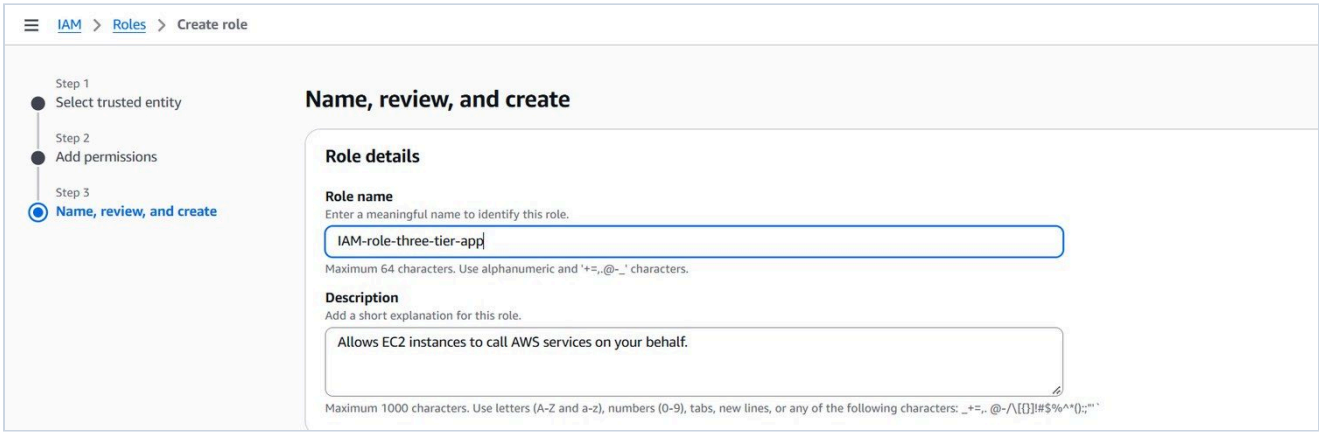


Figure 79. Role name and description.

7 Click **Create role**.

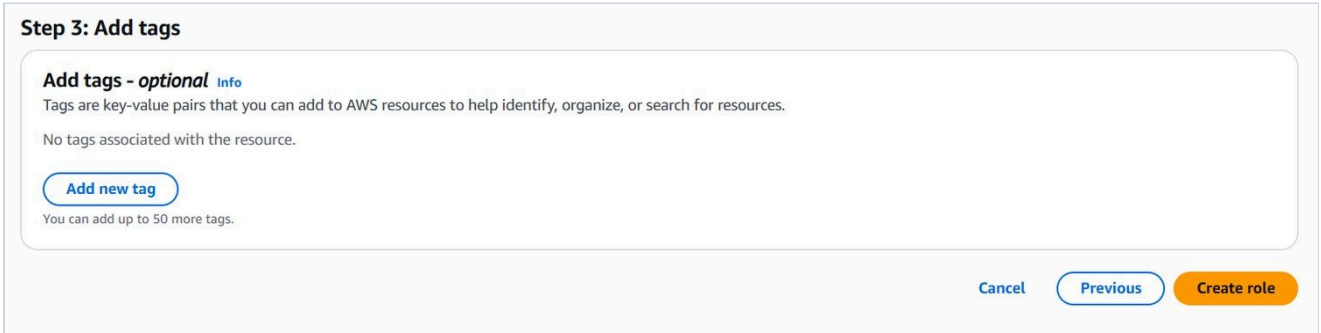


Figure 80. Review and Create role.

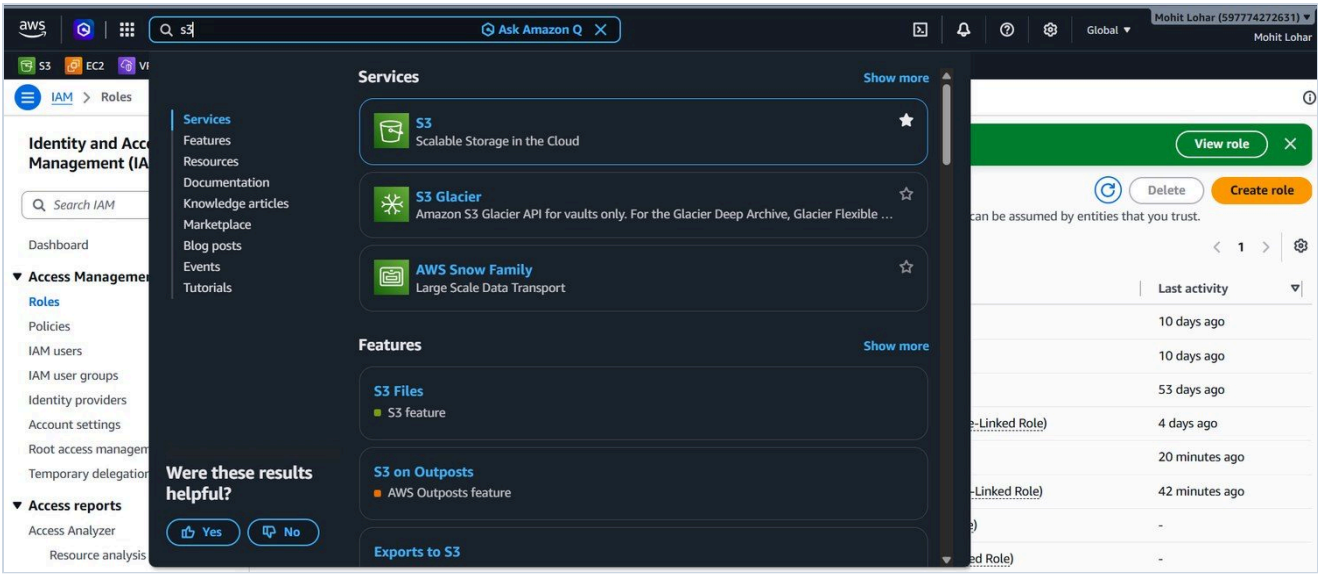


Figure 81. The role is created successfully.

CHECKPOINT

The role `IAM-role-three-tier-app` exists with both policies attached.

STEP 06 OF 15 DATA AND ACCESS

Create an S3 Bucket and Upload the Source Code

Stock the shelf the servers will read from

WHY THIS STEP?

Instead of copying files onto every server by hand, the code sits in one S3 bucket. Any server with the IAM role can fetch it in a single command.

Create the bucket

1. Search for **S3**, open **Buckets** and click **Create bucket**.

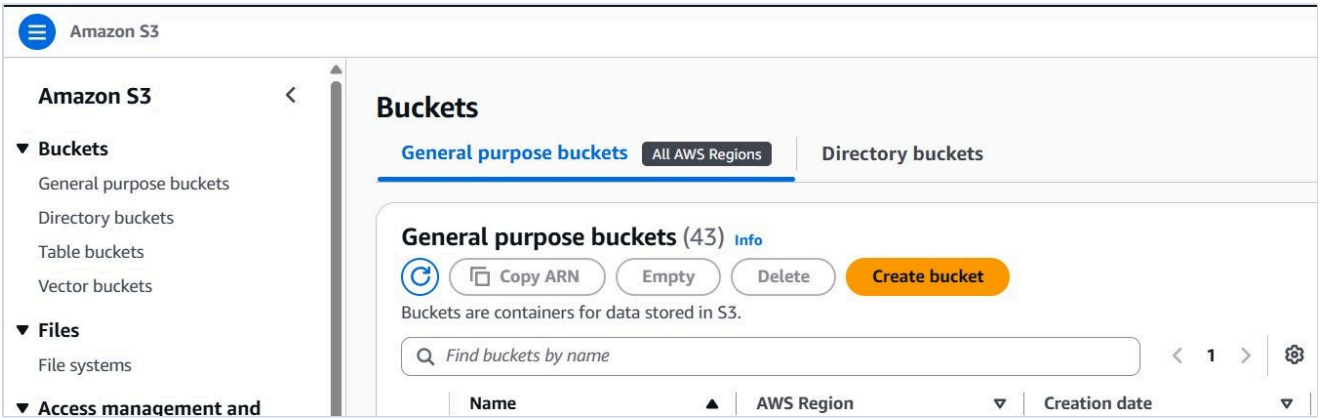


Figure 82. The S3 buckets page: Create bucket.

2. Bucket type **General purpose**, with a globally unique name (here `three-tier-app-bucket-5-11-26`) in `ap-southeast-2`.

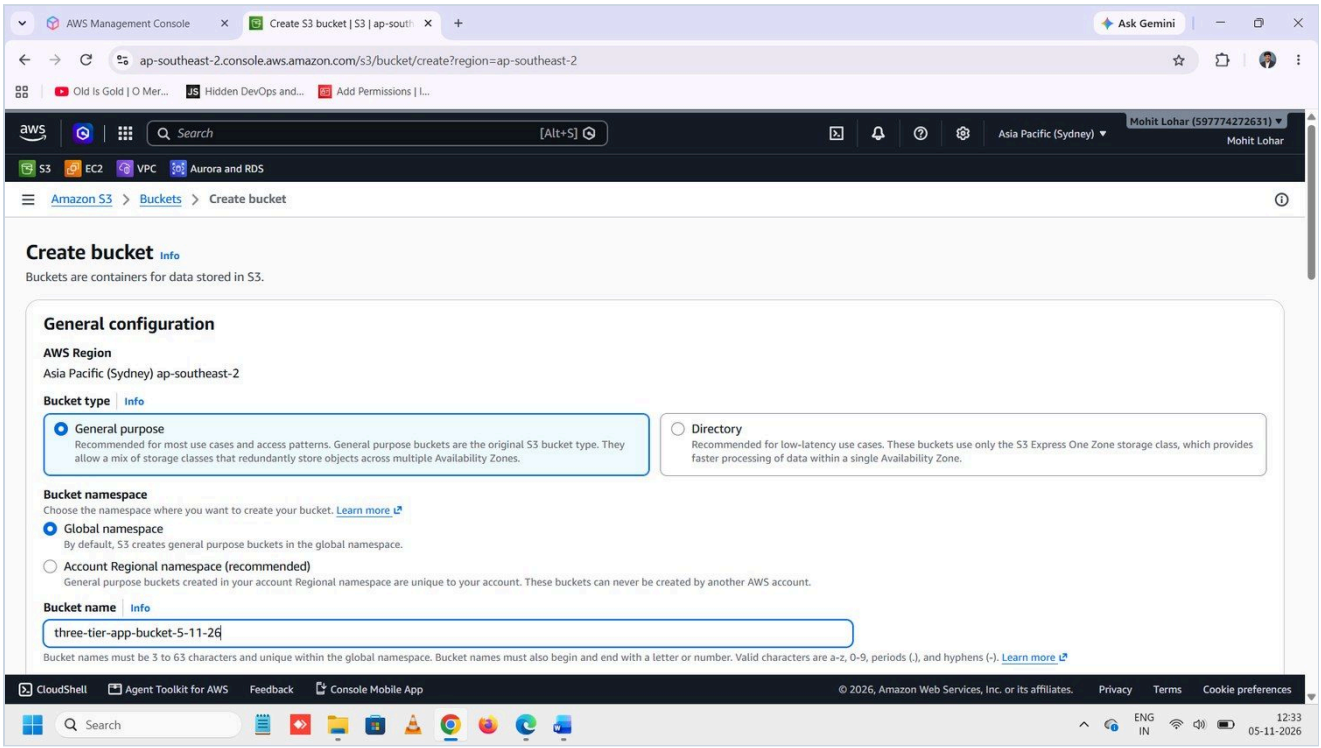


Figure 83. Bucket name and Region.

3. Object ownership: **ACLs disabled** (recommended).

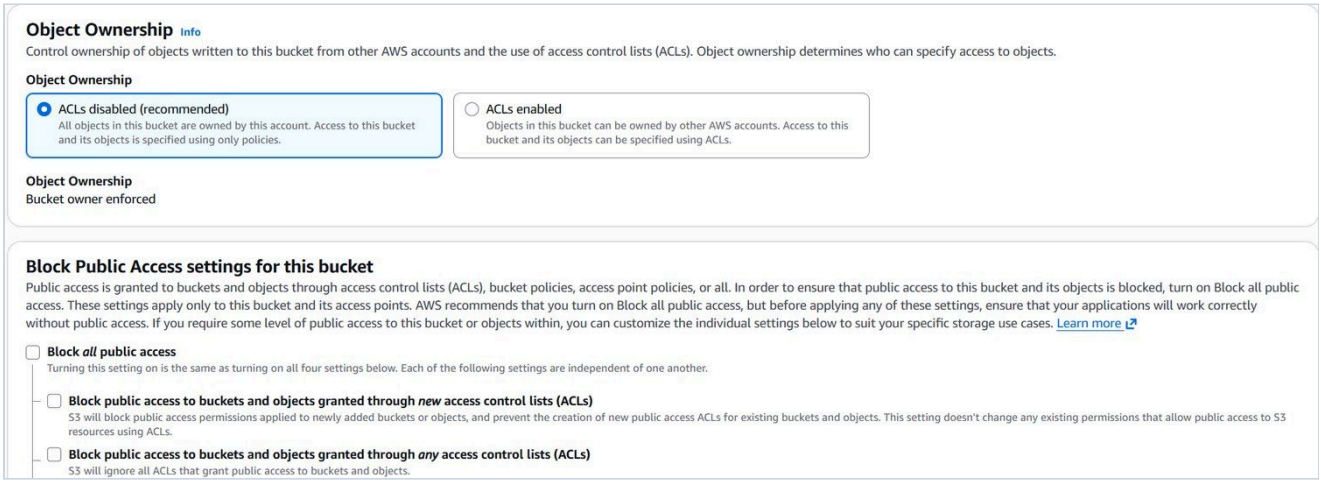


Figure 84. Object ownership and Block Public Access settings.

> **WARNING**
> This screenshot shows **Block all public access** unchecked. The servers read the code through the IAM role, so public access is not needed. Leave Block all public access ticked for this bucket.

4. Leave versioning disabled (or enable it if you want file history), keep the default SSE-S3 encryption, and click **Create bucket**.

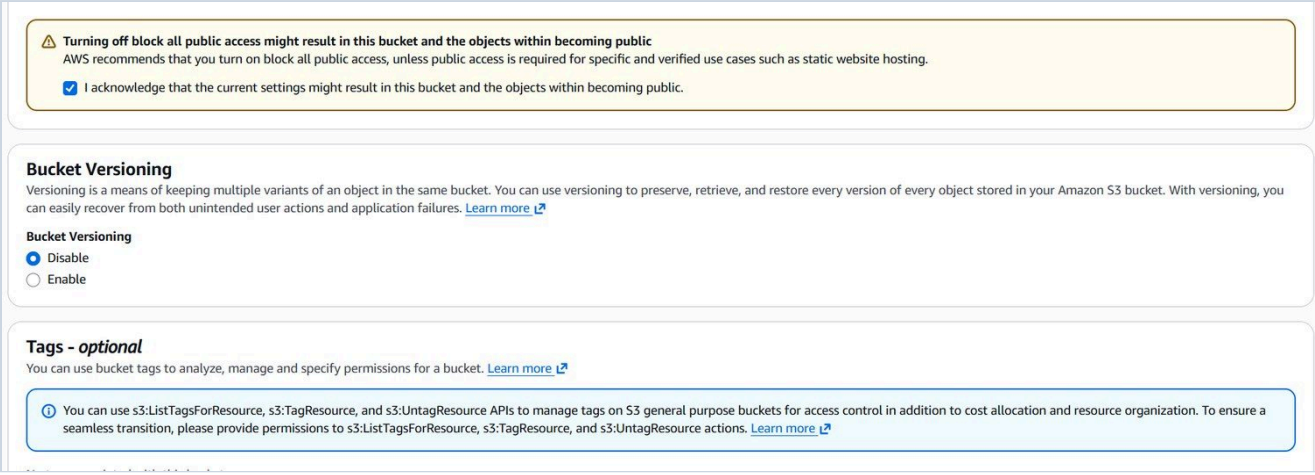


Figure 85. Public-access acknowledgement, bucket versioning and tags.

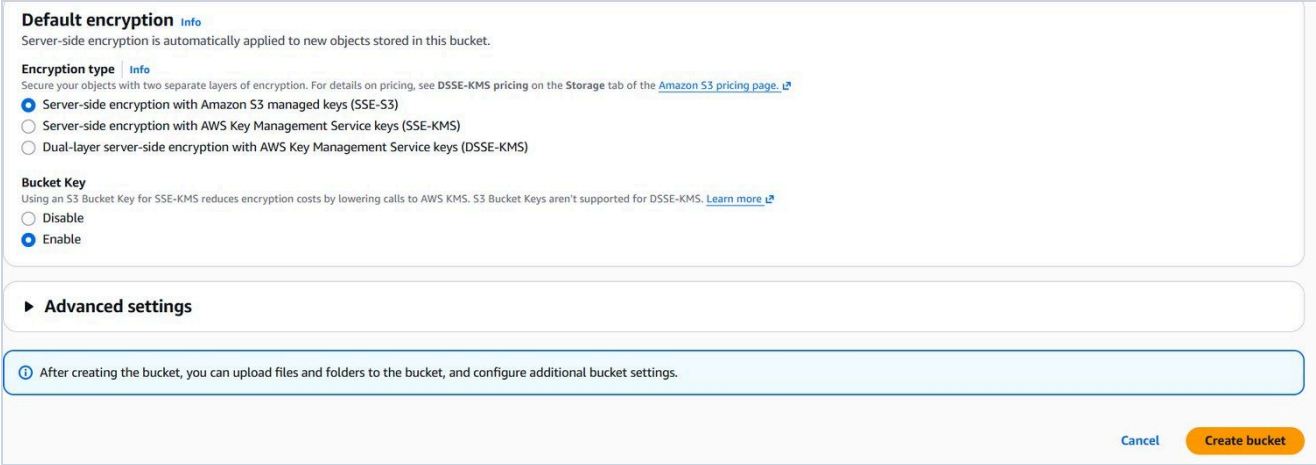


Figure 86. Default encryption and Create bucket.

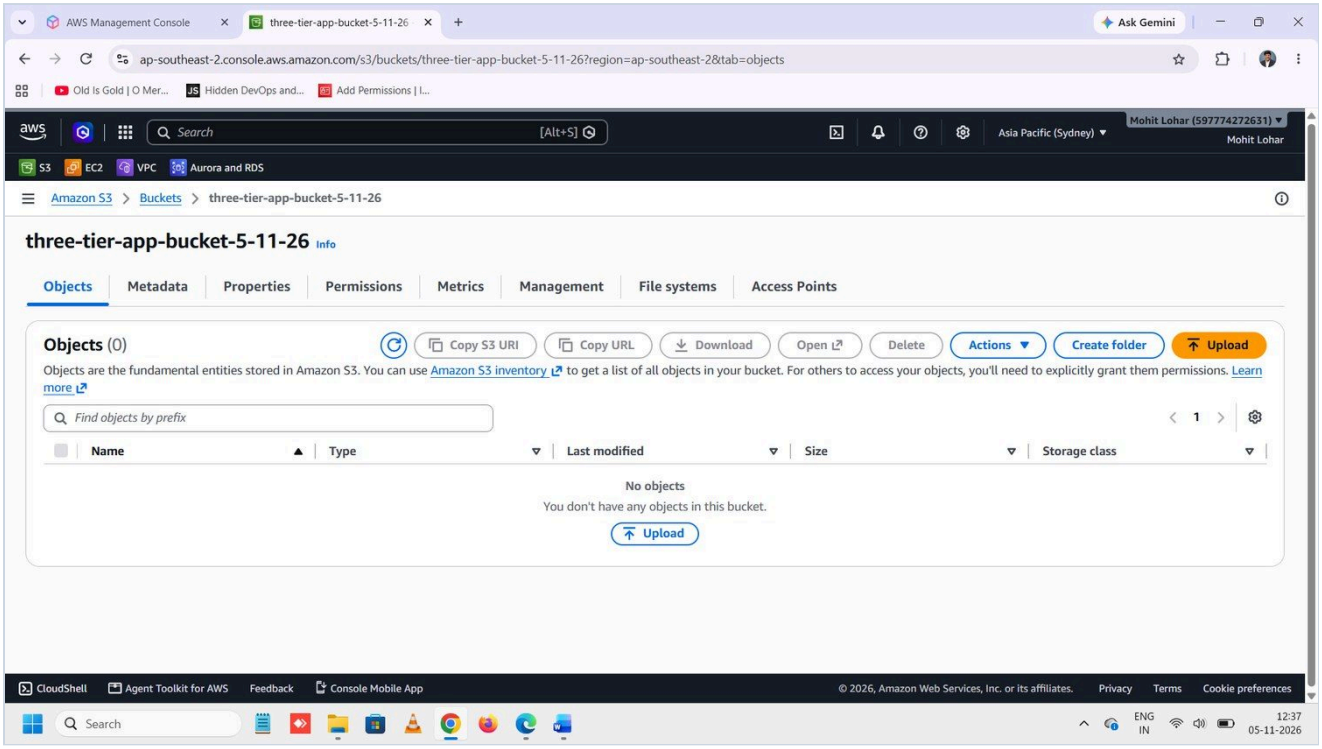


Figure 87. The empty bucket after creation, ready for upload.

Upload the application code

Clone or download the sample repository to your PC, then upload the `application-code` folder (it contains `app-tier`, `web-tier` and `nginx.conf`).

1. In the bucket click **Upload > Add folder** and select `application-code`.

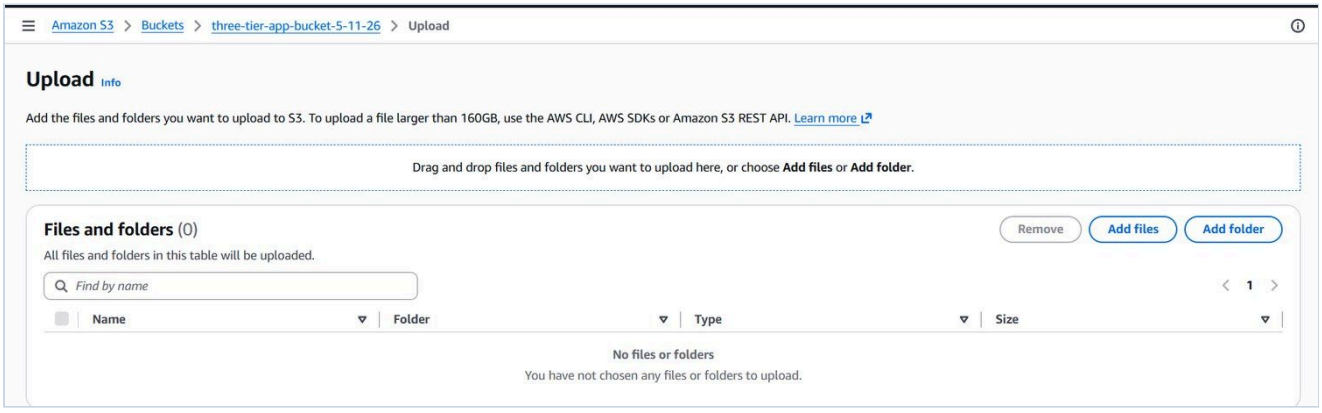


Figure 88. The Upload page.

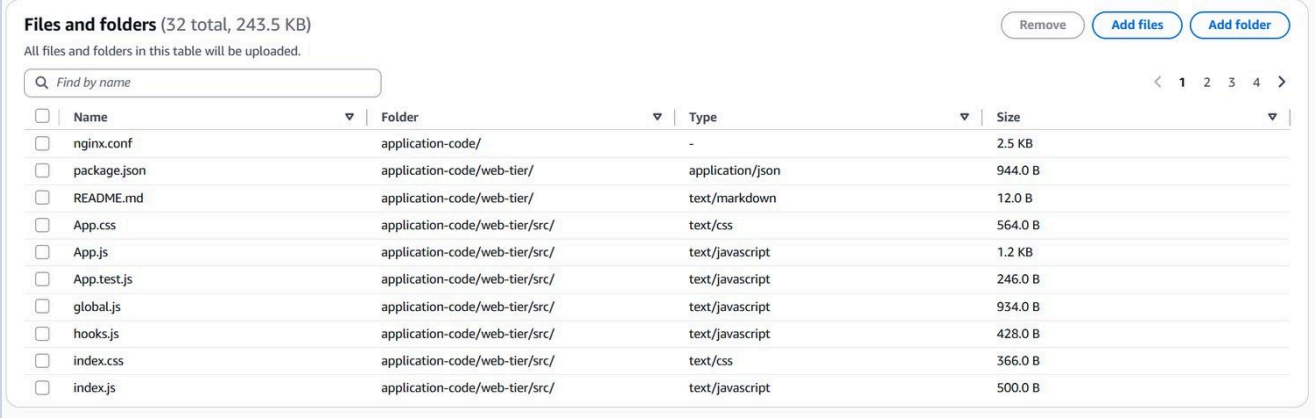


Figure 89. Files and folders queued for upload (web-tier and app-tier source).

2 Click **Upload**. The bucket now contains the `application-code/` folder.

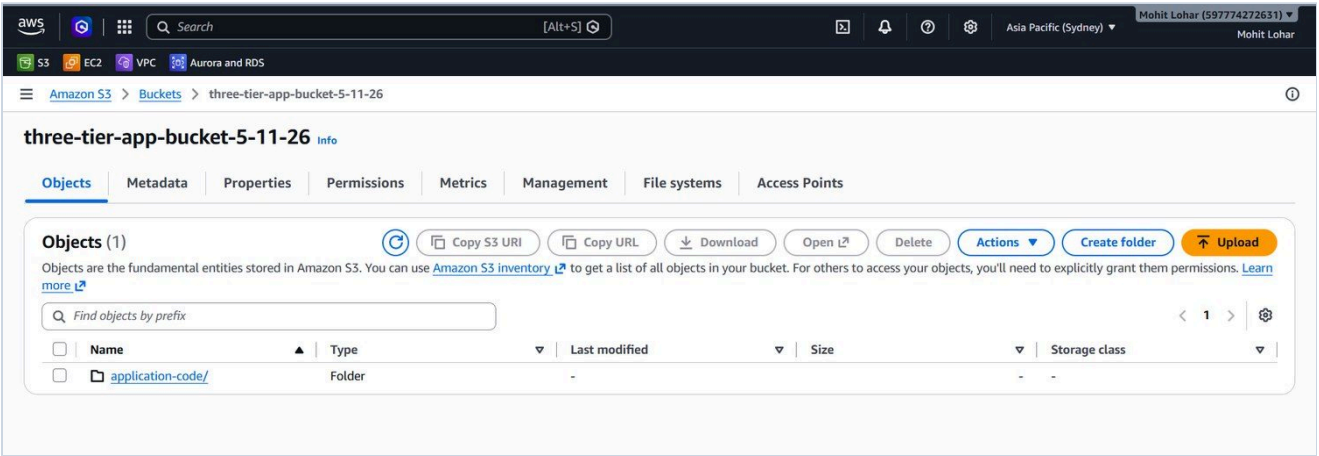


Figure 90. The application-code folder is in the bucket.

CHECKPOINT

The bucket lists the `application-code/` folder with the app-tier, web-tier and nginx.conf files inside.

STEP 07 OF 15 SERVERS AND TRAFFIC

Create the Test App Server and Set It Up

Build the app server once, by hand

WHY THIS STEP?

Rather than configuring every server separately, one test server per tier is configured and turned into an AMI. Auto Scaling then clones that image whenever it needs a new server.

Launch the test app server

SETTING	VALUE
Name	`Test-App-Server`
AMI	Amazon Linux 2023 (Quick Start)
Instance type	c7i-flex.large (free-tier eligible in this account)
Key pair	Proceed without a key pair (access through Session Manager)
VPC / Subnet	Three-Tier-Project-VPC / App-Tier-Subnet-1-Private
Security group	Select existing security group: `App-Tier-SG`
IAM instance profile (Advanced details)	`IAM-role-three-tier-app`

1. Search for **EC2** and open **Instances > Launch instances**.

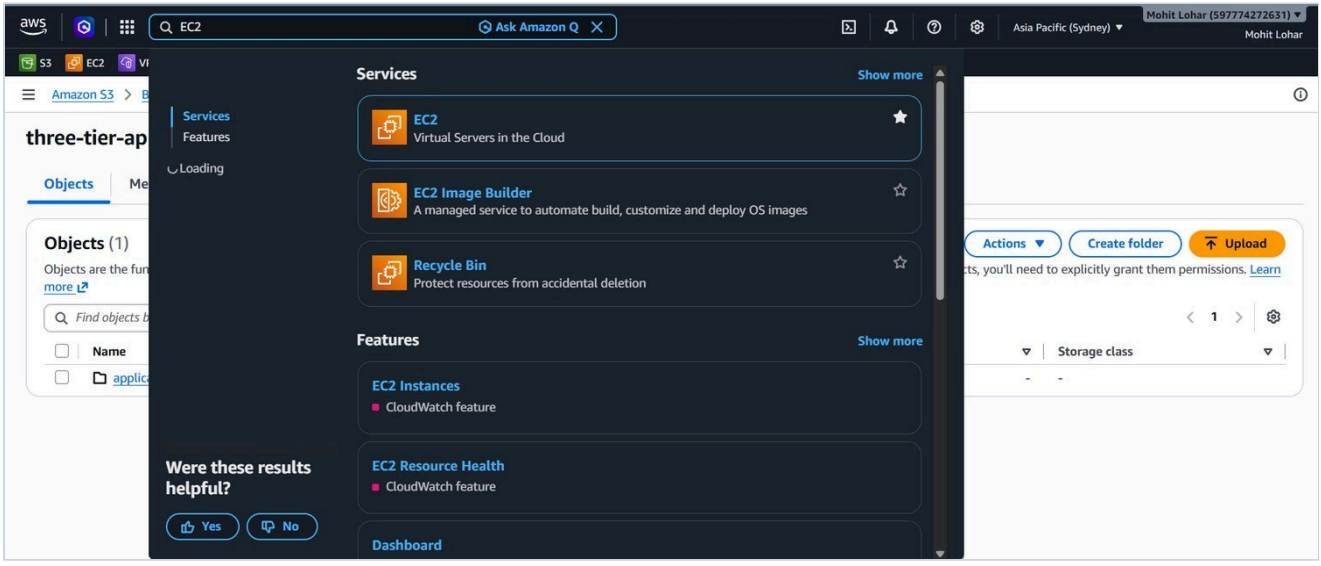


Figure 91. Opening the EC2 service.

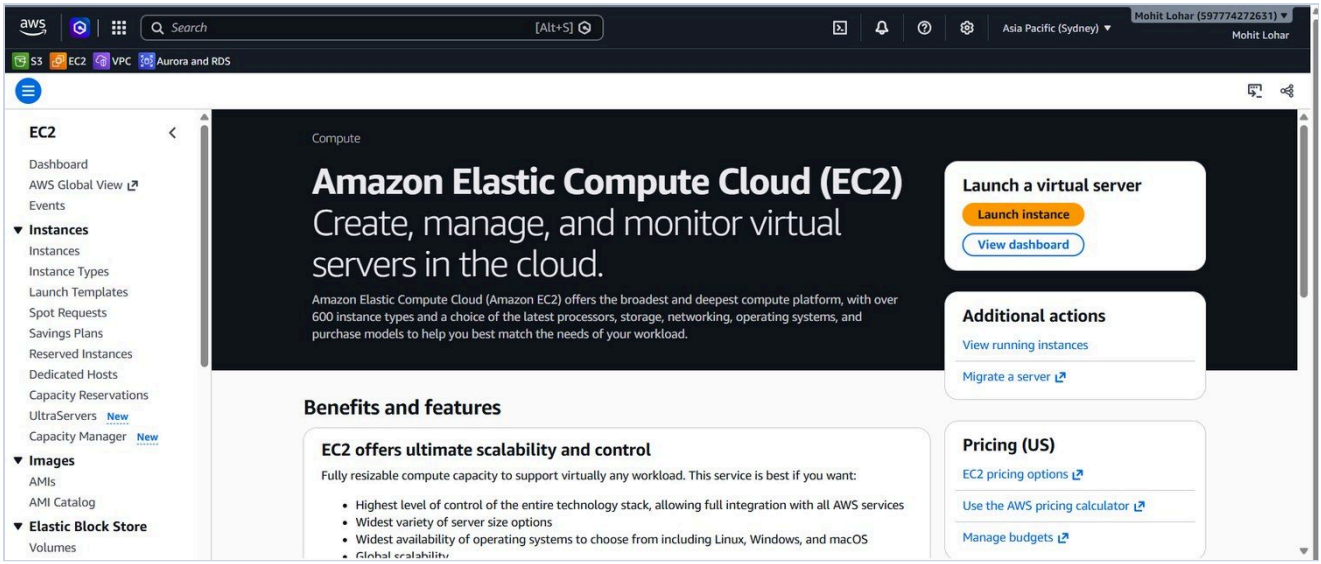


Figure 92. The EC2 dashboard: Launch instance.

2 Enter the name and choose **Amazon Linux 2023**.

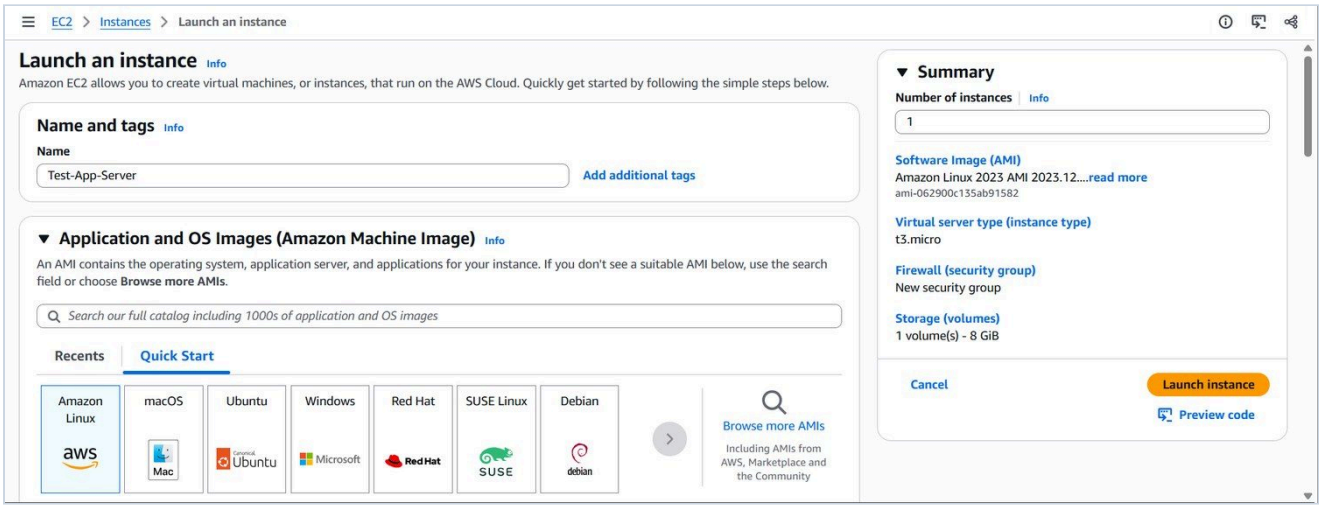


Figure 93. Name and AMI.

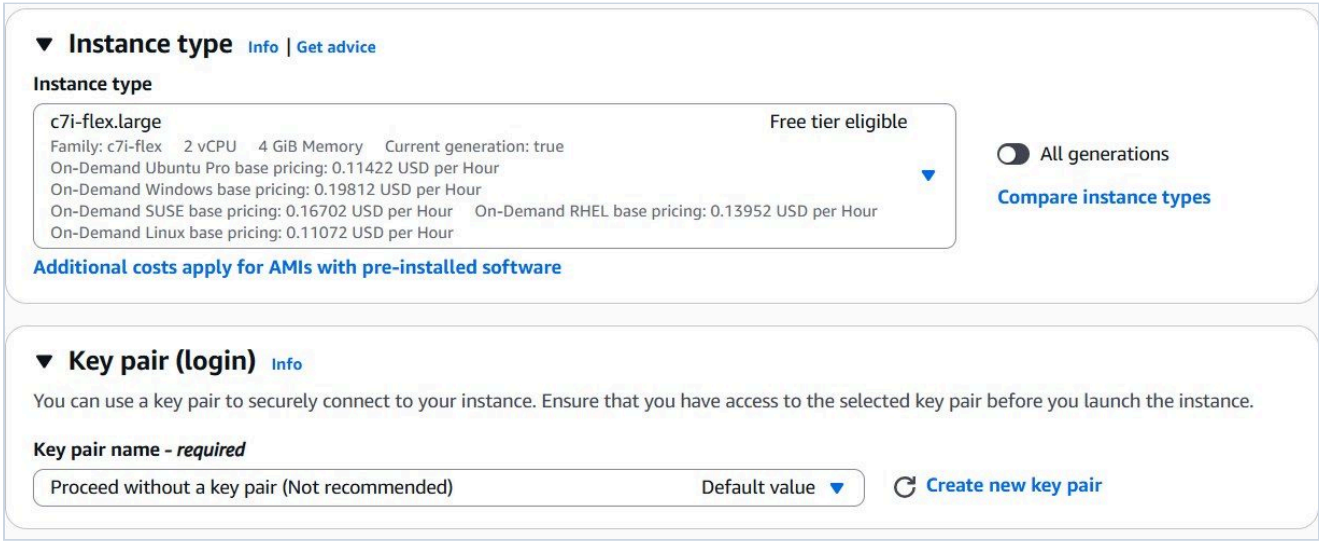


Figure 94. Instance type and key pair.

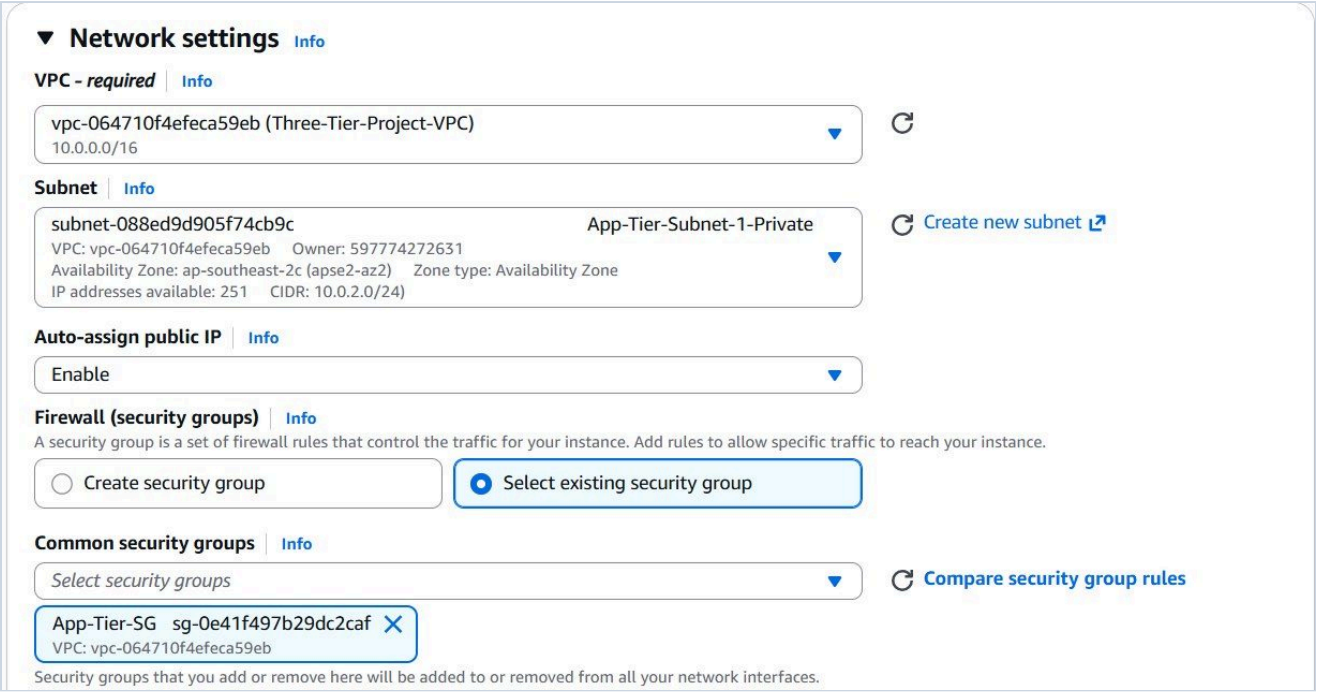


Figure 95. Network settings: App-Tier-Subnet-1-Private and App-Tier-SG.

3 Expand **Advanced details**, set the IAM instance profile and click **Launch instance**.

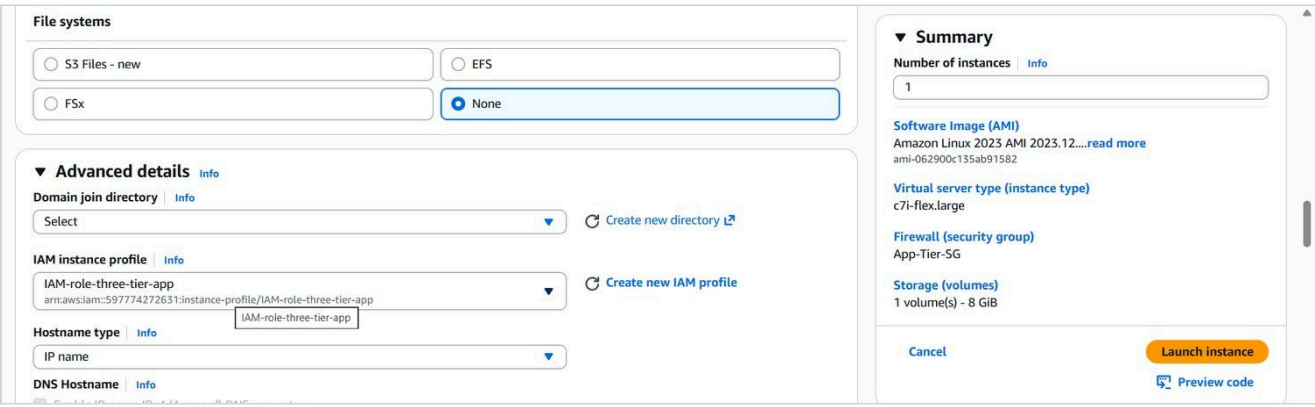


Figure 96. IAM instance profile and Launch instance.

> **NOTE**
> The subnet is private, so the instance has no route from the internet gateway even though auto-assign public IP shows Enable in the captured form. Disable it for private subnets when you reproduce this.

Put your database details in the code

1. On your PC open `application-code/app-tier/DbConfig.js` in your editor. The values are empty.

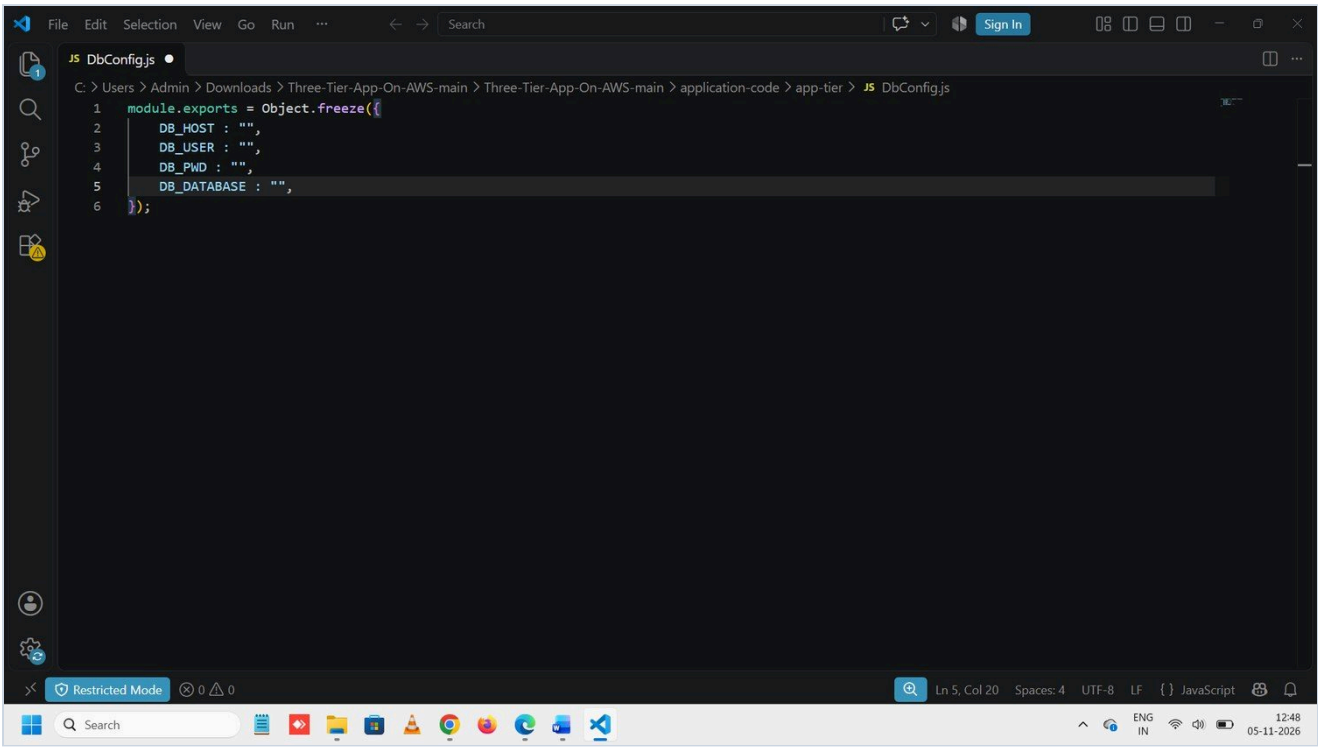


Figure 97. DbConfig.js before editing.

2. Open **RDS > Databases > database-1** and copy the **Endpoint** under Connectivity & security.

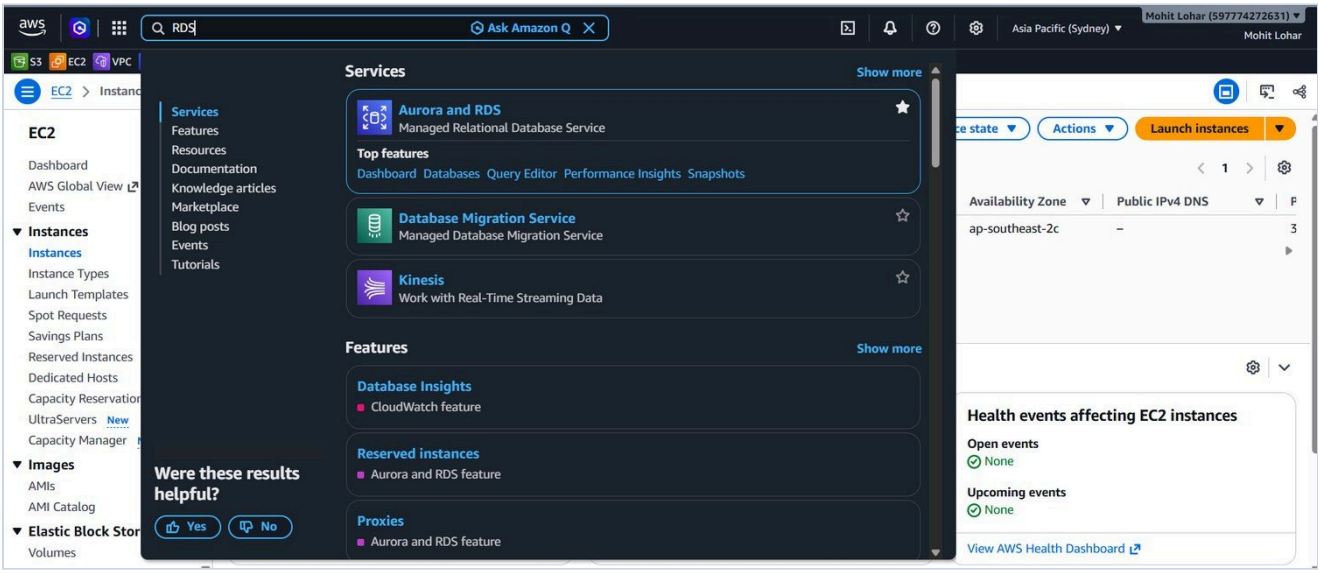


Figure 98. Opening RDS to find the endpoint.

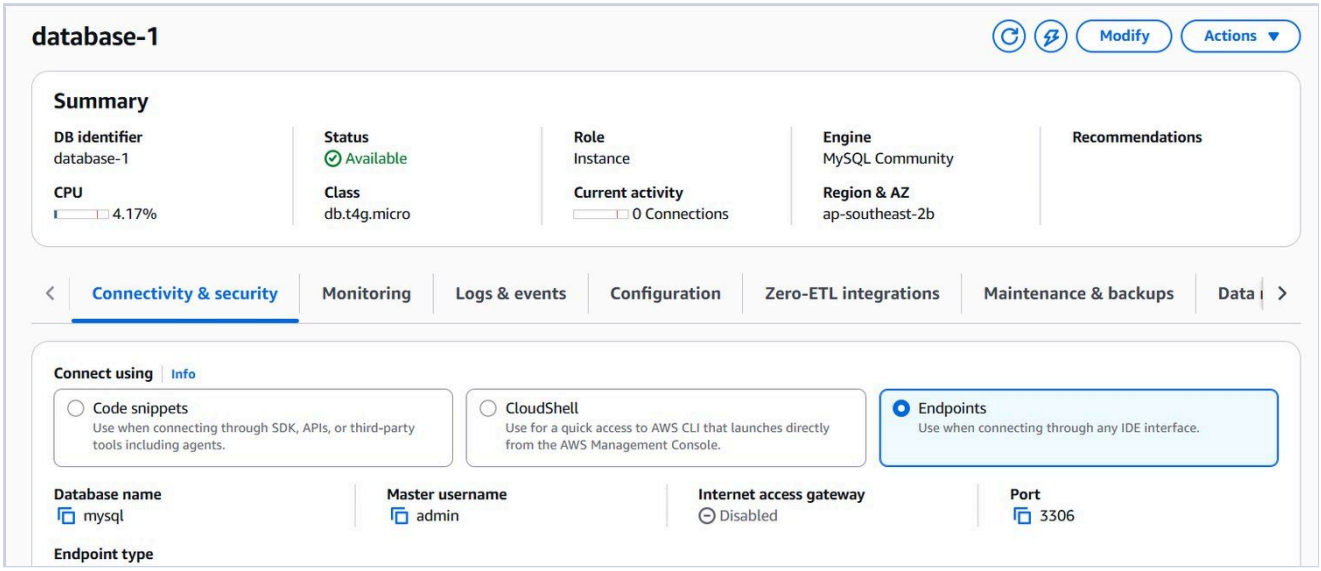


Figure 99. The database summary.

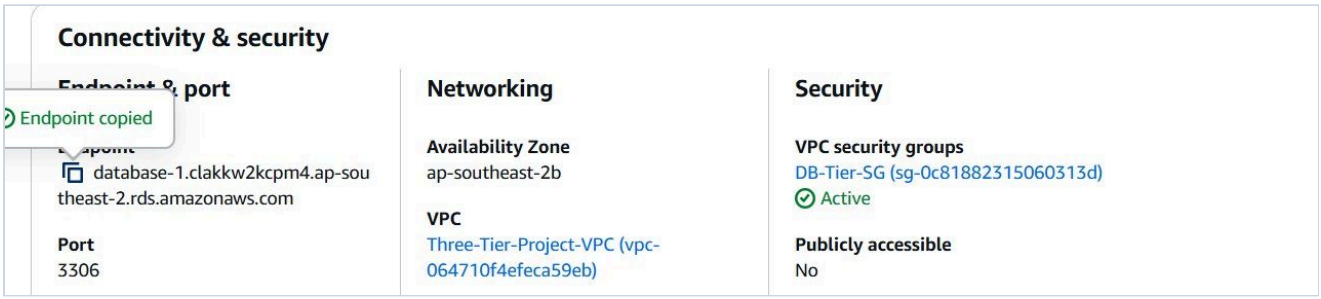


Figure 100. The endpoint value copied.

3 Fill in the four values and save the file:

KEY	VALUE
DB_HOST	The RDS endpoint you copied
DB_USER	admin
DB_PWD	The master password you chose (never commit it to Git)
DB_DATABASE	webappdb

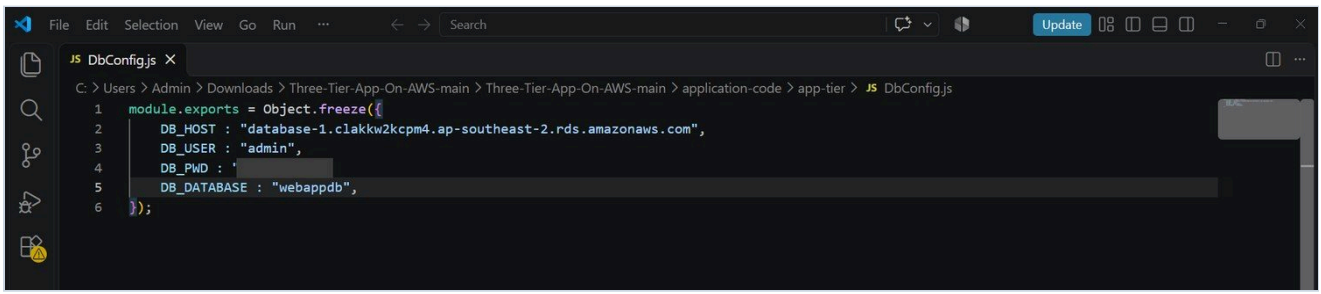


Figure 101. The edited DbConfig.js (password value hidden in this document).

4. In the S3 bucket open `application-code/app-tier/`, delete the old `DbConfig.js` and upload the edited one.

> **SCREENSHOT TO ADD**
> S3 console: replacing DbConfig.js in application-code/app-tier.

Open a terminal with Session Manager

1. In **EC2 > Instances** select `Test-App-Server` and click **Connect**.

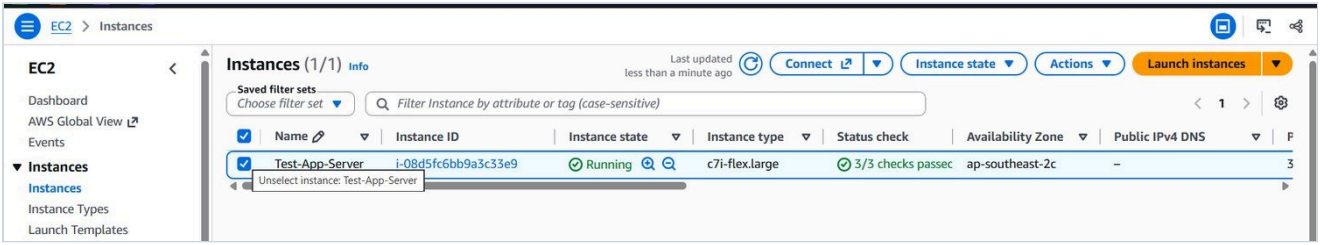


Figure 102. Select Test-App-Server and click Connect.

2. Open the **Session Manager** tab and click **Connect**. A browser terminal opens even though the server is private, because of the IAM role and the SSM agent.

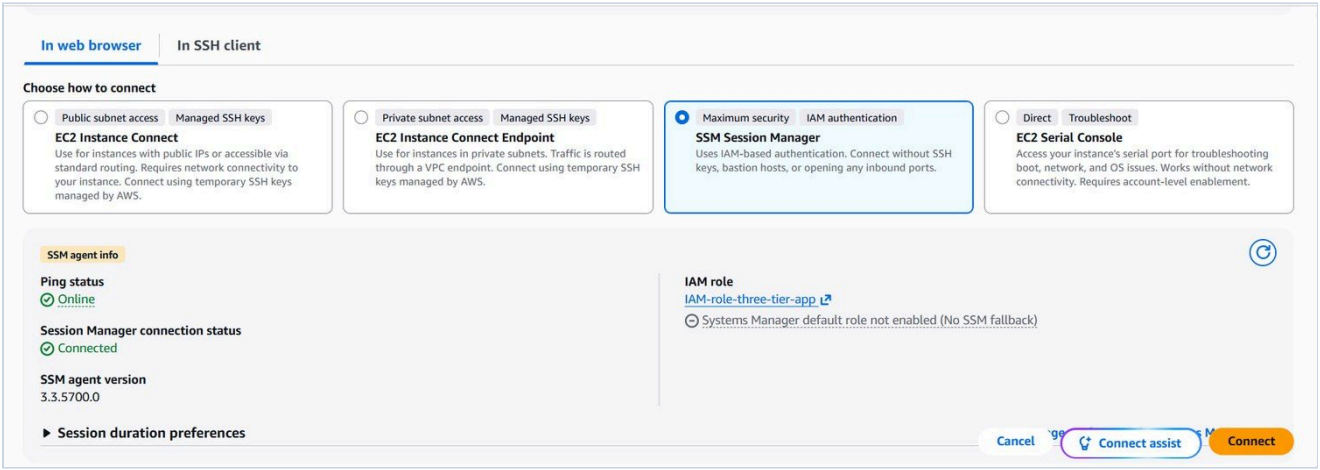


Figure 103. Session Manager tab with the SSM connection status Connected.

Configure the app tier

> **PRO TIP**
> Paste commands one block at a time and wait for each to finish. If something fails, fix it before moving on; errors compound.

1. Switch user and install the MySQL client

TERMINAL BASH

```
sudo -su ec2-user
sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
sudo dnf install mysql-community-client -y
mysql --version
```

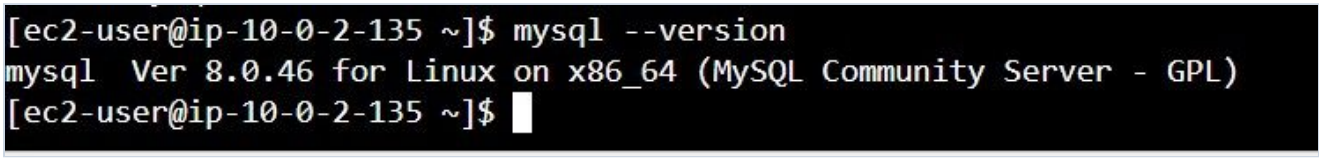


Figure 104. The MySQL client installed; mysql --version reports the client.

2. Connect to RDS and create the schema

Use the RDS endpoint and the user `admin`. Type the password when prompted.

TERMINAL BASH

```
mysql -h <RDS-ENDPOINT> -u admin -p
```

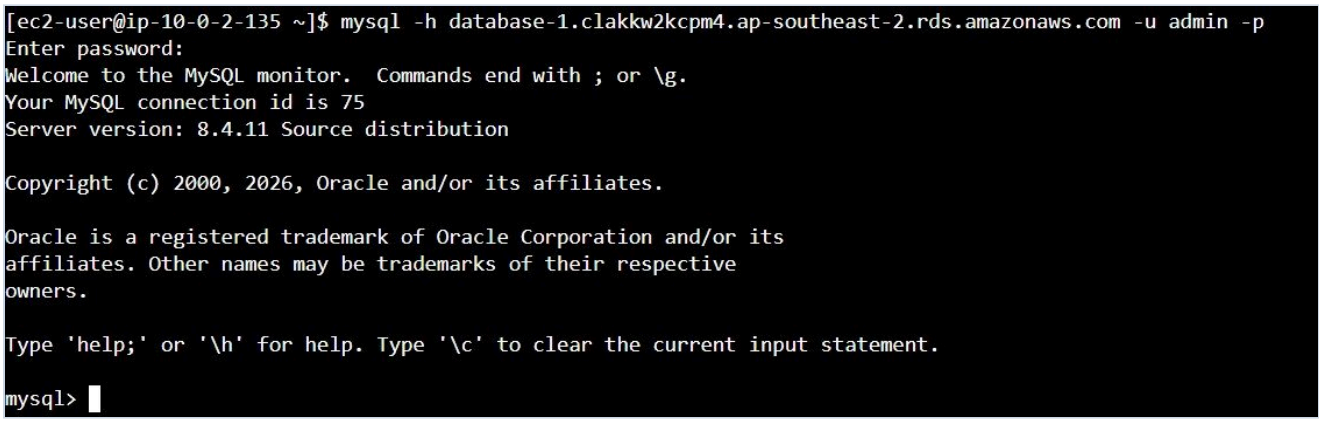


Figure 105. Connected to the MySQL server on RDS.

TERMINAL SQL

```
CREATE DATABASE webappdb;
SHOW DATABASES;
USE webappdb;

CREATE TABLE IF NOT EXISTS transactions(
   id INT NOT NULL AUTO_INCREMENT,
   amount DECIMAL(10,2),
   description VARCHAR(100),
   PRIMARY KEY(id)
);
SHOW TABLES;
INSERT INTO transactions (amount, description) VALUES (400, 'groceries');
SELECT * FROM transactions;
exit
```

```
mysql> create DATABASE webappdb;
Query OK, 1 row affected (0.02 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| webappdb           |
+--------------------+
5 rows in set (0.01 sec)

mysql> use webappdb;
Database changed
mysql>
```

Figure 106. CREATE DATABASE webappdb and SHOW DATABASES.

```
mysql> CREATE TABLE IF NOT EXISTS transactions (
    ->     id INT NOT NULL AUTO_INCREMENT,
    ->     amount DECIMAL(10,2),
    ->     description VARCHAR(100),
    ->     PRIMARY KEY (id)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> show tables;
+--------------------+
| Tables_in_webappdb |
+--------------------+
| transactions       |
+--------------------+
1 row in set (0.00 sec)

mysql>
```

Figure 107. The transactions table is created.

```
mysql> INSERT INTO transactions (amount, description)
    -> VALUES (400.00, 'groceries');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM transactions;
+----+--------+-------------+
| id | amount | description |
+----+--------+-------------+
|  1 | 400.00 | groceries   |
+----+--------+-------------+
1 row in set (0.00 sec)

mysql>
```

Figure 108. A test row is inserted and read back.

3. Download the app code from S3

TERMINAL BASH

```
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/app-tier app-tier --recursive
cd app-tier
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

```
[ec2-user@ip-10-0-2-135 ~]$ cd /home/ec2-user
[ec2-user@ip-10-0-2-135 ~]$ sudo aws s3 cp s3://three-tier-app-bucket-5-11-26/application-code/app-tier/ app-tier/ --recursive
download: s3://three-tier-app-bucket-5-11-26/application-code/app-tier/package.json to app-tier/package.json
download: s3://three-tier-app-bucket-5-11-26/application-code/app-tier/README.md to app-tier/README.md
download: s3://three-tier-app-bucket-5-11-26/application-code/app-tier/TransactionService.js to app-tier/TransactionService.js
download: s3://three-tier-app-bucket-5-11-26/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://three-tier-app-bucket-5-11-26/application-code/app-tier/package-lock.json to app-tier/package-lock.json
download: s3://three-tier-app-bucket-5-11-26/application-code/app-tier/index.js to app-tier/index.js
[ec2-user@ip-10-0-2-135 ~]$ cd app-tier
[ec2-user@ip-10-0-2-135 app-tier]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-135 app-tier]$ sudo chmod -R 755 /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-135 app-tier]$
```

Figure 109. The app-tier files copied from S3 and permissions set.

4. Install Node.js, PM2 and the dependencies

TERMINAL BASH

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
npm install -g pm2
npm install
npm audit fix
```

> **NOTE**
> The captured run reports Node.js v20.20.2 (see the PM2 log below); the reference guide uses Node.js 16. The install commands themselves were not captured in the screenshots.

5. Start the app with PM2

TERMINAL BASH

```
pm2 start index.js
pm2 logs
```

```
[ec2-user@ip-10-0-2-135 app-tier]$ pm2 logs
[TAILING] Tailing last 15 lines for [all] processes (change the value with --lines option)
/home/ec2-user/.pm2/pm2.log last 15 lines:
PM2        | 2026-11-05T07:49:13: PM2 log: Node.js version      : 20.20.2
PM2        | 2026-11-05T07:49:13: PM2 log: Current arch         : x64
PM2        | 2026-11-05T07:49:13: PM2 log: PM2 home             : /home/ec2-user/.pm2
PM2        | 2026-11-05T07:49:13: PM2 log: PM2 PID file         : /home/ec2-user/.pm2/pm2.pid
PM2        | 2026-11-05T07:49:13: PM2 log: RPC socket file      : /home/ec2-user/.pm2/rpc.sock
PM2        | 2026-11-05T07:49:13: PM2 log: BUS socket file      : /home/ec2-user/.pm2/pub.sock
PM2        | 2026-11-05T07:49:13: PM2 log: Application log path : /home/ec2-user/.pm2/logs
PM2        | 2026-11-05T07:49:13: PM2 log: Worker Interval      : 30000
PM2        | 2026-11-05T07:49:13: PM2 log: Process dump file    : /home/ec2-user/.pm2/dump.pm2
PM2        | 2026-11-05T07:49:13: PM2 log: Concurrent actions   : 2
PM2        | 2026-11-05T07:49:13: PM2 log: SIGTERM timeout      : 1600
PM2        | 2026-11-05T07:49:13: PM2 log: Runtime Binary       : /home/ec2-user/.nvm/versions/node/v20.20.2/bin/node
PM2        | 2026-11-05T07:49:13: PM2 log: ================================================================================
PM2        | 2026-11-05T07:49:13: PM2 log: App [index:0] starting in -fork mode-
PM2        | 2026-11-05T07:49:13: PM2 log: App [index:0] online

/home/ec2-user/.pm2/logs/index-error.log last 15 lines:
/home/ec2-user/.pm2/logs/index-out.log last 15 lines:
0|index    | AB3 backend app listening at http://localhost:4000
```

Figure 110. PM2 logs: the backend is listening on port 4000.

If the log shows the backend listening on port 4000, press `Ctrl+C`, then make PM2 start at boot, save the process list and test the health endpoint:

TERMINAL BASH

```
pm2 startup
sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/<version>/bin <command-printed-by-pm2-
startup>
pm2 save
curl http://localhost:4000/health
```

```
[PM2] Remove init script via:
$ pm2 unstartup systemd
[ec2-user@ip-10-0-2-135 app-tier]$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ec2-user/.pm2/dump.pm2
[ec2-user@ip-10-0-2-135 app-tier]$ curl http://localhost:4000/health
"This is the health check"[ec2-user@ip-10-0-2-135 app-tier]$
```

Figure 111. pm2 startup and pm2 save, then a successful health check response.

CHECKPOINT

PM2 shows the app online, and `curl http://localhost:4000/health` returns the health-check message.

STEP 08 OF 15 SERVERS AND TRAFFIC

Create an AMI of the App Server

Freeze the server into a reusable image

WHY THIS STEP?

An AMI is a saved copy of the configured server. Every future app server will start from this image, already configured and running.

1 In **EC2 > Instances** select `Test-App-Server` and choose **Actions > Image and templates > Create image**.

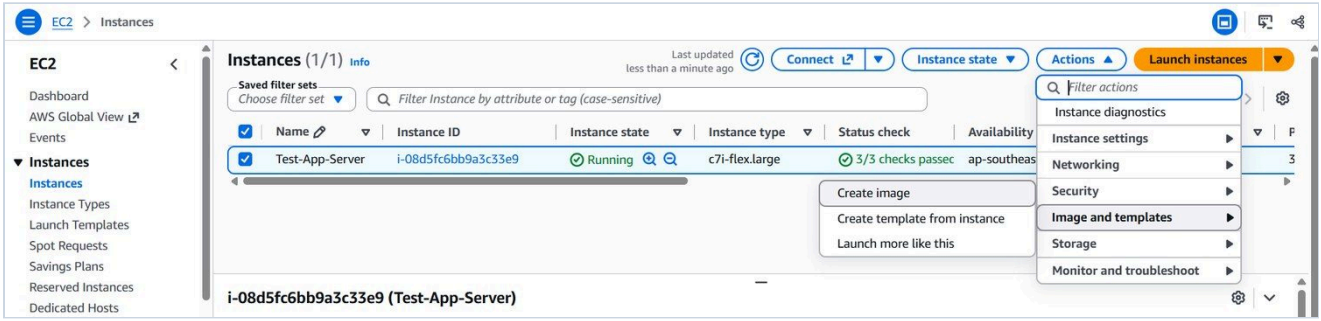


Figure 112. Actions > Image and templates > Create image.

2 Enter the image name and description `App-server-image`. Keep *Reboot instance* ticked so the file system is consistent.

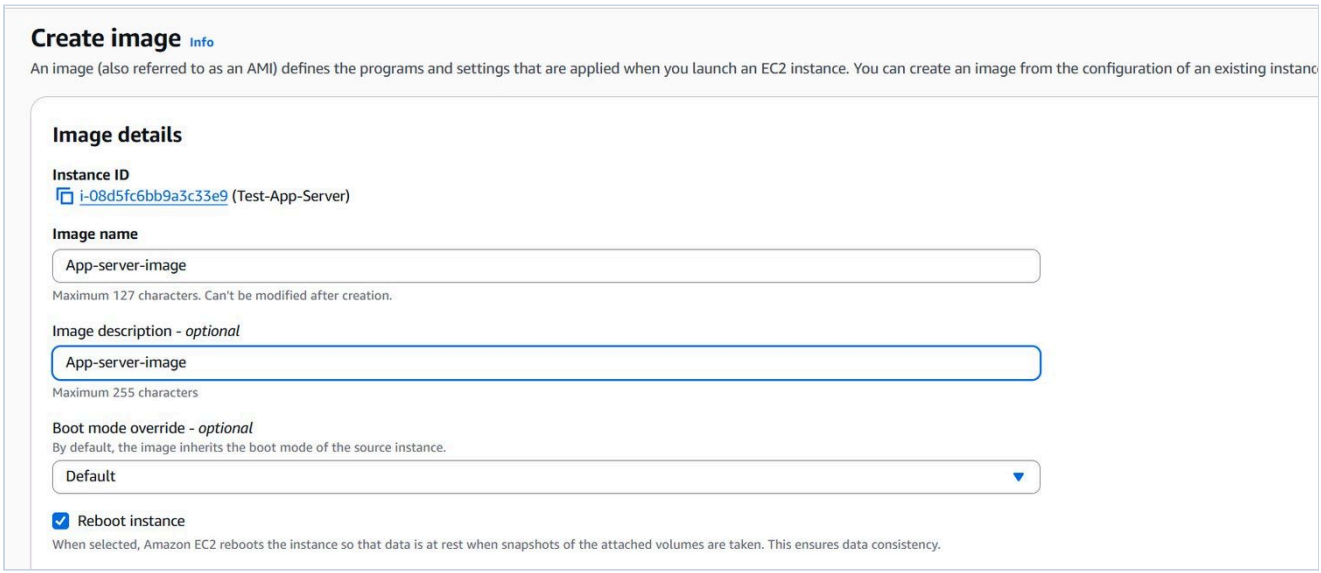


Figure 113. Image details.

3 Click **Create image**.

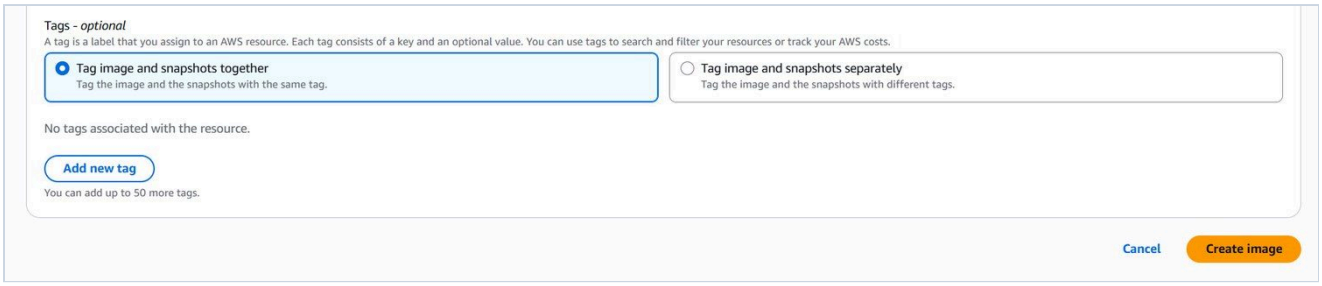


Figure 114. Create image.

4 Open **EC2 > AMIs** and wait until the image is **Available**.

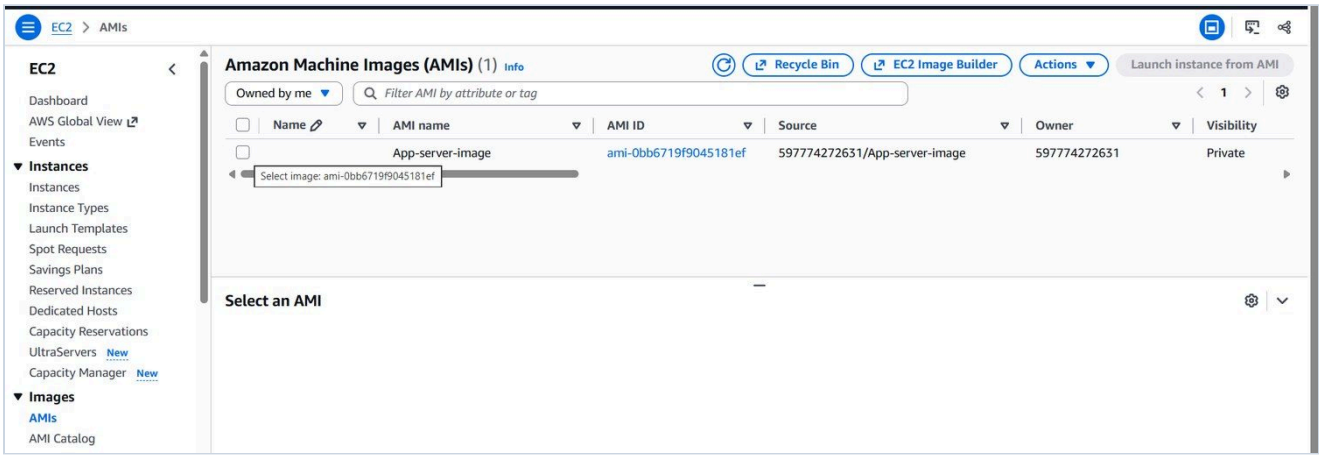


Figure 115. App-server-image listed under AMIs.

CHECKPOINT

The AMI `App-server-image` is listed under AMIs and its status turns Available.

STEP 09 OF 15 SERVERS AND TRAFFIC

Target Group, Launch Template and Internal Load Balancer (App Tier)

Wire the app tier for traffic

WHY THIS STEP?

The target group says which servers receive traffic, the launch template says how to build them, and the internal load balancer is the private front door of the app tier.

A. Create the target group

SETTING	VALUE
Target type	Instances
Name	`App-Server-TG`
Protocol : Port	HTTP : 4000
VPC	Three-Tier-Project-VPC
Health check path	`/health`
Advanced health checks	Traffic port; healthy 2, unhealthy 2, timeout 2 s, interval 5 s
Targets	Register Test-App-Server on port 4000 (Include as pending below)

1 Go to **EC2 > Target Groups > Create target group**.

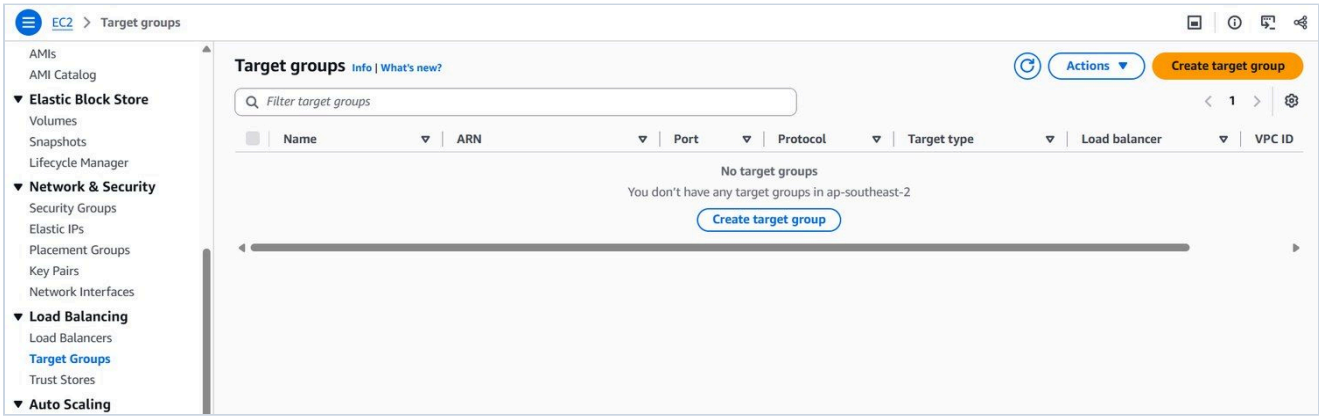


Figure 116. The Target groups page (empty).

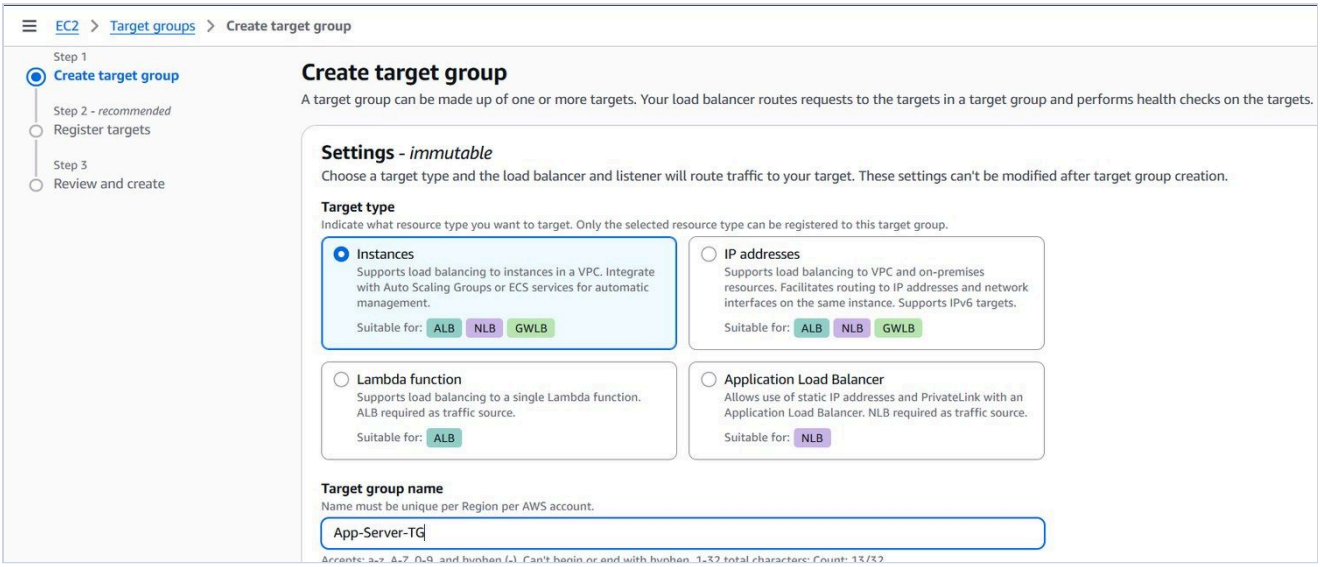


Figure 117. Target type Instances and the name App-Server-TG.

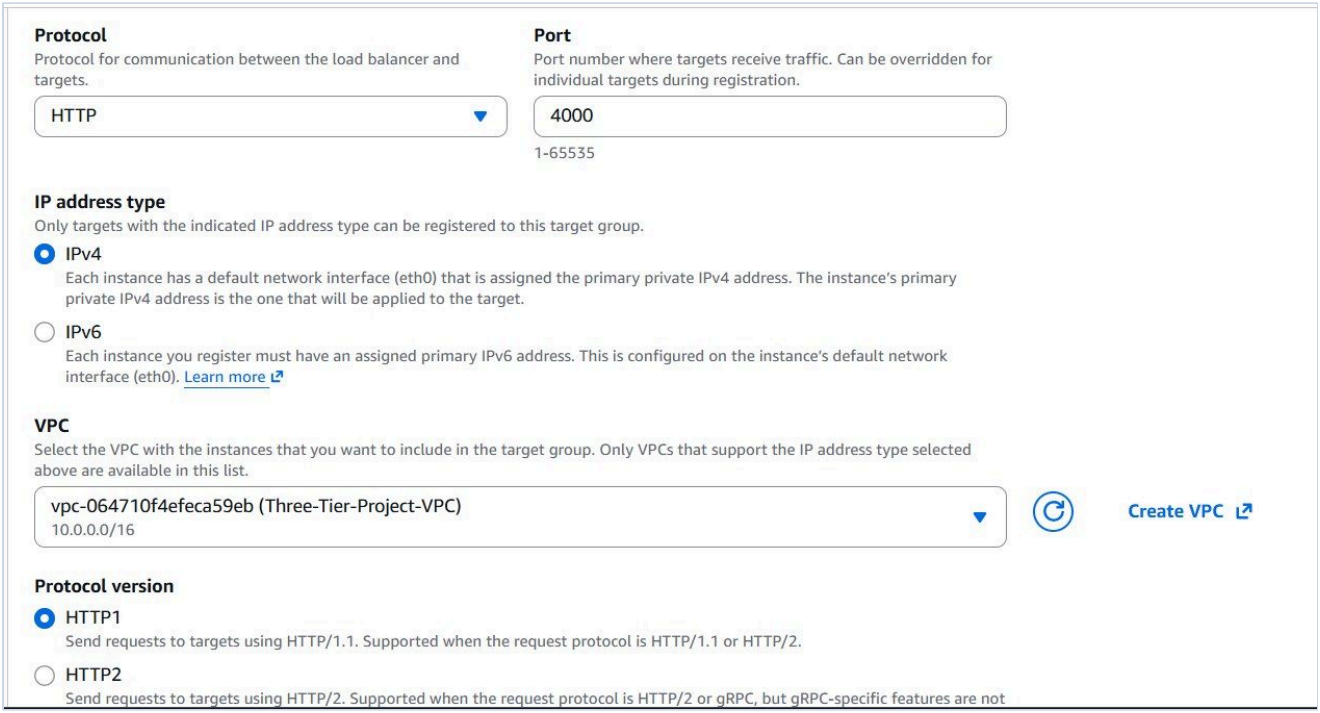


Figure 118. Protocol HTTP, port 4000 and the VPC.

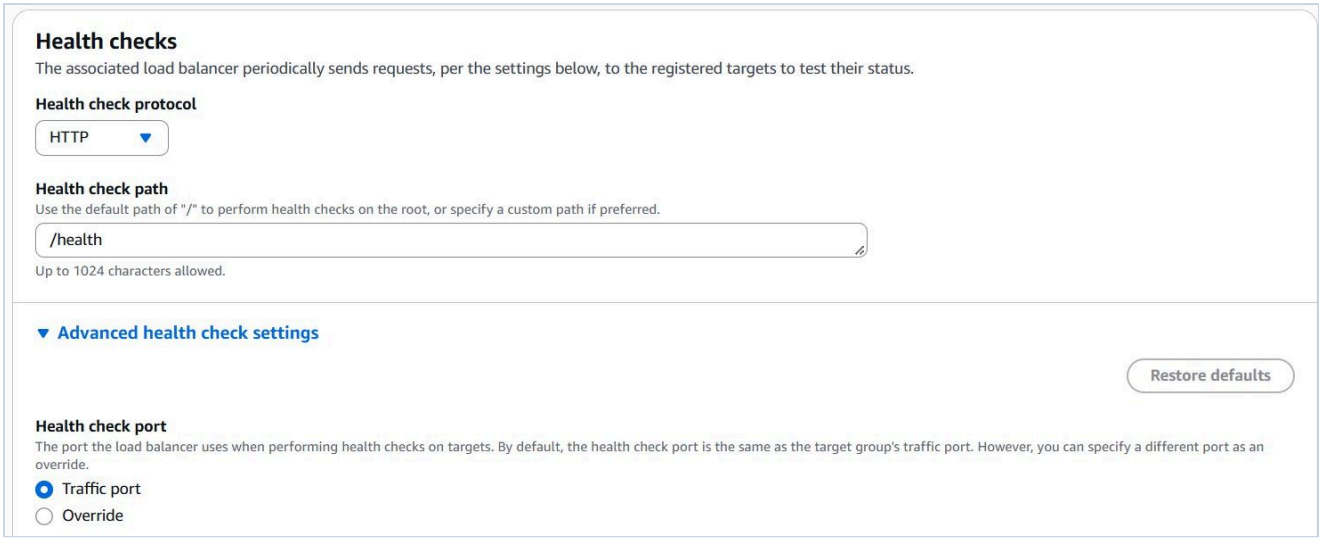


Figure 119. Health check protocol and path /health.

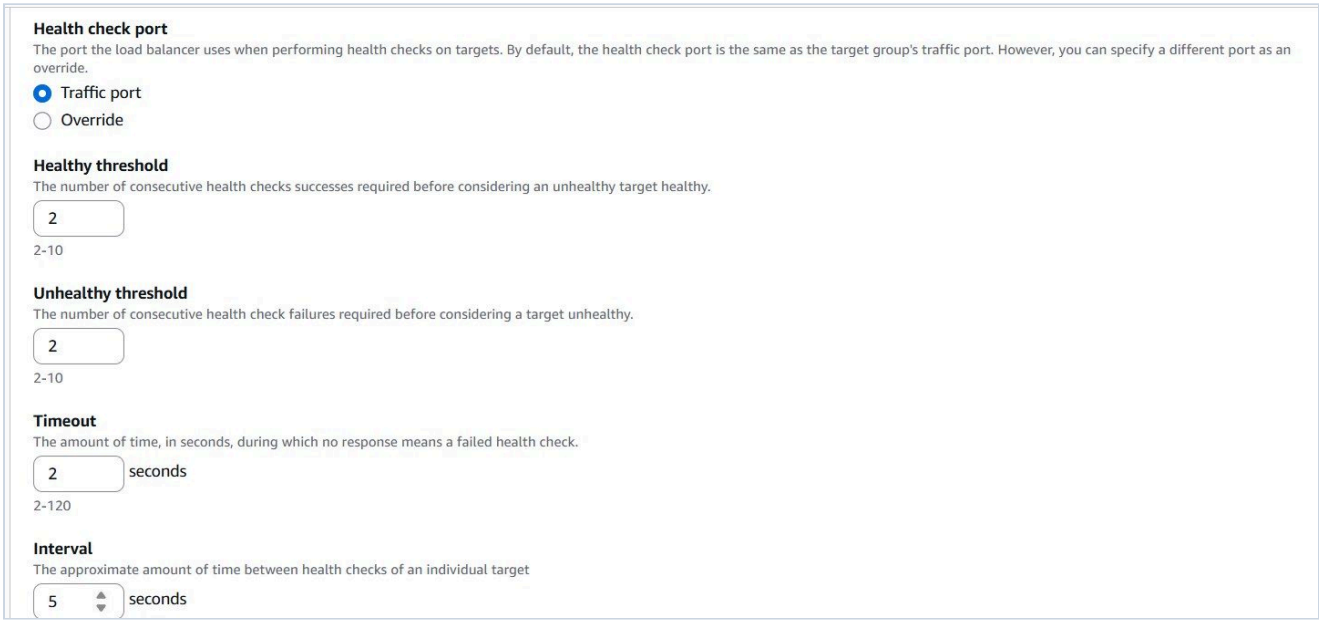


Figure 120. Advanced health-check settings.

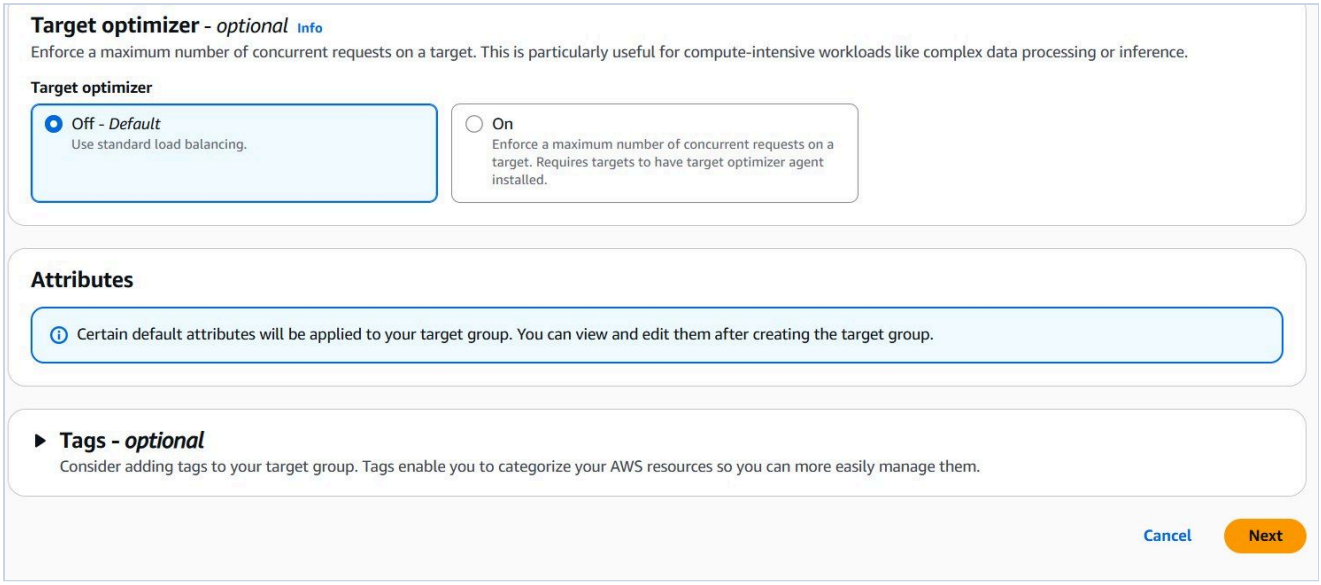


Figure 121. Target optimizer and attributes: click Next.

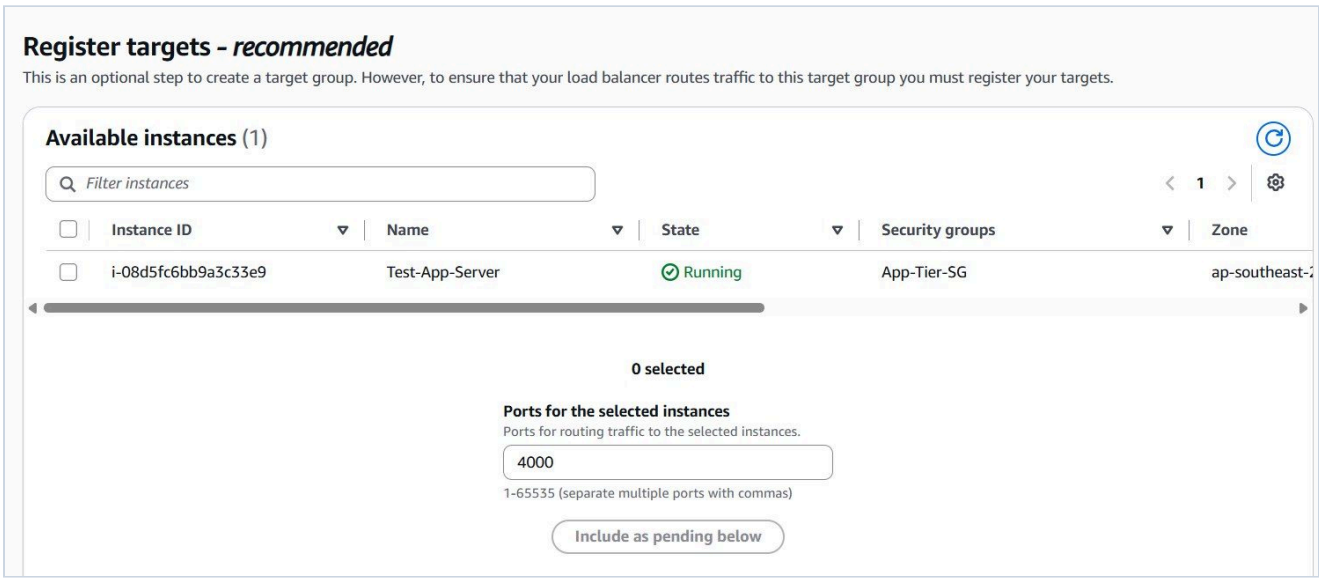


Figure 122. Available instances: port 4000.

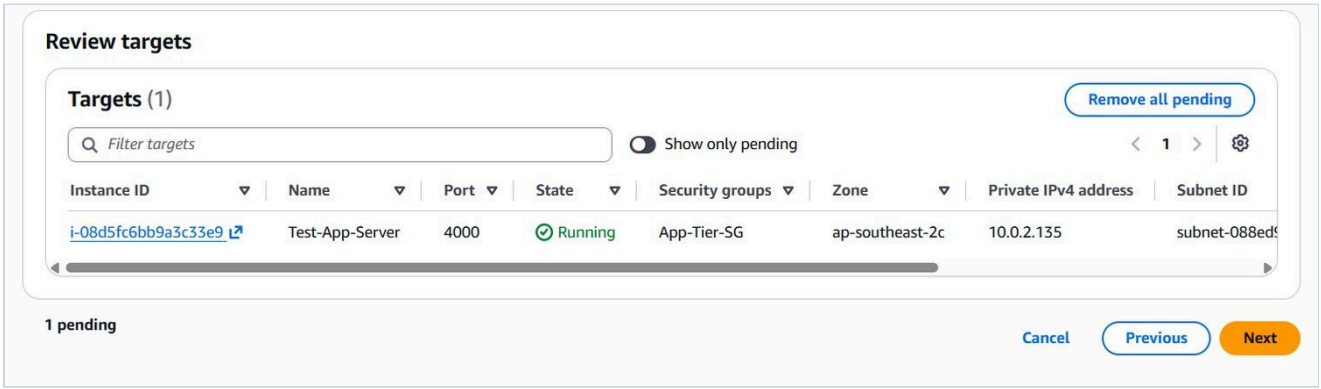


Figure 123. The pending target is registered.

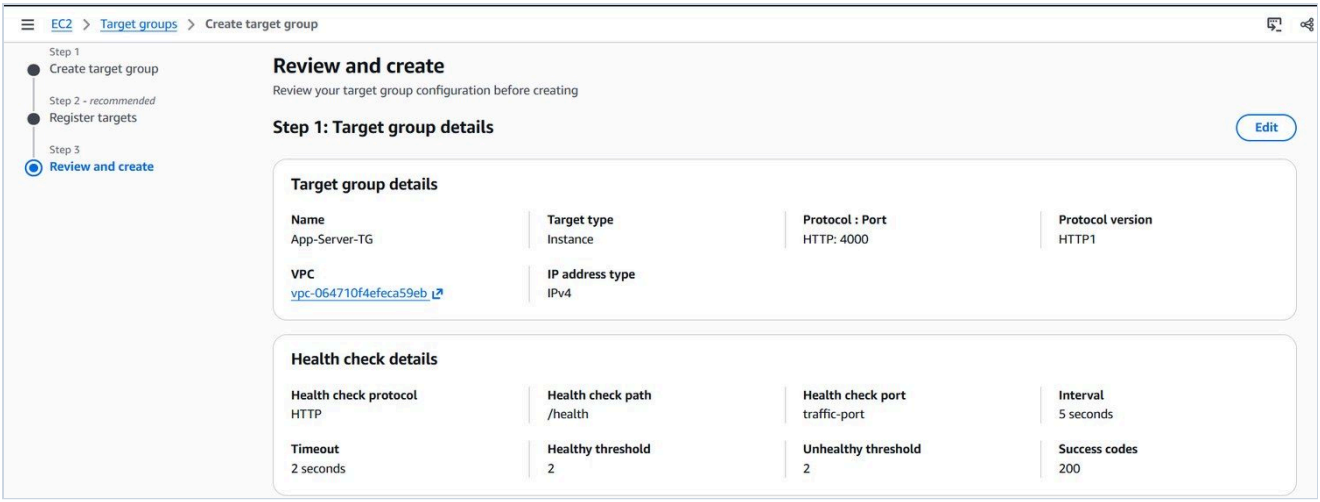


Figure 124. Review the target group details.

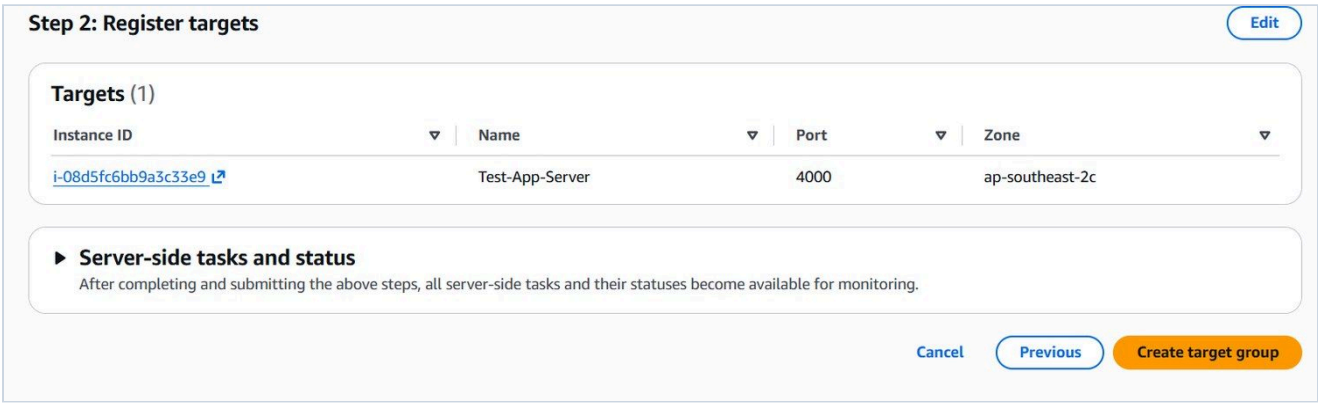


Figure 125. Targets summary, then Create target group.

B. Create the launch template

SETTING	VALUE
Name	`App-Server-Launch-Template`
Auto Scaling guidance	"Provide guidance to help me set up a template that I can use with EC2 Auto Scaling" (ticked)
AMI	My AMIs > Owned by me > App-server-image
Instance type	c7i-flex.large
Key pair / Subnet	Don't include in launch template (Auto Scaling chooses the subnets)
Security group	`App-Tier-SG`
IAM instance profile	`IAM-role-three-tier-app`

1 Go to **EC2 > Launch templates > Create launch template.**

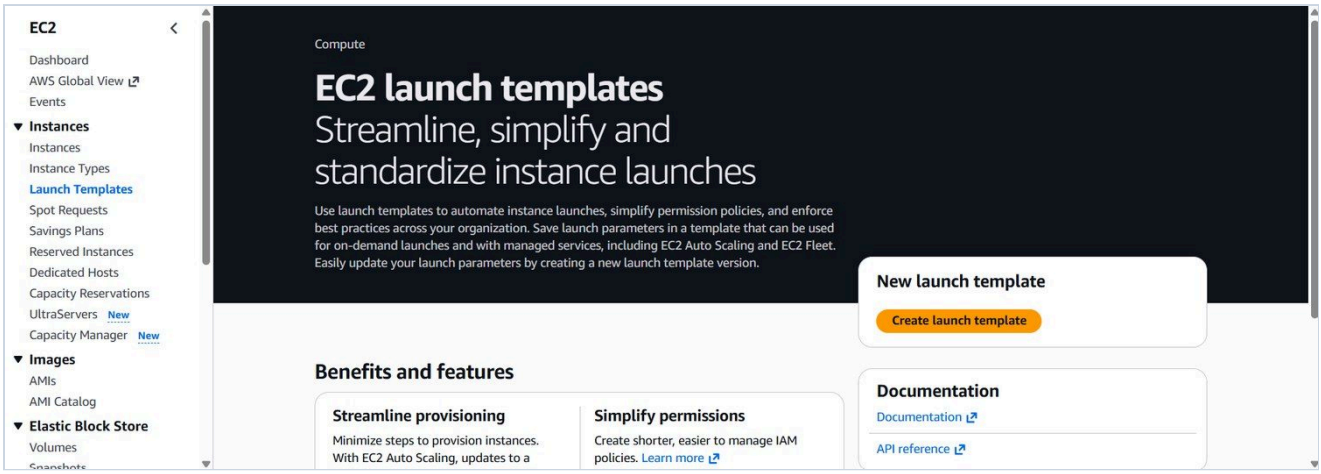


Figure 126. The Launch templates page.

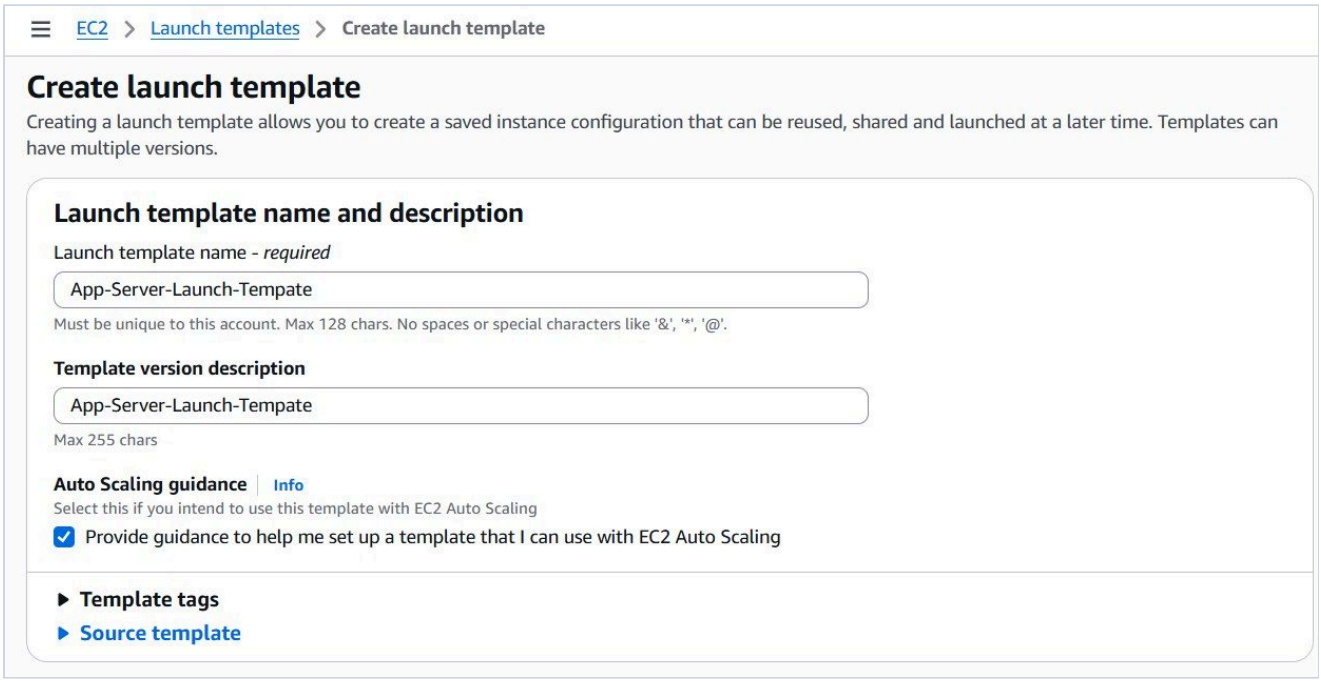


Figure 127. Launch template name and description.

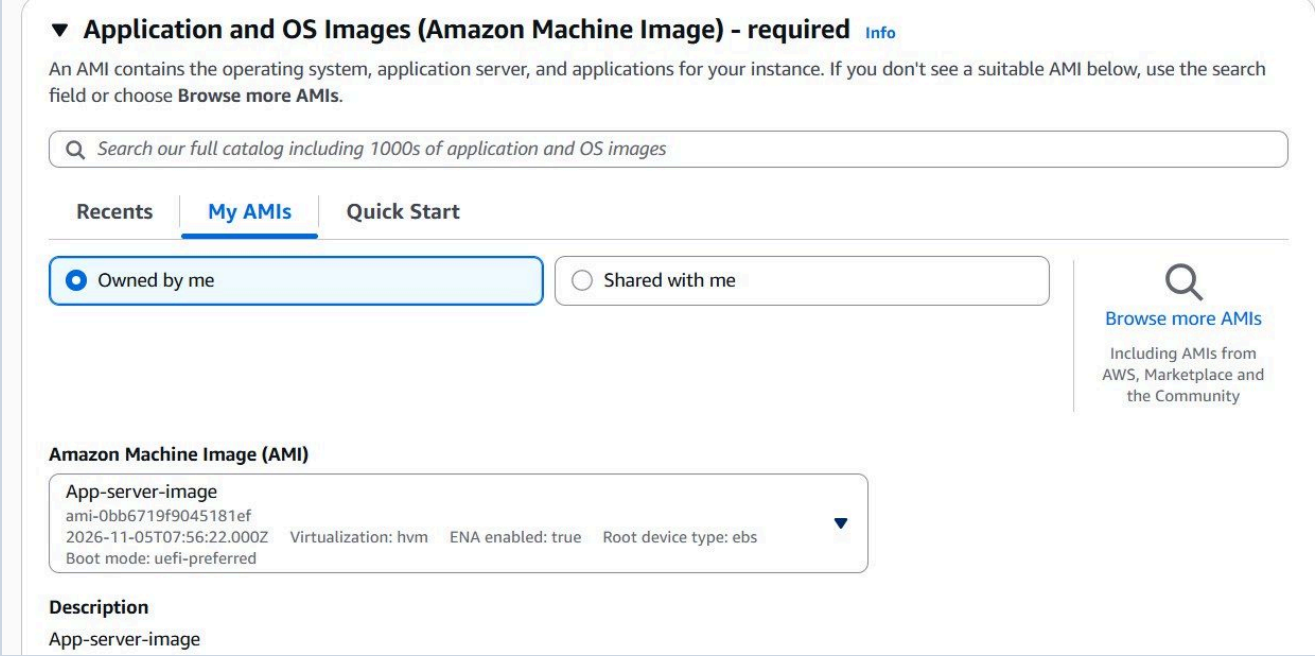


Figure 128. Selecting the App-server-image AMI.

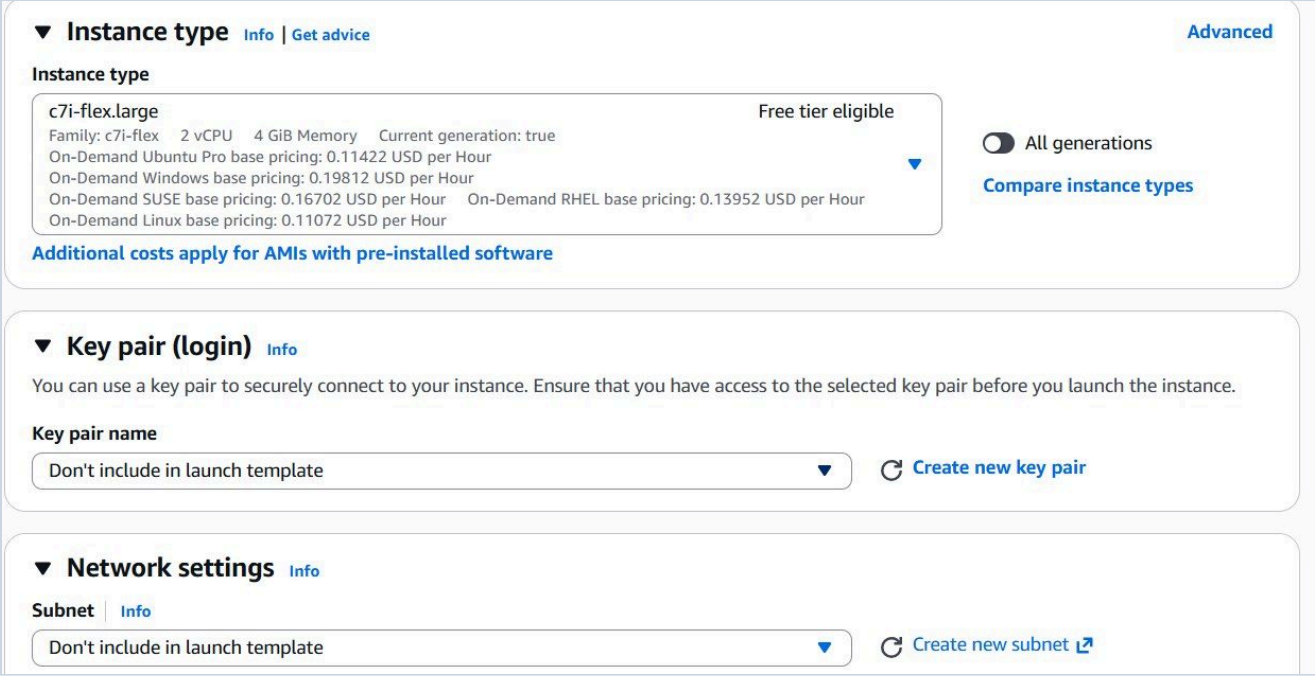


Figure 129. Instance type, key pair and network settings.

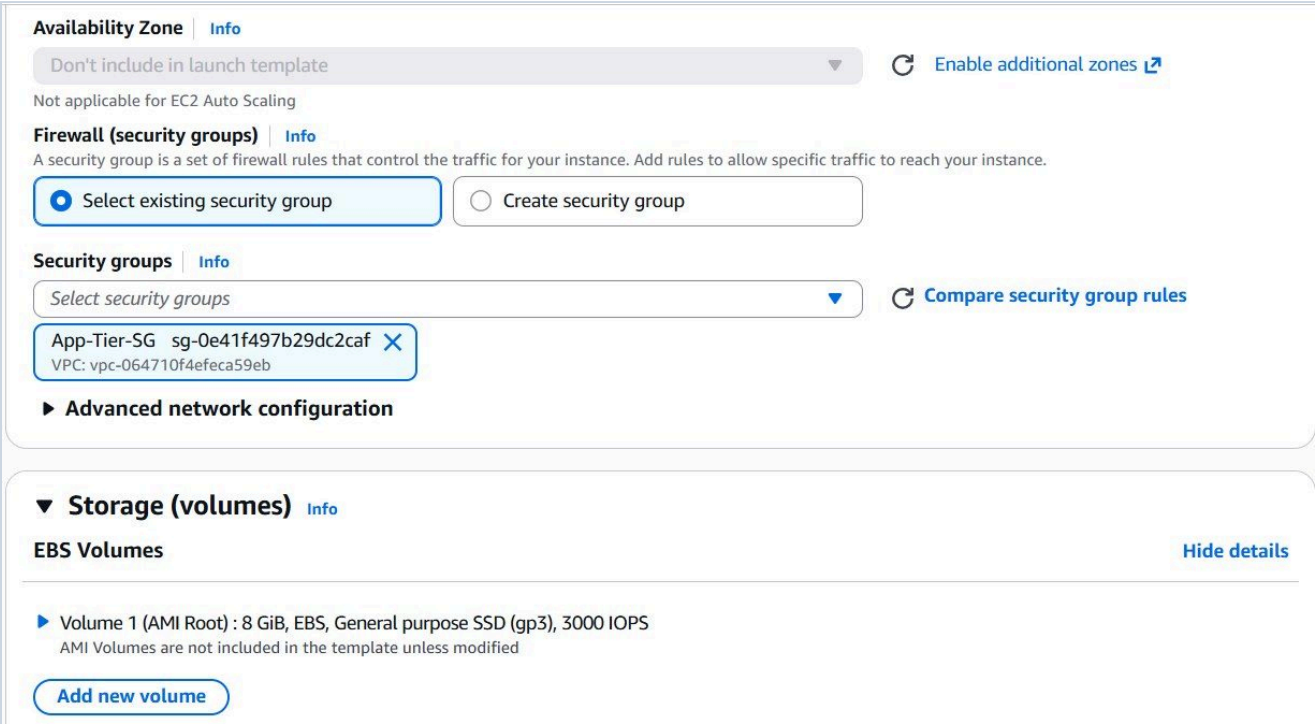


Figure 130. Security group App-Tier-SG.

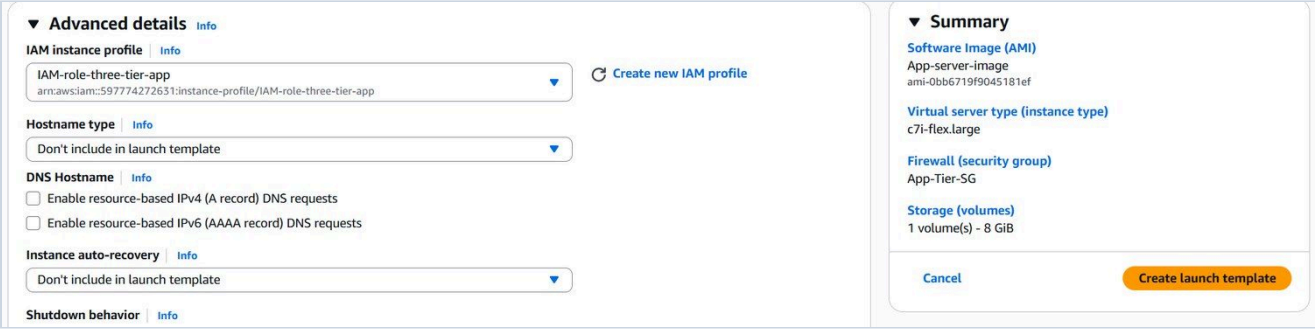


Figure 131. IAM instance profile, then Create launch template.

C. Create the internal Application Load Balancer

SETTING	VALUE
Type	Application Load Balancer
Name / Scheme	`Application-Load-Balancer` / Internal
VPC	Three-Tier-Project-VPC
Security group	`Internal-Load-Balancer-SG`
Listener	HTTP : 80 forwarding to App-Server-TG

1 Go to **EC2 > Load Balancers > Create load balancer** and choose **Application Load Balancer**.

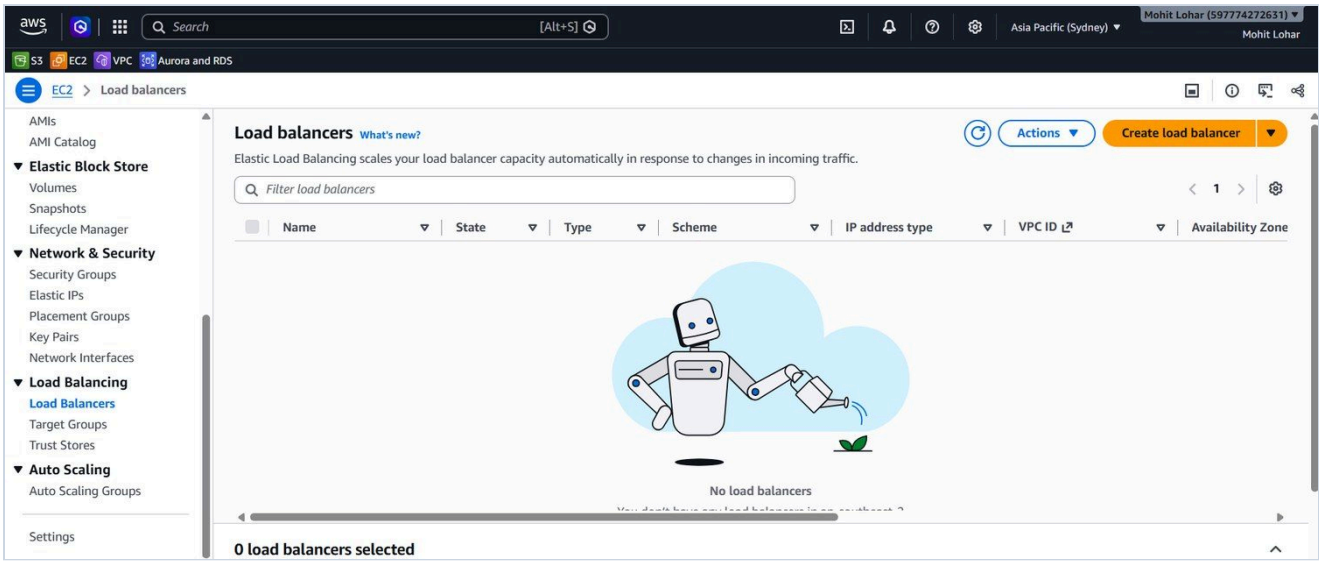


Figure 132. The Load balancers page (empty).

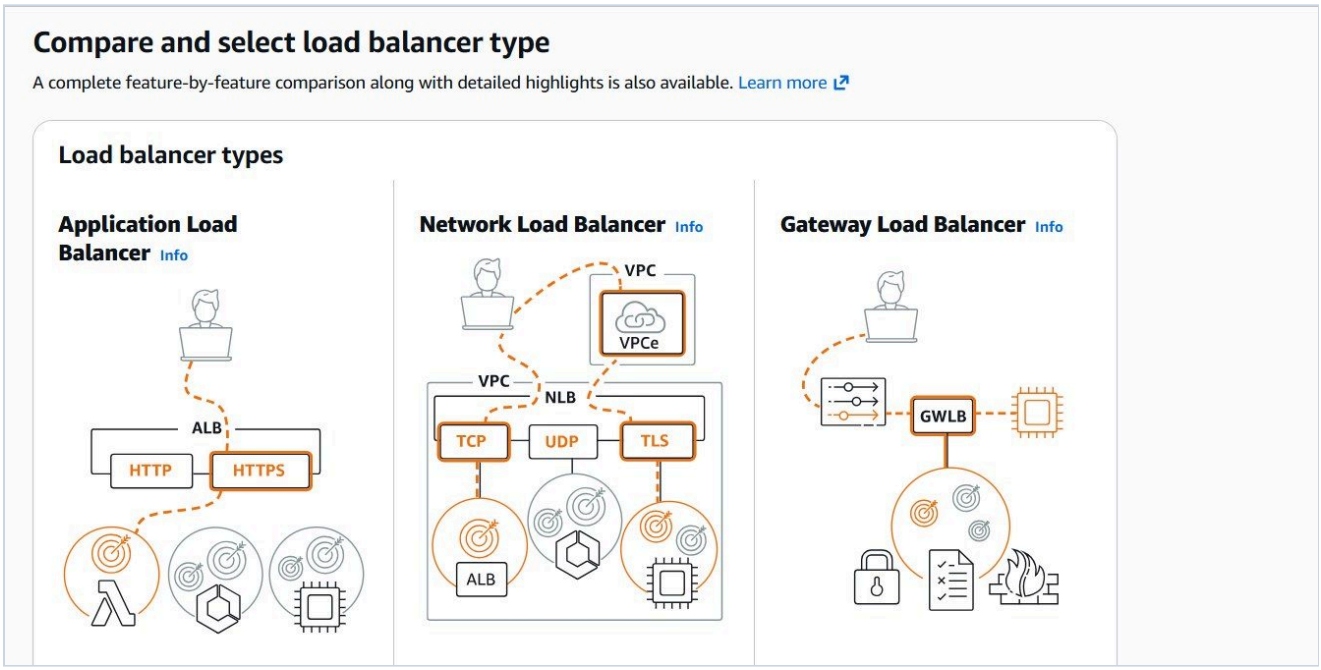


Figure 133. Load balancer types.

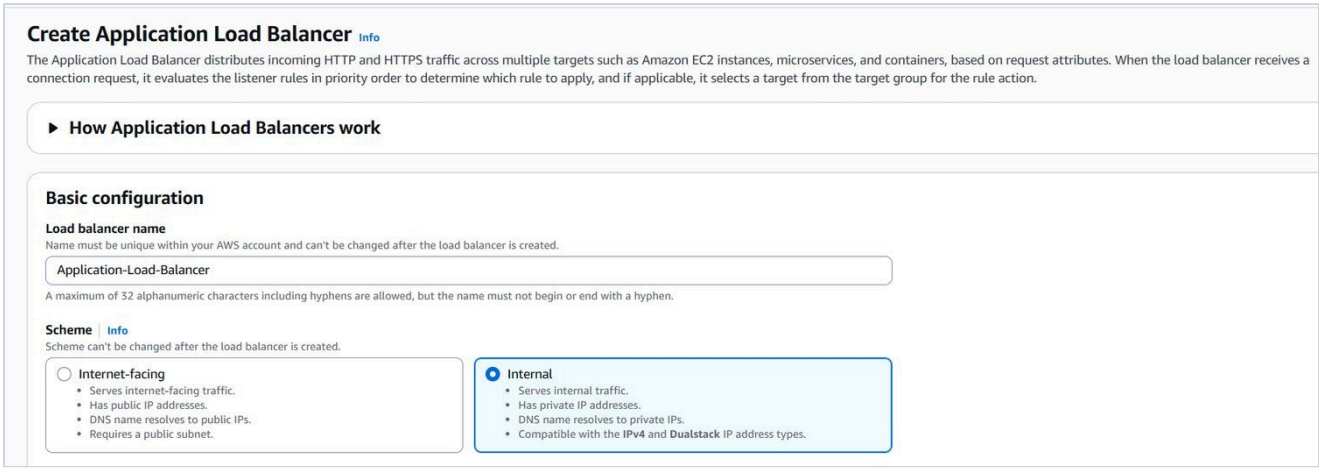


Figure 134. Name and the Internal scheme.

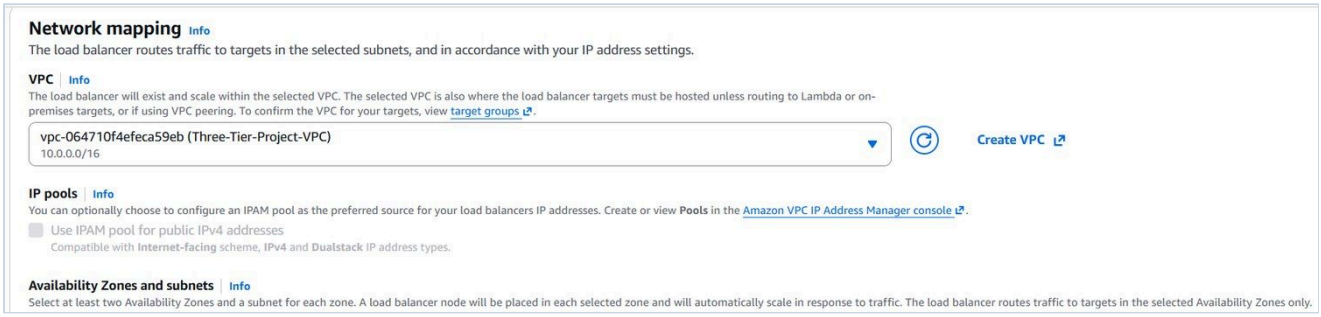


Figure 135. Network mapping: the VPC.

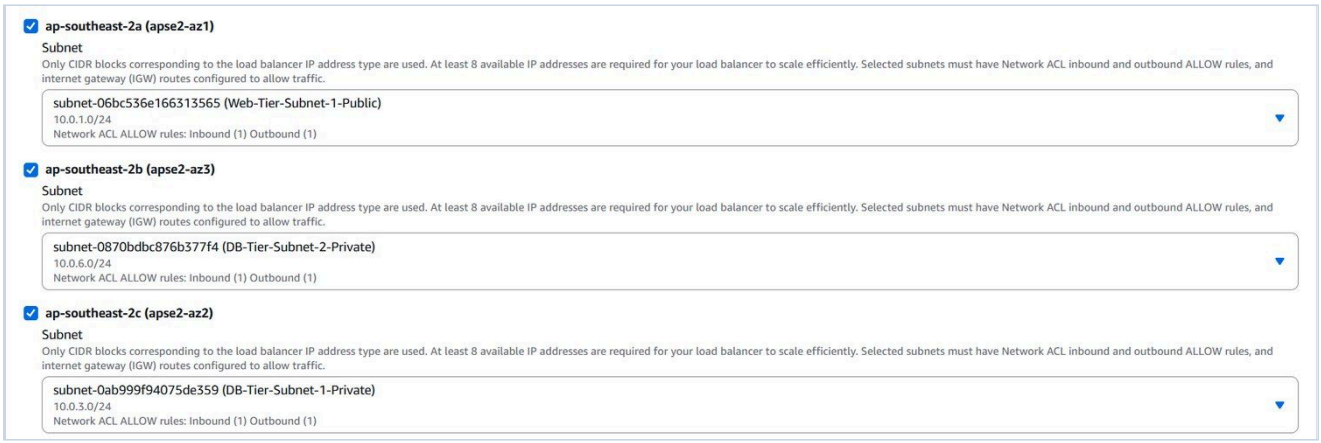


Figure 136. Availability Zones and subnets for the internal ALB.

IMPORTANT

As captured, the internal ALB was mapped to Web-Tier-Subnet-1-Public (2a), DB-Tier-Subnet-2-Private (2b) and DB-Tier-Subnet-1-Private (2c). The reference design maps it to the app-tier private subnets; prefer those when you reproduce this.

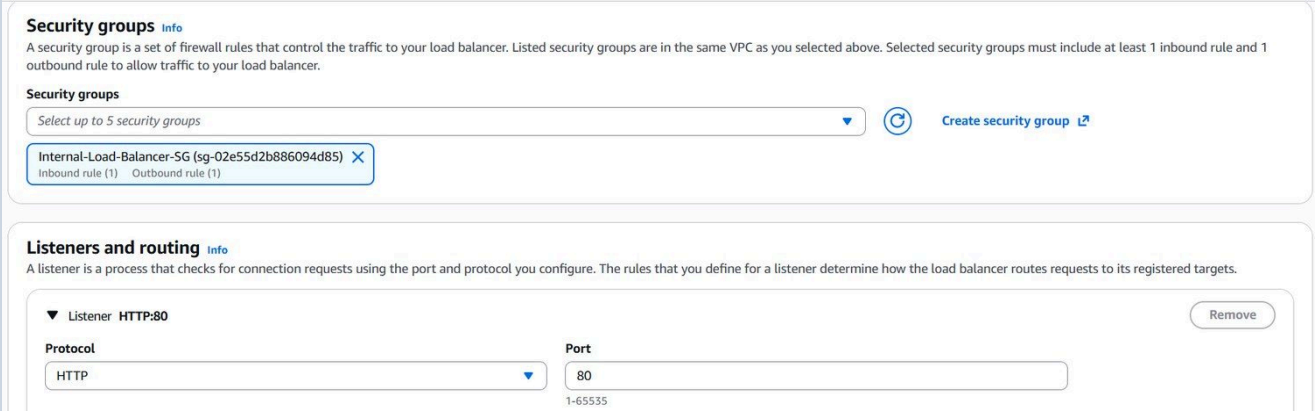


Figure 137. Security group and the HTTP:80 listener.

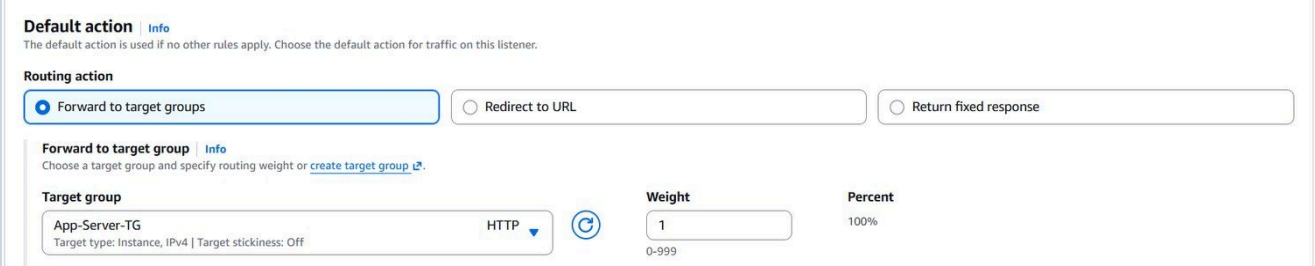


Figure 138. Listener default action: forward to App-Server-TG.

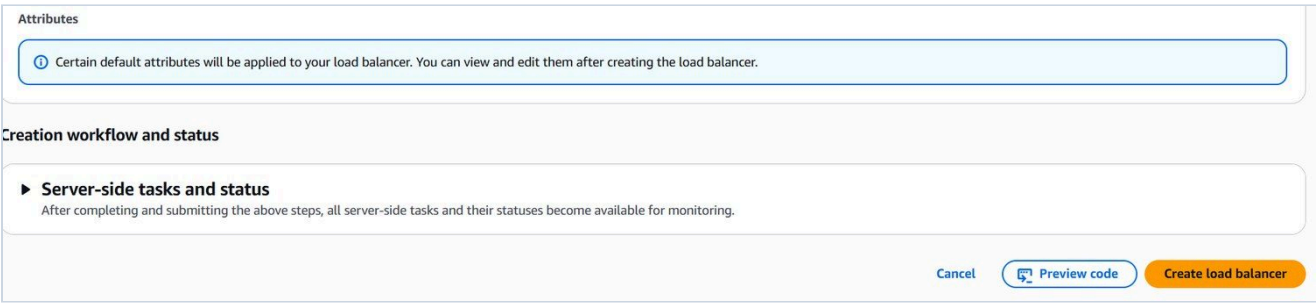


Figure 139. Review, then Create load balancer.

CHECKPOINT

The target group, launch template and internal load balancer exist; the load balancer becomes Active.

STEP 10 OF 15

SERVERS AND TRAFFIC

Create the Test Web Server and Set It Up

Build the web server once, by hand

WHY THIS STEP?

The web tier follows the same pattern as the app tier. This time the server runs Nginx and a React build, and Nginx forwards /api/ calls to the internal load balancer.

Launch the test web server

SETTING	VALUE
Name	`Test-Web-Server`
AMI	Amazon Linux 2023
Instance type	m7i-flex.large
Key pair	Proceed without a key pair
VPC / Subnet	Three-Tier-Project-VPC / Web-Tier-Subnet-1-Public (auto-assign public IP: Enable)
Security group	`Web-Tier-SG`
IAM instance profile	`IAM-role-three-tier-app`

1 Go to **EC2 > Launch instances** and use the settings above.

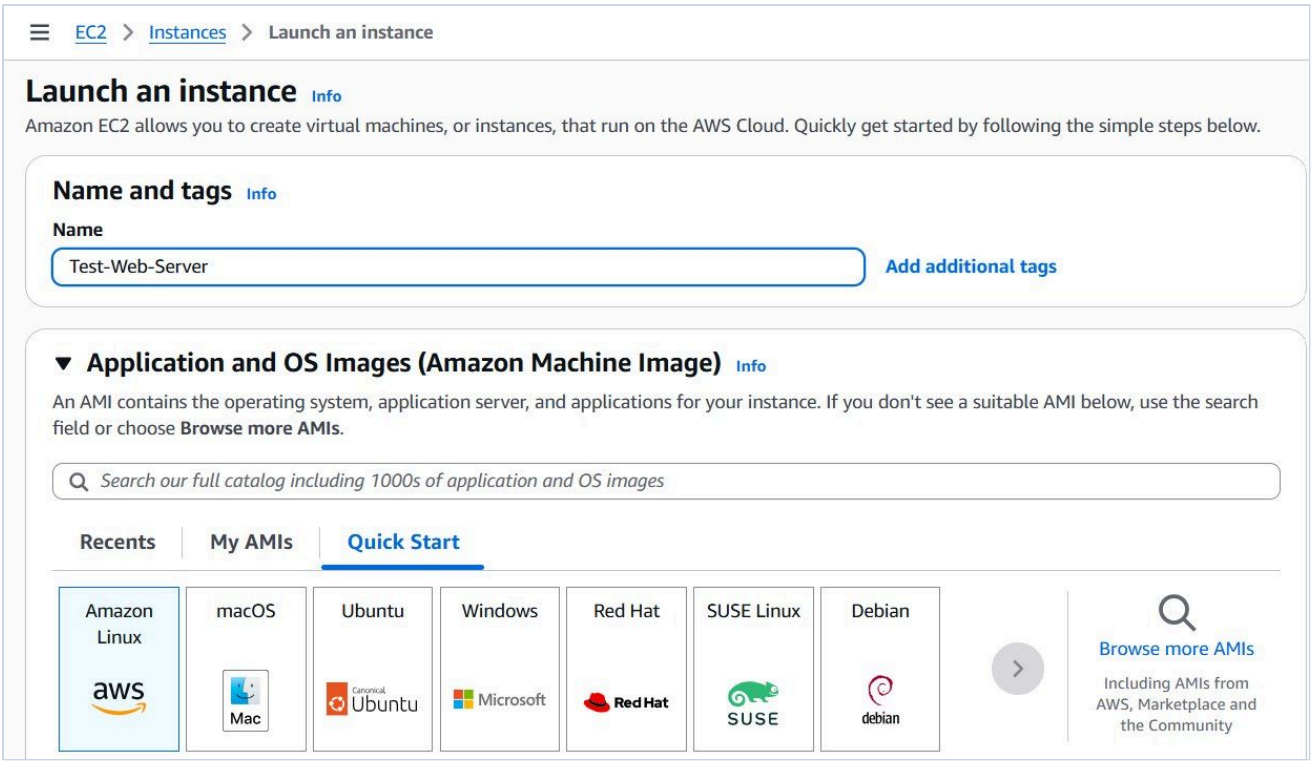


Figure 140. Name and AMI for the web test server.

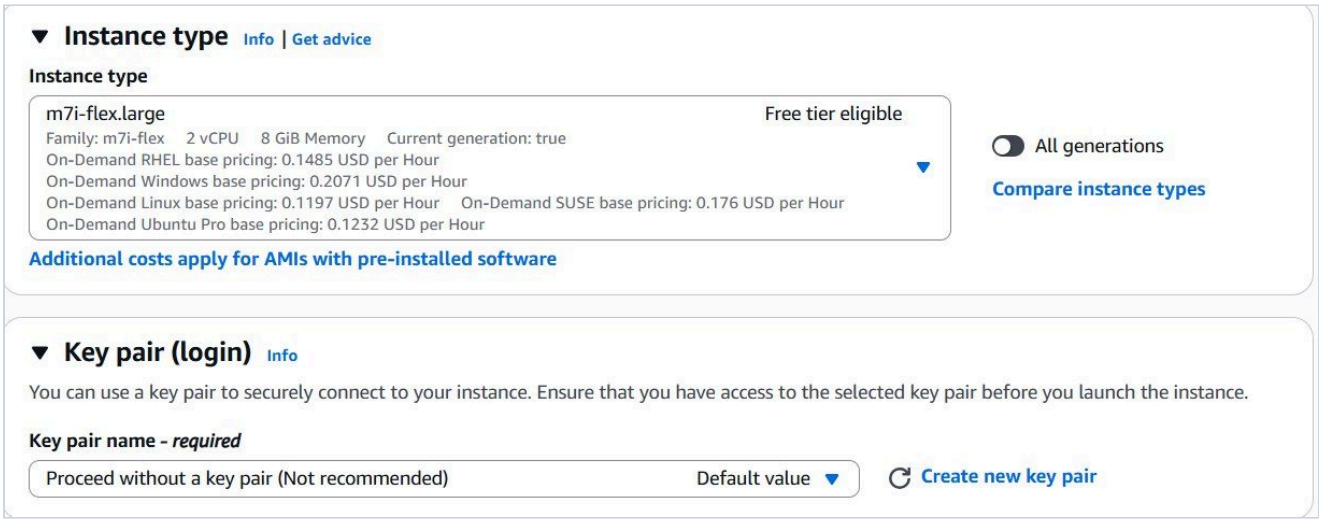


Figure 141. Instance type and key pair.

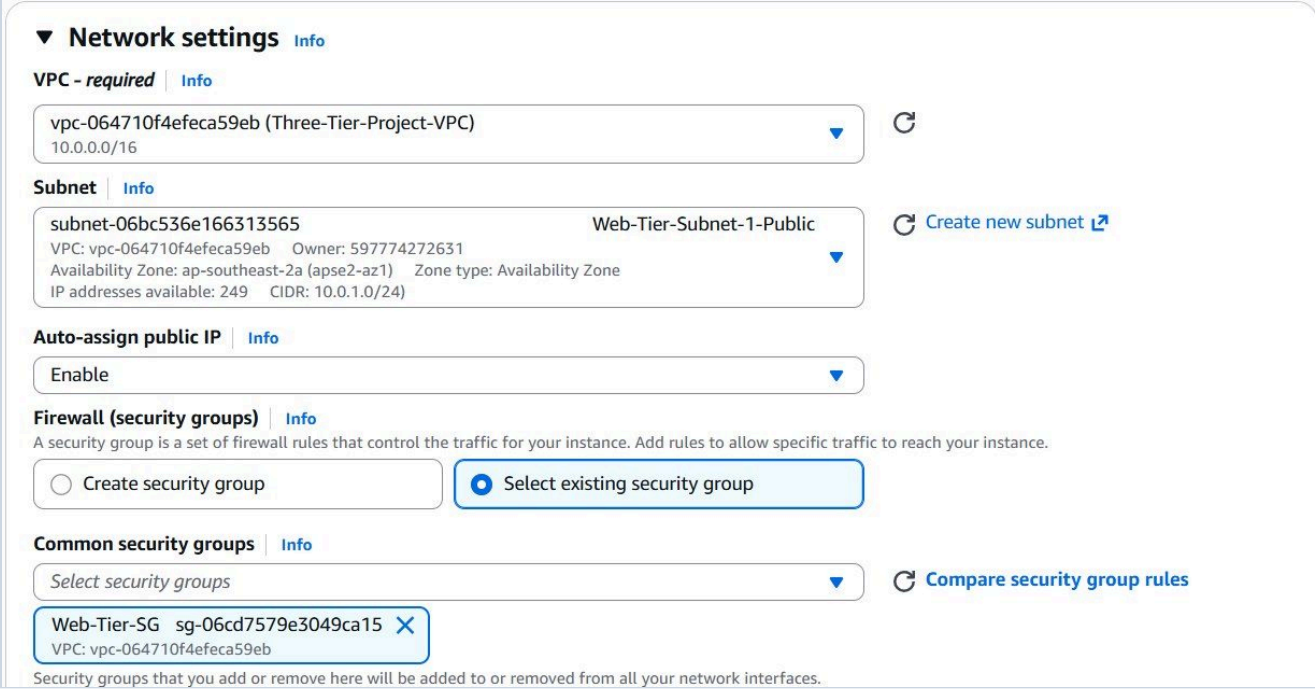


Figure 142. Network settings: Web-Tier-Subnet-1-Public and Web-Tier-SG.

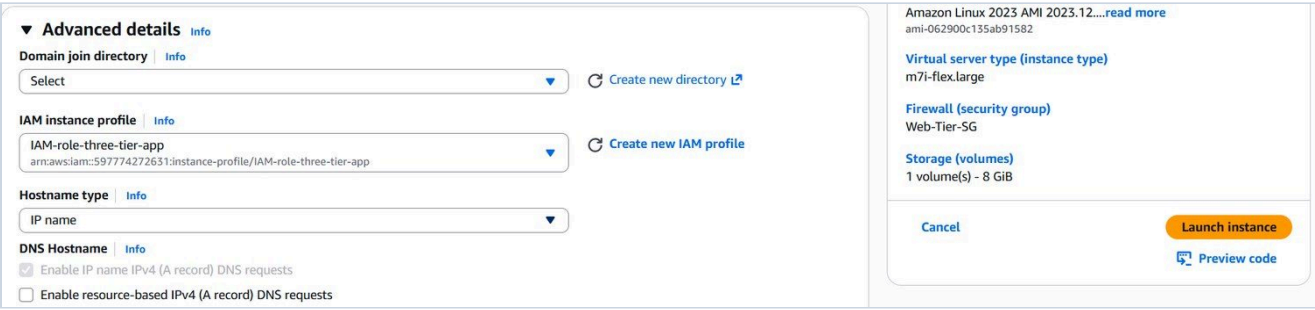


Figure 143. IAM instance profile, then Launch instance.

Point nginx.conf at the internal load balancer

1. In **EC2 > Load Balancers** select the internal load balancer and copy its **DNS name**.

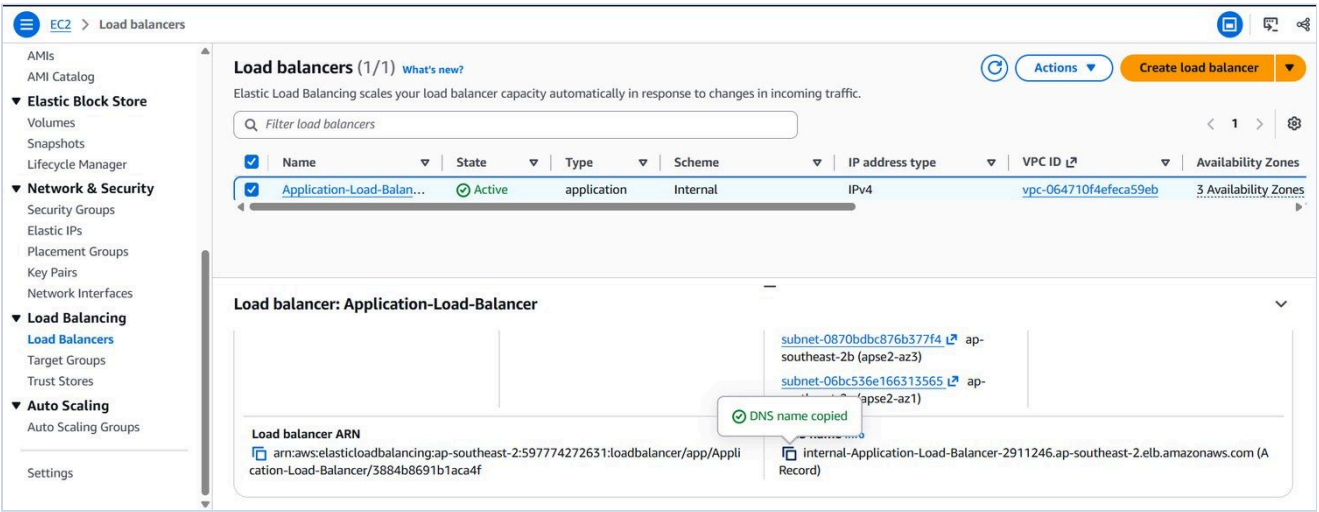


Figure 144. The DNS name of the internal ALB copied.

2. On your PC open `application-code/nginx.conf`, paste the DNS name into the `proxy_pass` line of the `/api/` location, and save.

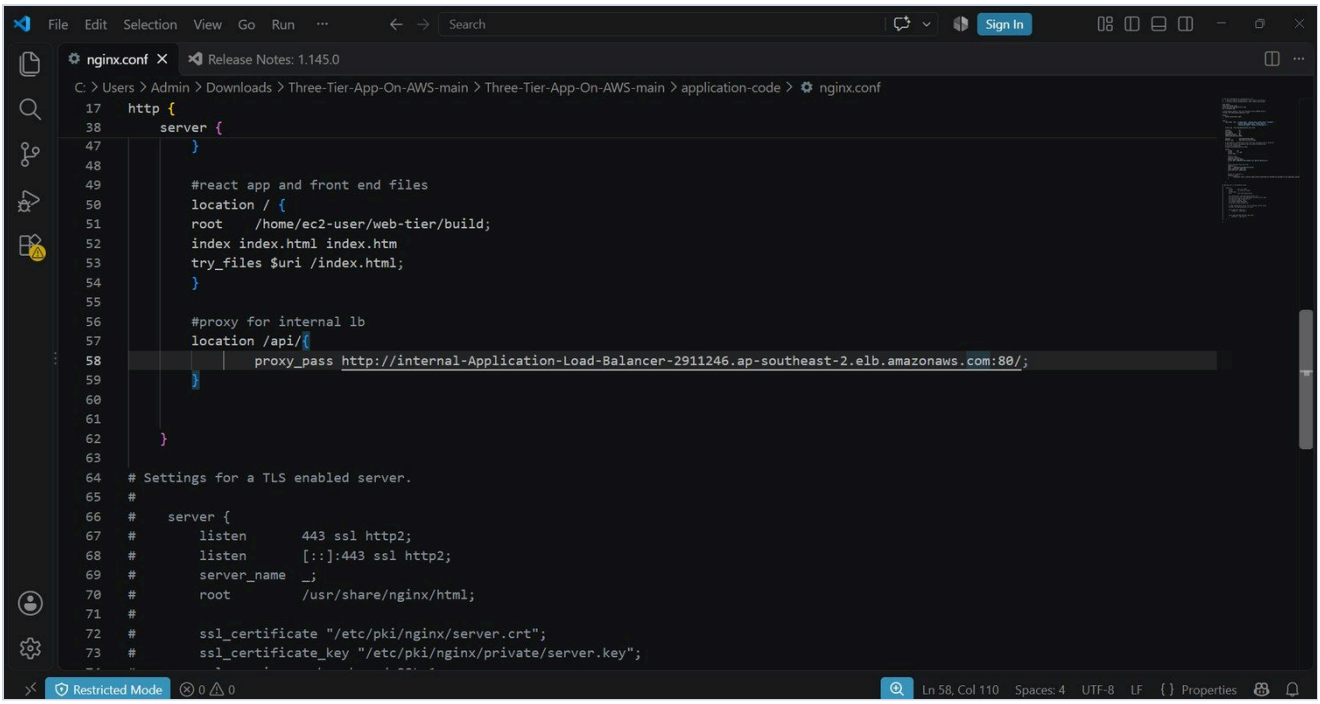


Figure 145. nginx.conf with the internal ALB DNS name in proxy_pass.

3. In S3 delete the old `nginx.conf` from `application-code/` and upload the edited one.

SCREENSHOT TO ADD

S3 console: replacing nginx.conf in application-code.

Configure the web server

1. Wait until `Test-Web-Server` shows **3/3 checks passed**, then select it and choose **Connect > Session Manager > Connect**.

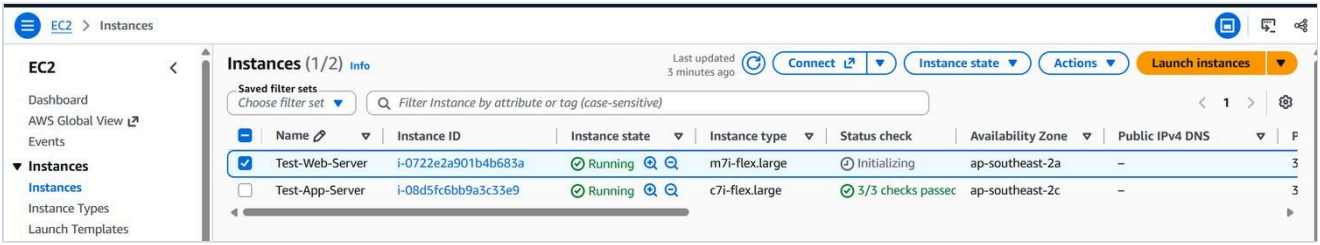


Figure 146. The instance list: Test-Web-Server is still initialising.

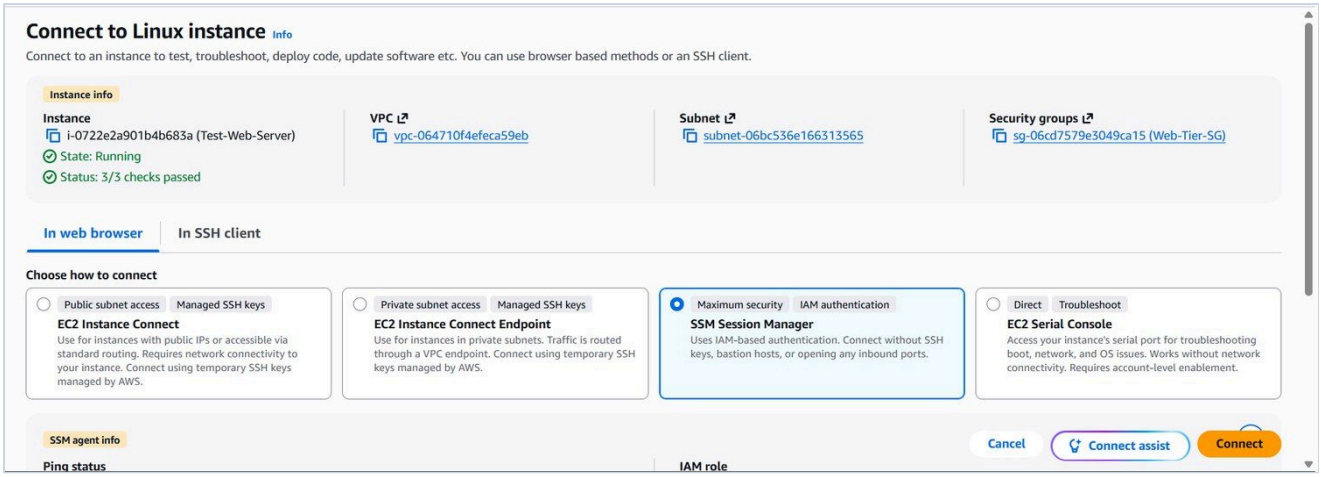


Figure 147. Connecting to Test-Web-Server through Session Manager.

1. Download the web-tier code

TERMINAL BASH

```
sudo -su ec2-user
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/web-tier web-tier --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user
sudo chmod -R 755 /home/ec2-user
```

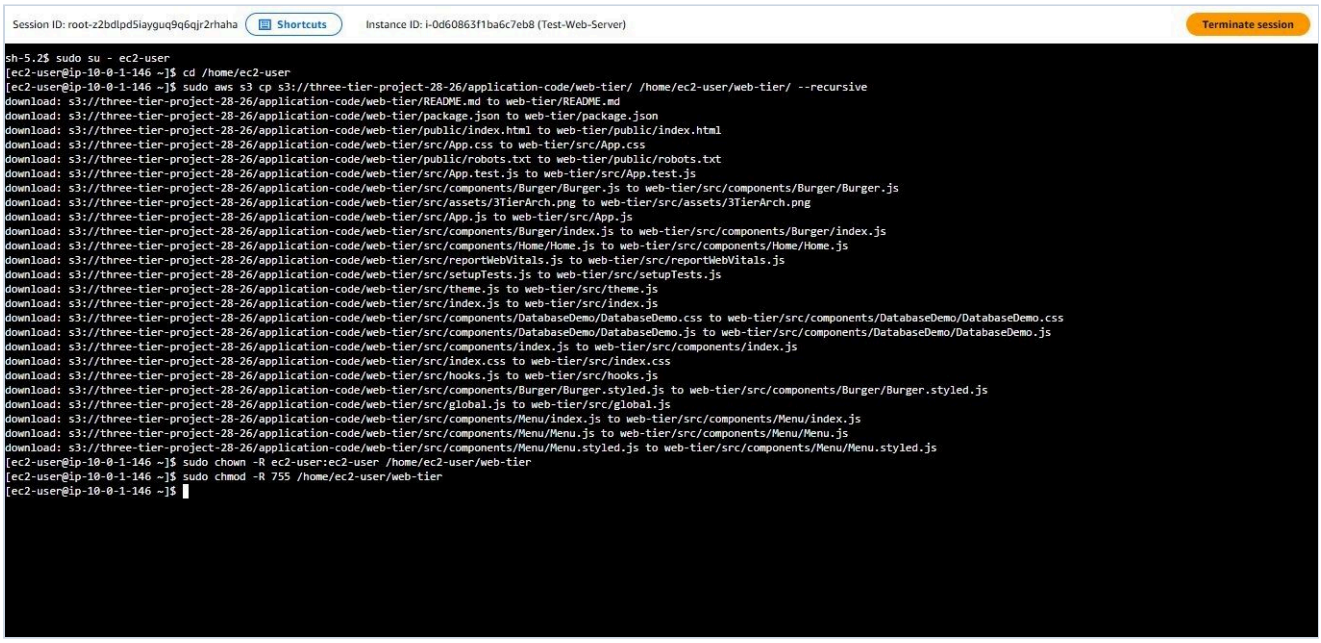


Figure 148. Reference screenshot from the source guide: Session Manager terminal on the web server while the web-tier files download from S3 and permissions are set.

2. Install Node.js and build the React app

TERMINAL BASH

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
cd /home/ec2-user/web-tier
npm install
npm run build
```

3. Install and configure Nginx

TERMINAL BASH

```
sudo yum install nginx -y
cd /etc/nginx
sudo mv nginx.conf nginx-backup.conf
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/nginx.conf .
sudo chmod -R 755 /home/ec2-user
sudo service nginx restart
sudo chkconfig nginx on
```

SCREENSHOT TO ADD

Terminal output of the web-tier commands (npm run build and the Nginx restart).

CHECKPOINT

The React build completes, Nginx restarts without errors and is enabled at boot.

STEP 11 OF 15 SERVERS AND TRAFFIC

Create an AMI of the Web Server

Freeze the web server too

WHY THIS STEP?

As with the app tier, an image of the configured web server lets Auto Scaling build identical web servers on demand.

1 Select `Test-Web-Server` and choose **Actions > Image and templates > Create image**.

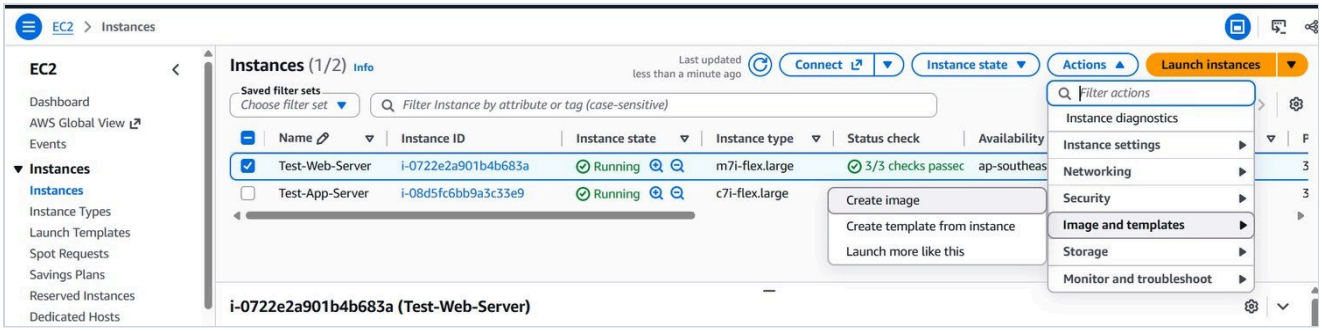


Figure 149. Actions > Image and templates > Create image.

2 Enter the image name and description `Web-Server-Image`, keep *Reboot instance* ticked.

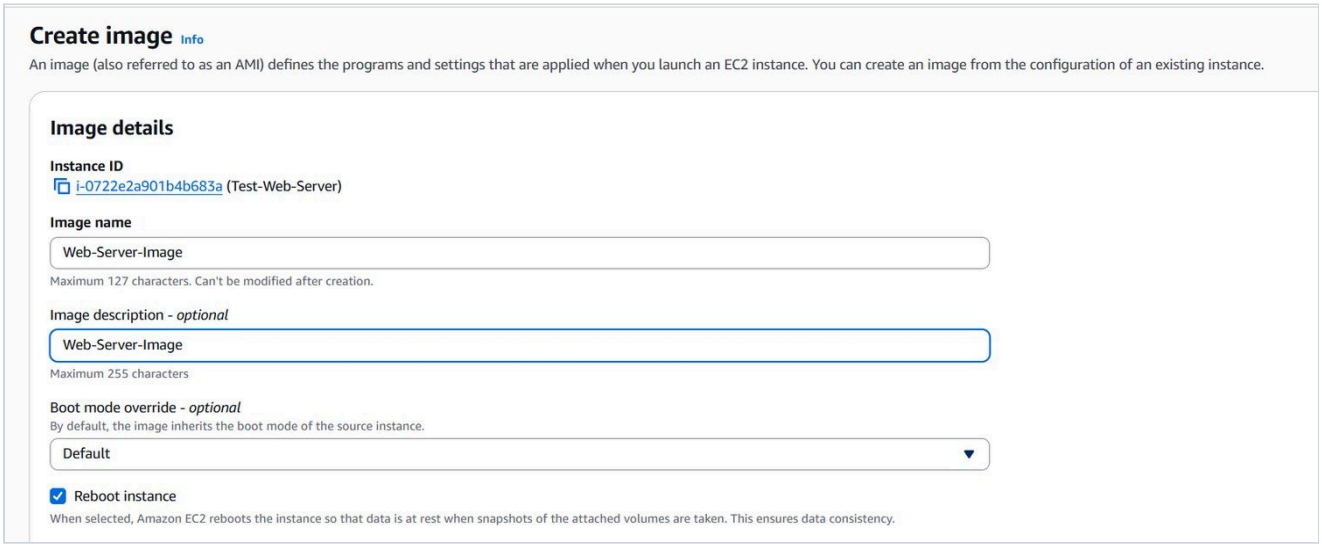


Figure 150. Web-Server-Image details.

3 Click **Create image** and wait for the AMI to become Available.

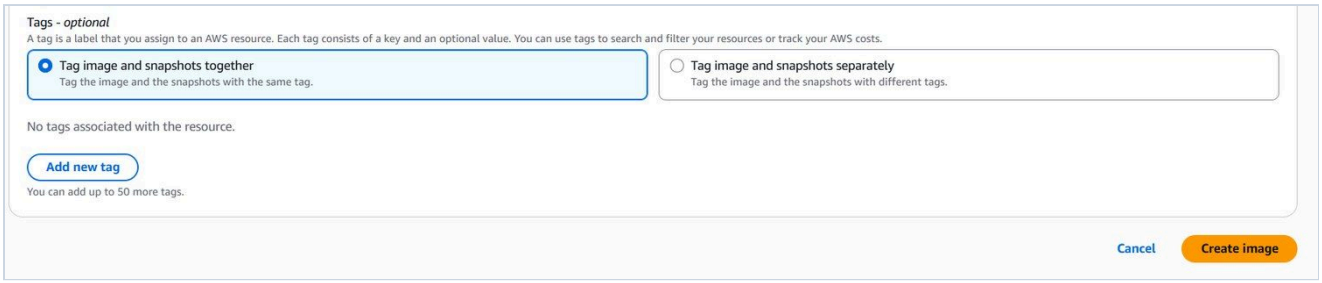


Figure 151. Create image.

CHECKPOINT

Both `App-server-image` and `Web-Server-Image` are Available under AMIs.

STEP 12 OF 15 SERVERS AND TRAFFIC

Target Group, Launch Template and External Load Balancer (Web Tier)

Open the app to the internet

WHY THIS STEP?

This repeats Step 9 for the web tier. The one big difference is that the load balancer here is internet-facing and sits in the public subnets.

A. Create the target group (web tier)

SETTING	VALUE
Target type / Name	Instances / `Web-Server-TG`
Protocol : Port	HTTP : 80
VPC	Three-Tier-Project-VPC
Health check path	`/health`
Advanced health checks	Traffic port; healthy 2, unhealthy 2, timeout 2 s, interval 5 s

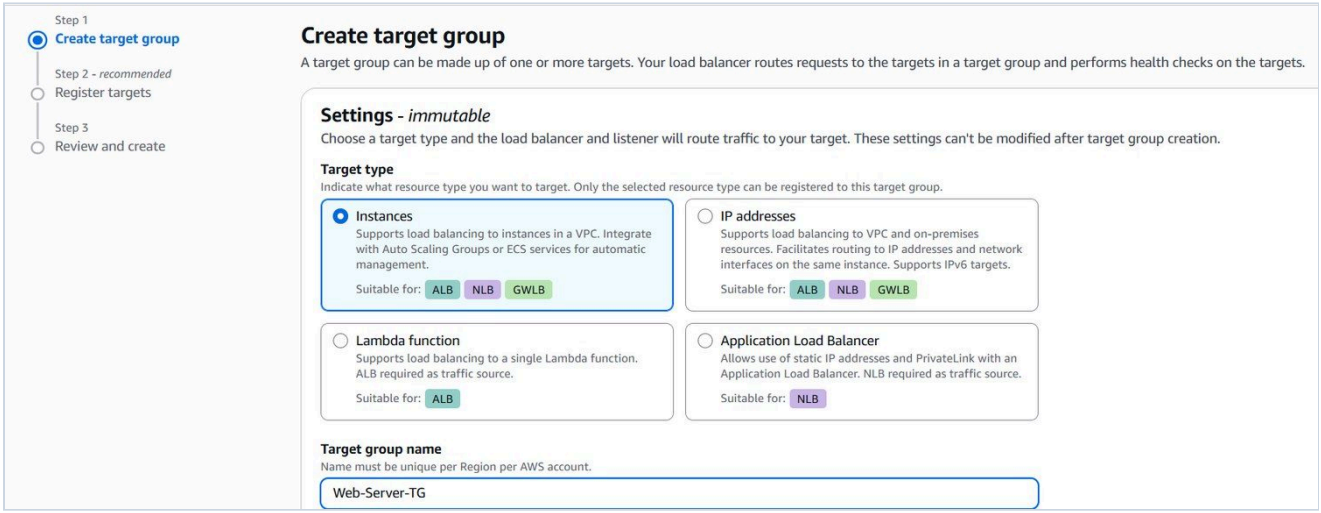


Figure 152. Target group name and type.

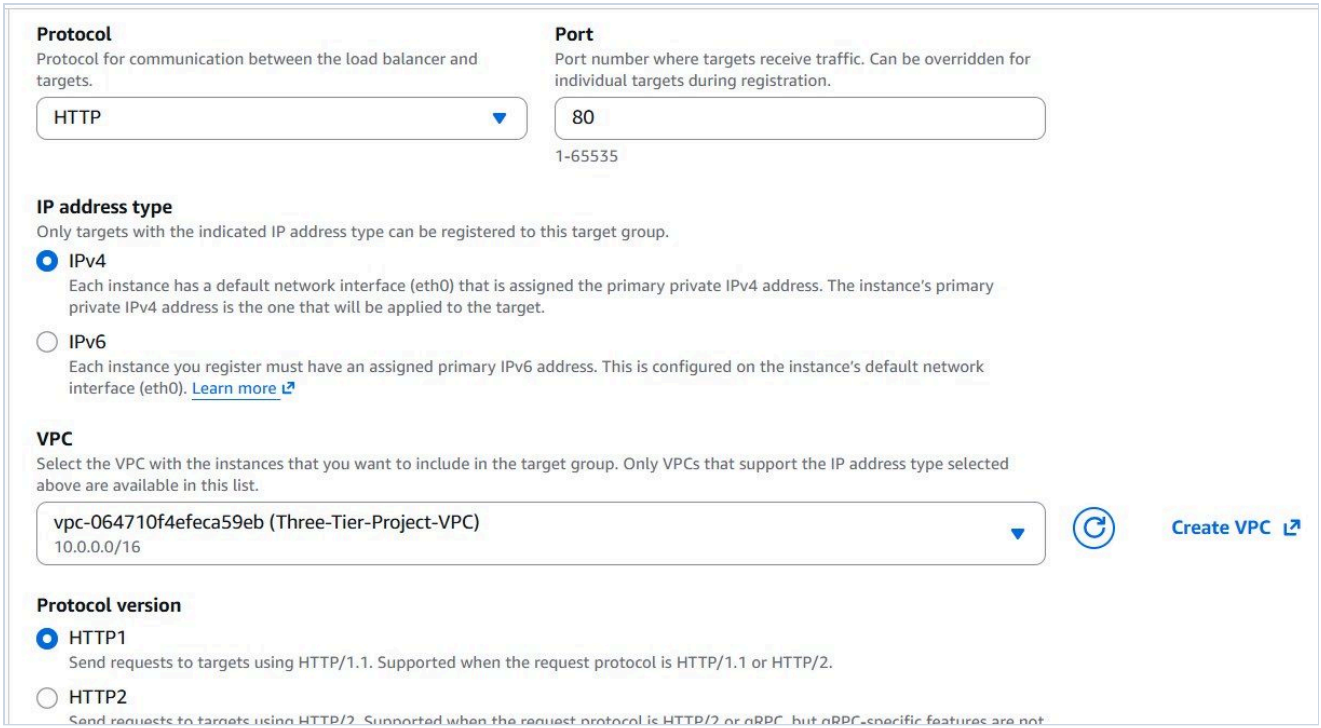


Figure 153. Protocol HTTP, port 80 and the VPC.

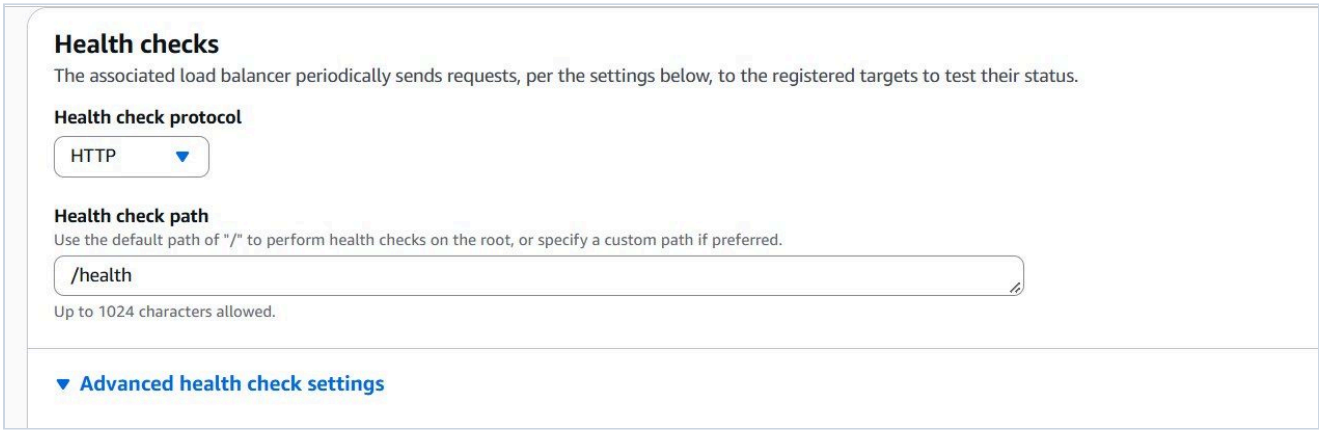


Figure 154. Health check settings.

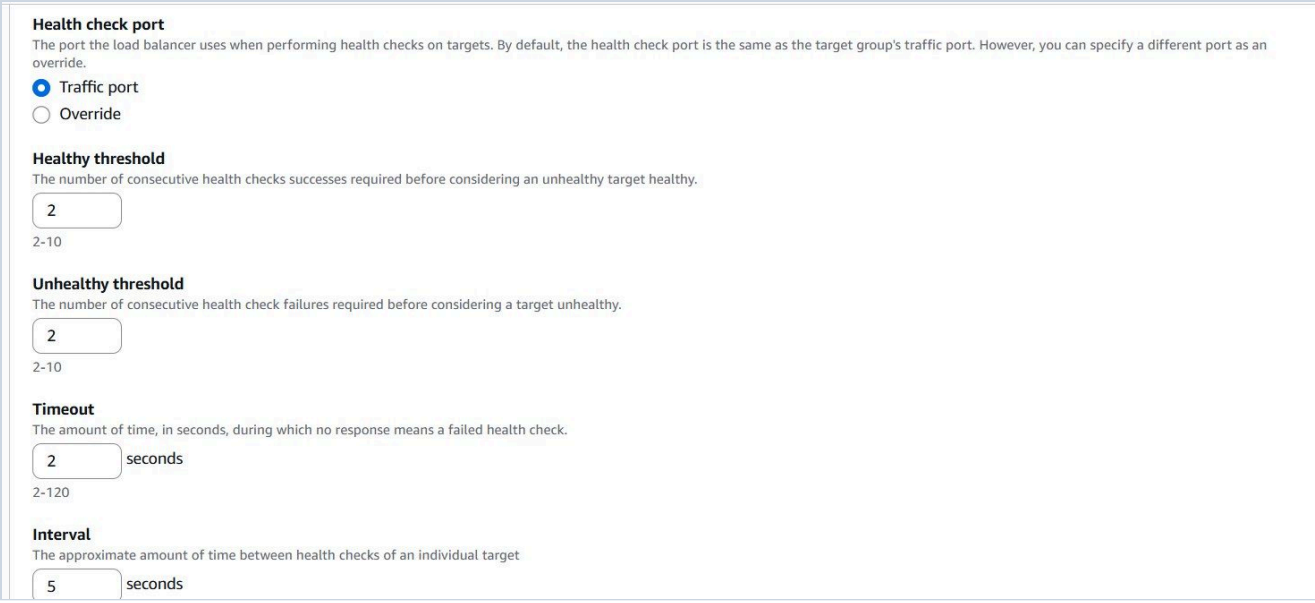


Figure 155. Advanced health-check settings.

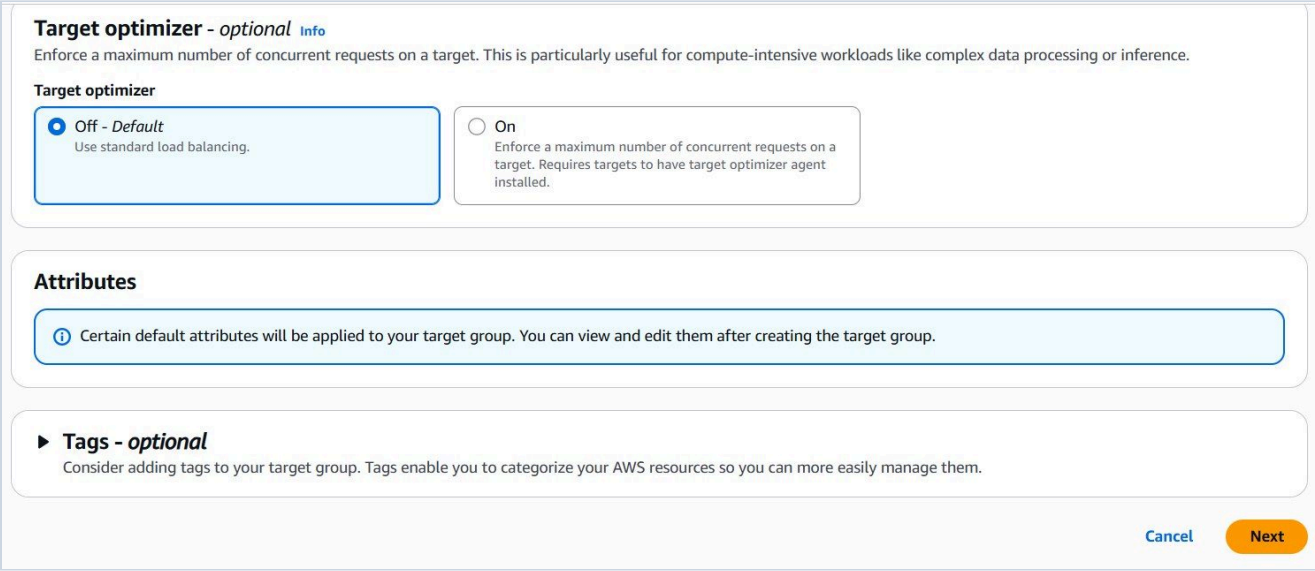


Figure 156. Target optimizer: click Next.

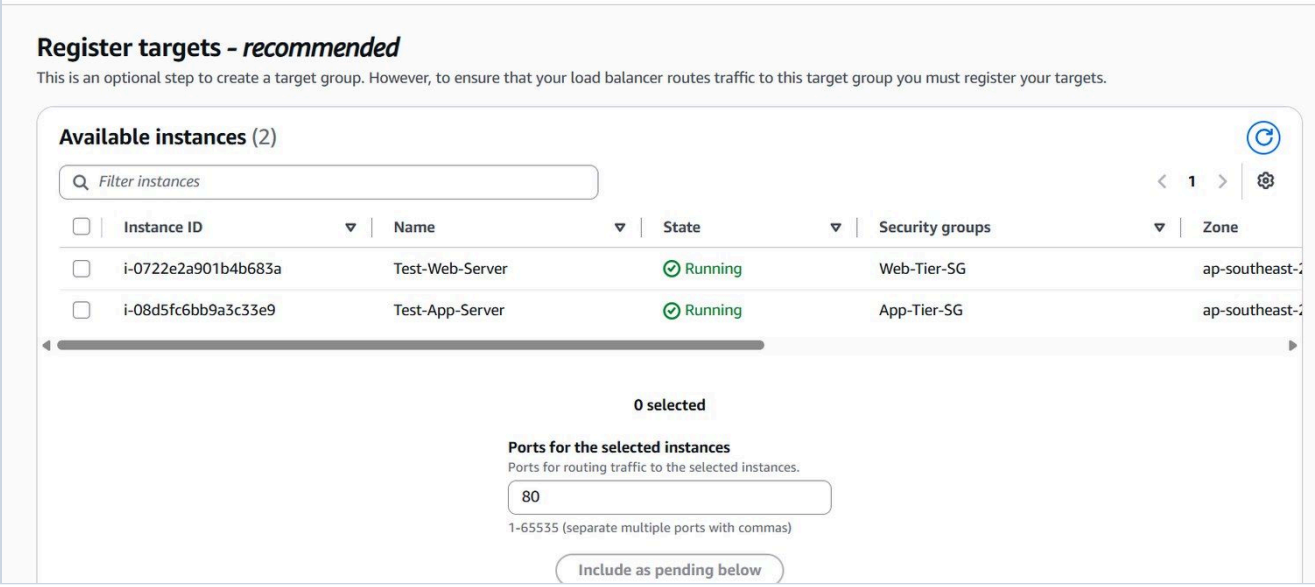


Figure 157. Register targets on port 80.

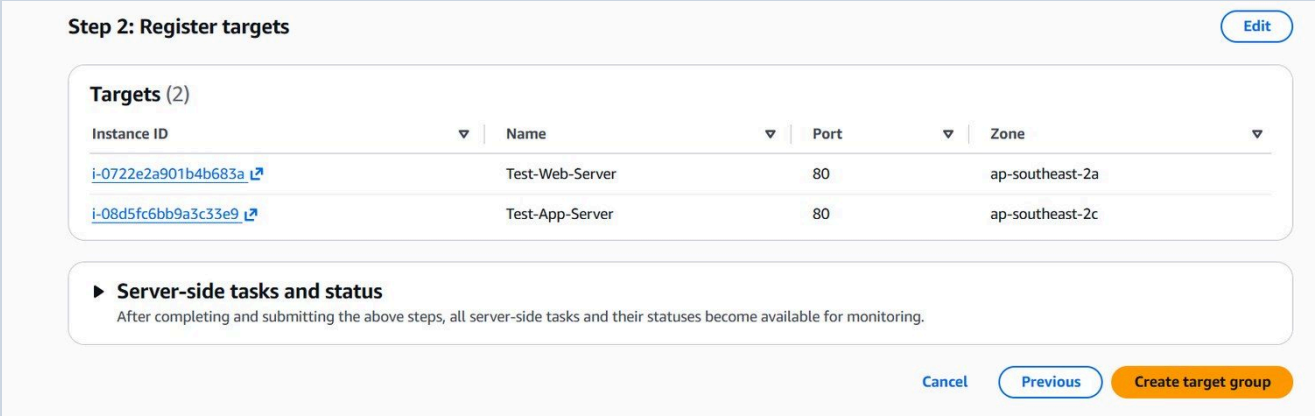


Figure 158. Review the targets, then Create target group.

> **NOTE**
> Registering targets here is optional because the Auto Scaling group registers its own instances. Only Test-Web-Server is needed; the screenshot shows both test instances selected.

B. Create the launch template (web tier)

SETTING	VALUE
Name	`Web-Server-Launch-Template`
AMI	My AMIs > Web-Server-Image
Instance type	c7i-flex.large
Key pair / Subnet	Don’t include in launch template
Security group	`Web-Tier-SG`
IAM instance profile	`IAM-role-three-tier-app`

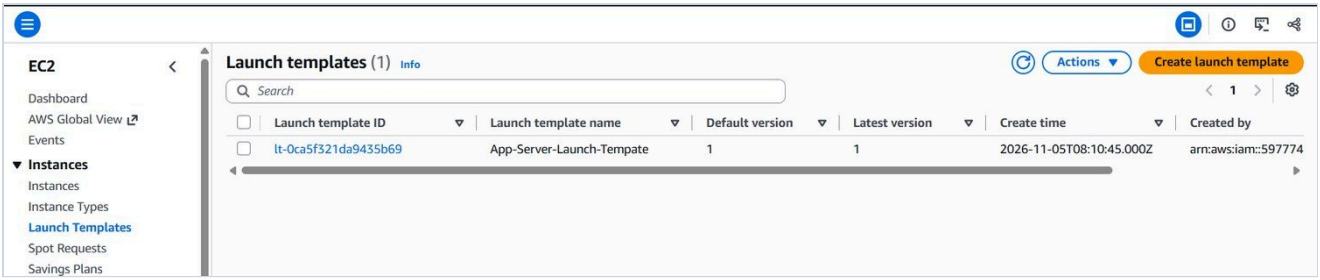


Figure 159. The Launch templates list (the app template already exists).

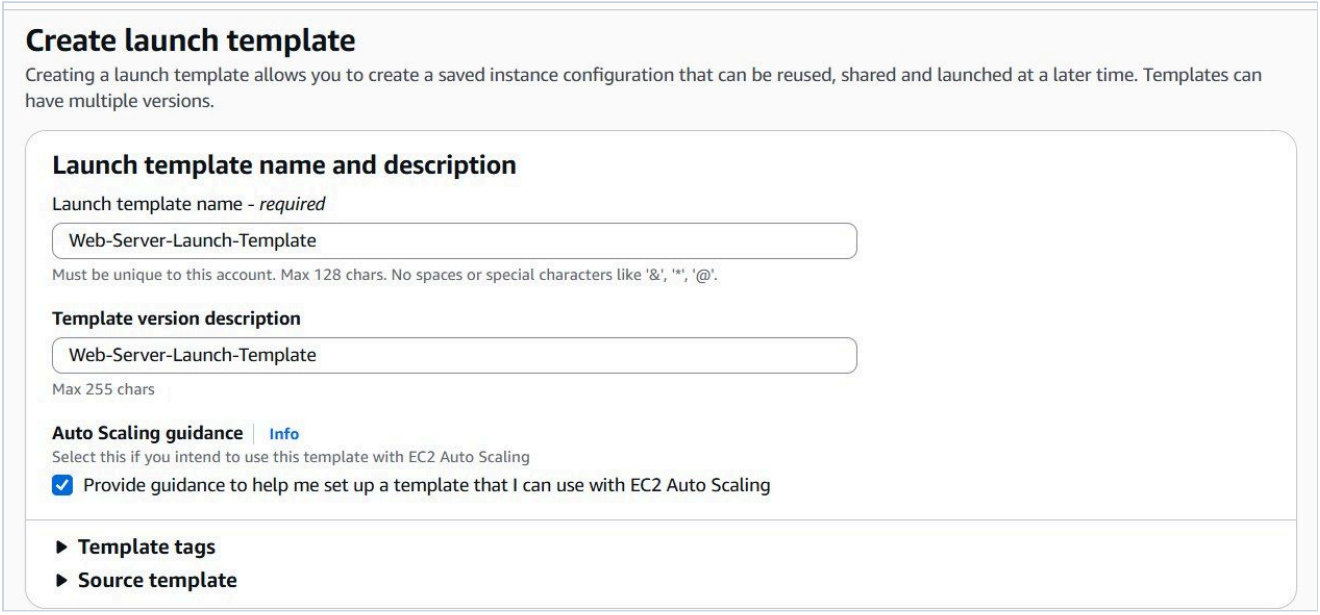


Figure 160. Launch template name and description.

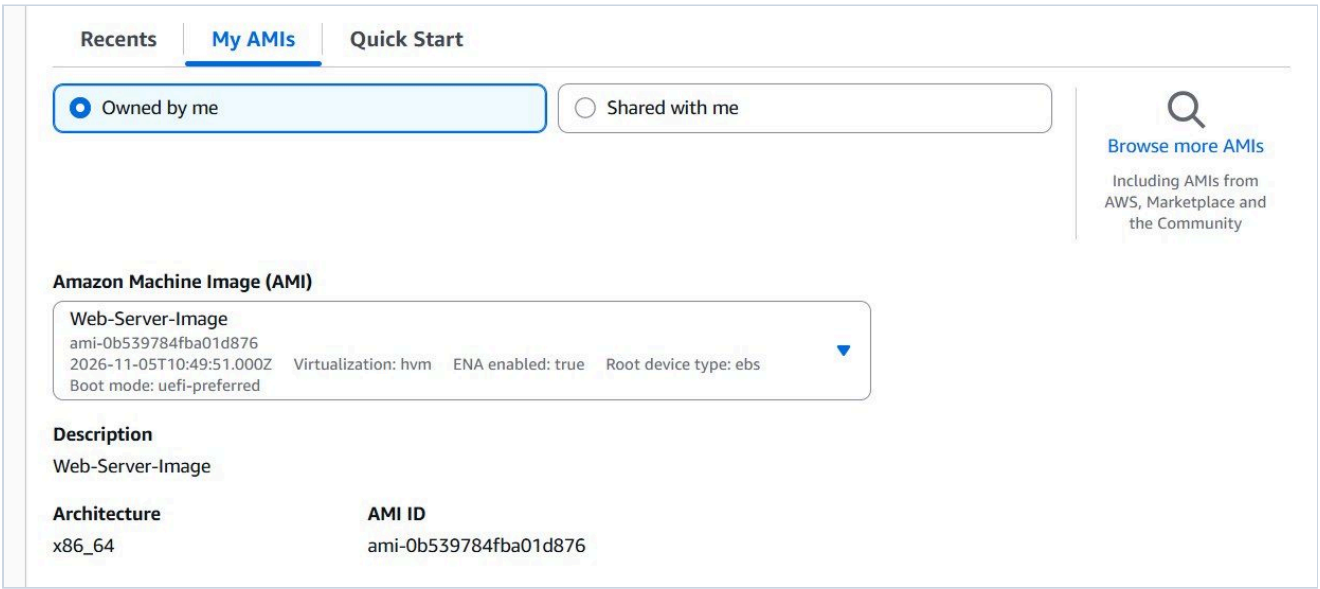


Figure 161. Selecting Web-Server-Image.

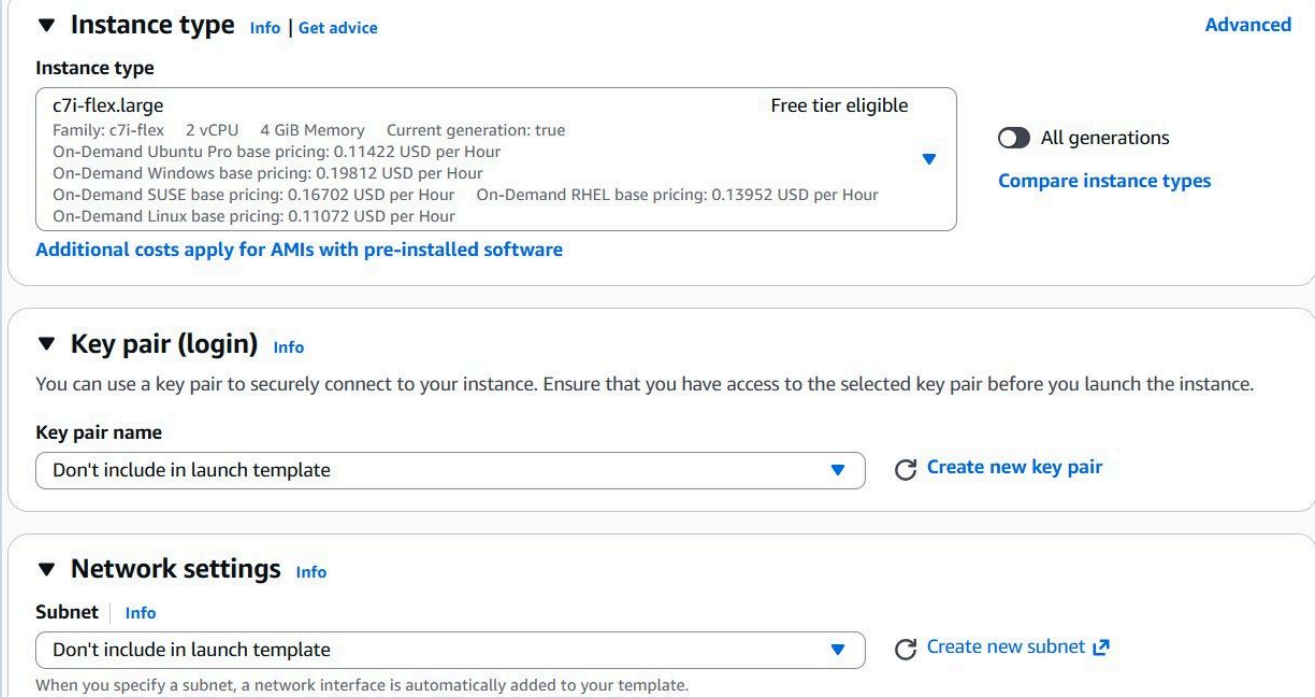


Figure 162. Instance type, key pair and network settings.

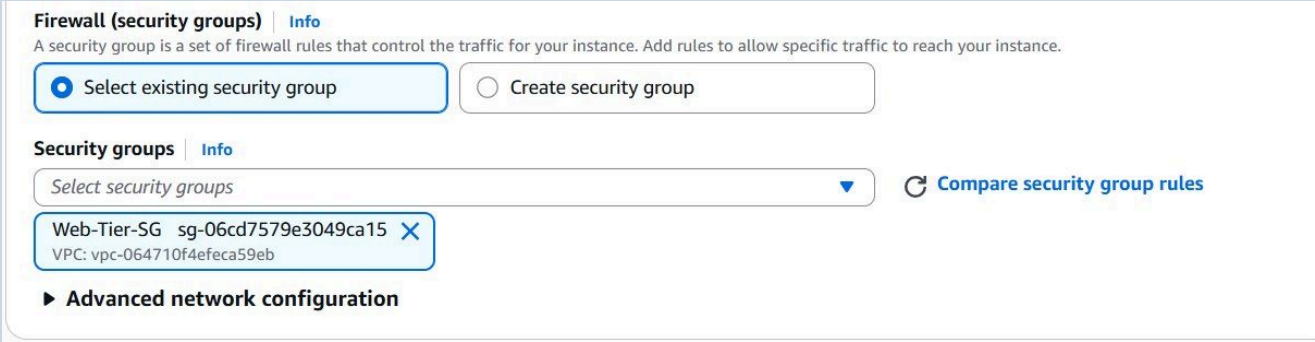


Figure 163. Security group Web-Tier-SG.

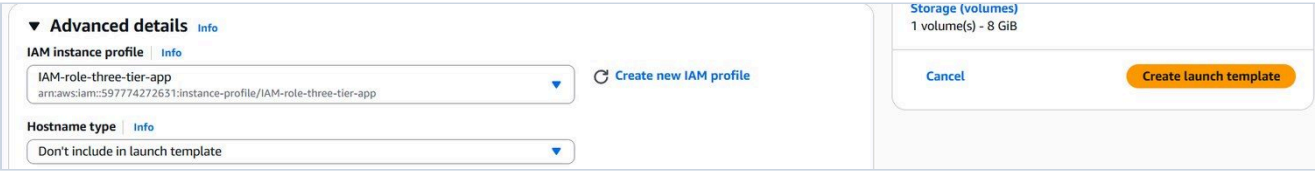


Figure 164. IAM instance profile, then Create launch template.

C. Create the external (internet-facing) load balancer

SETTING	VALUE
Type / Name	Application Load Balancer / `External-Load-Balancer`
Scheme	Internet-facing
VPC / Subnets	Three-Tier-Project-VPC / Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public
Security group	`External-Load-Balancer-SG`

SETTING	VALUE
Listener	HTTP : 80 forwarding to Web-Server-TG

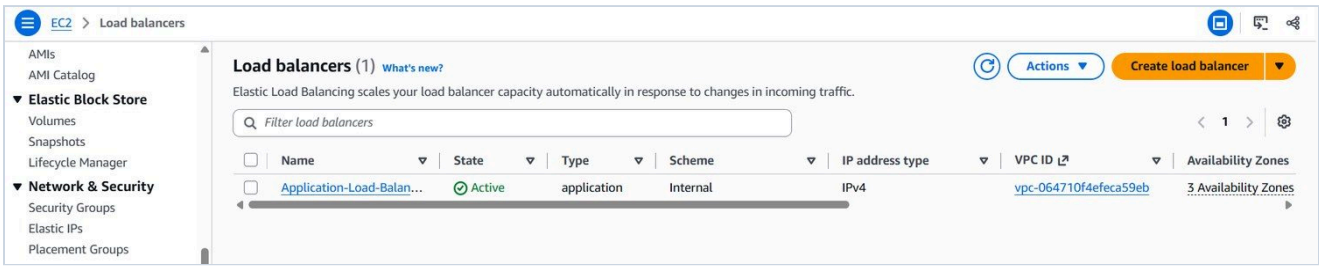


Figure 165. The Load balancers list (the internal ALB already exists).

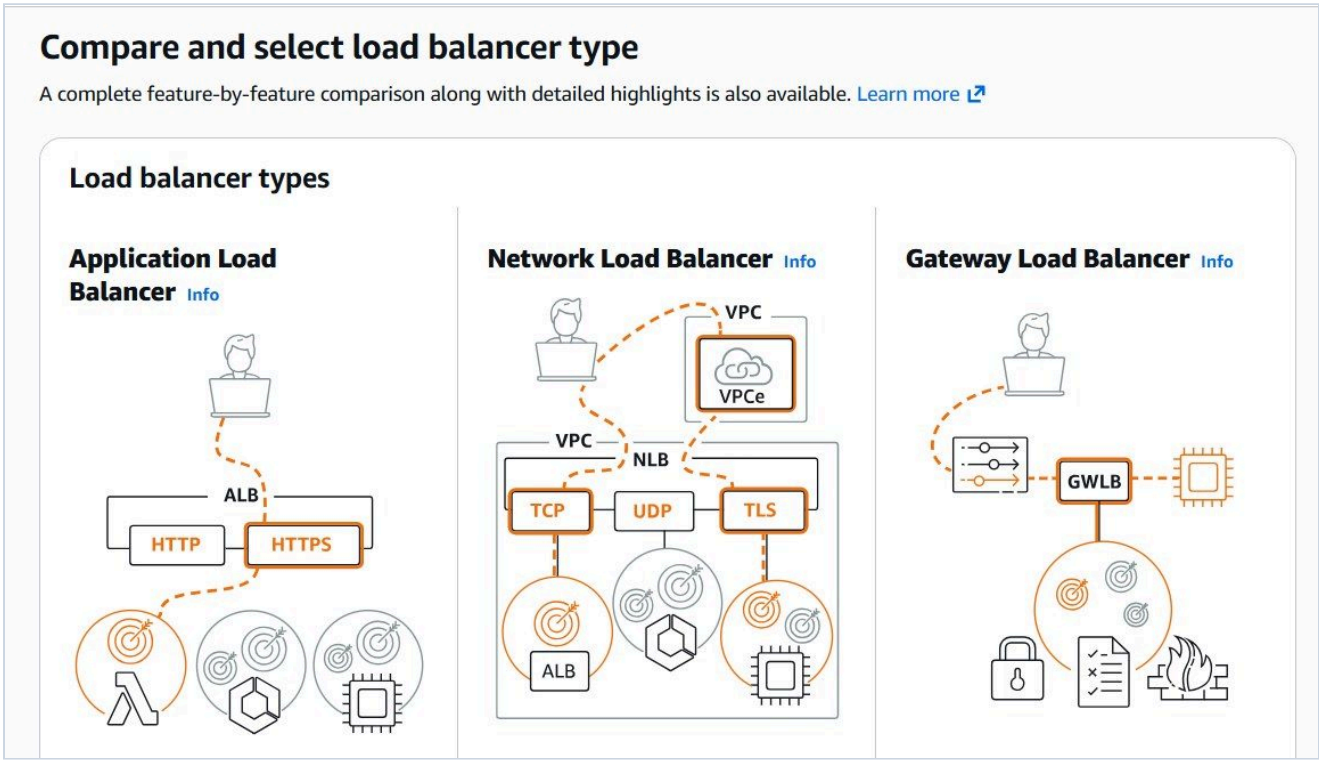


Figure 166. Load balancer types.

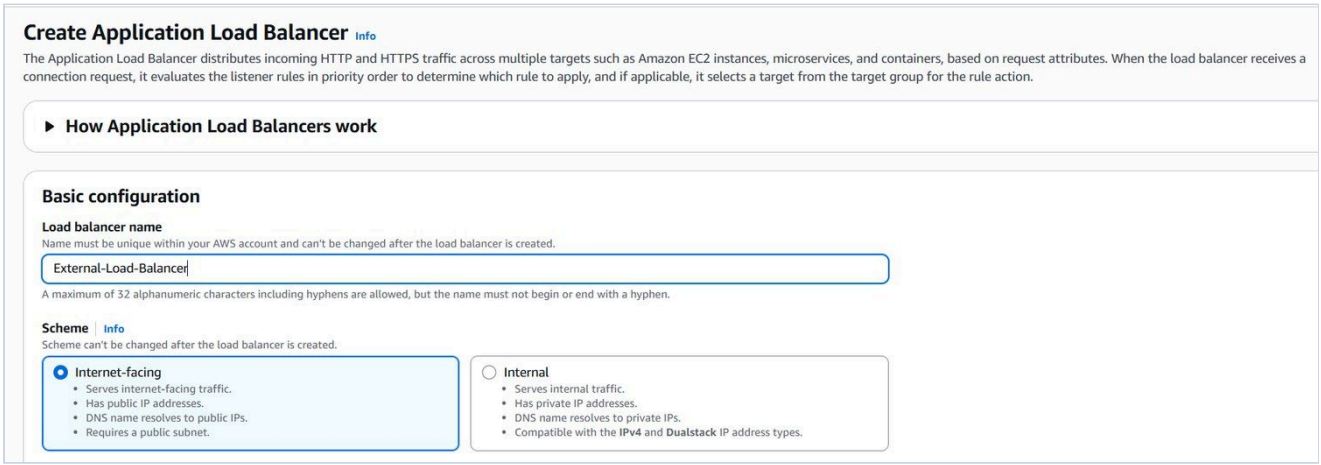


Figure 167. Name and the Internet-facing scheme.

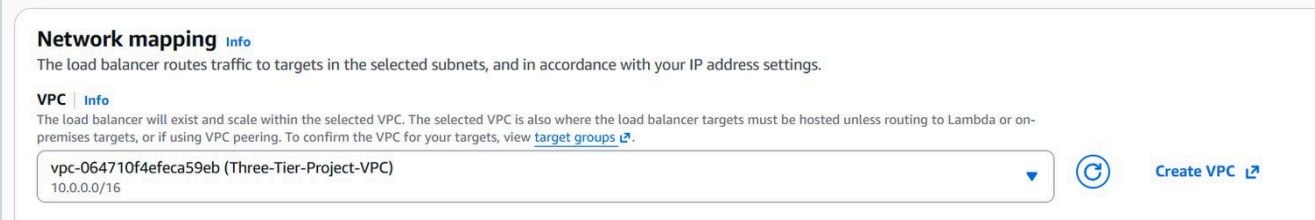


Figure 168. Network mapping: the VPC.

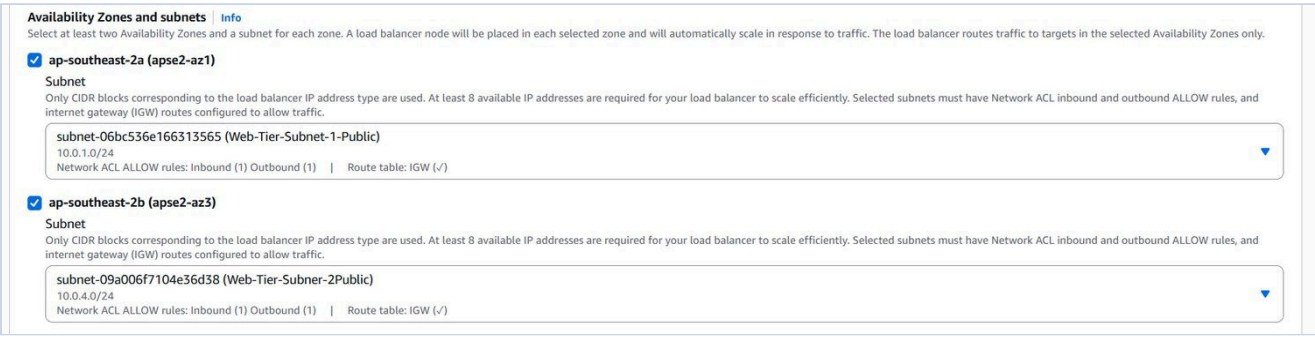


Figure 169. Availability Zones and the public web subnets.

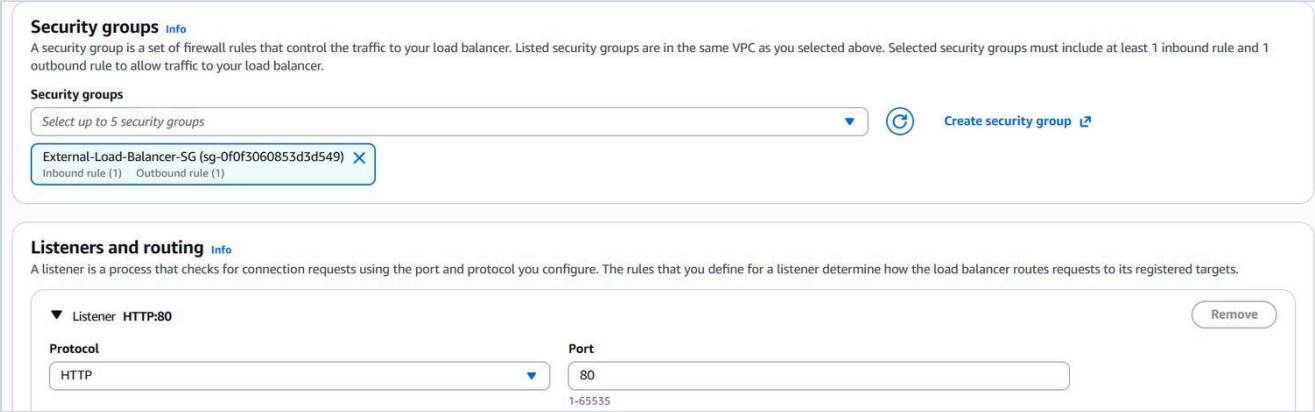


Figure 170. Security group and the HTTP:80 listener.

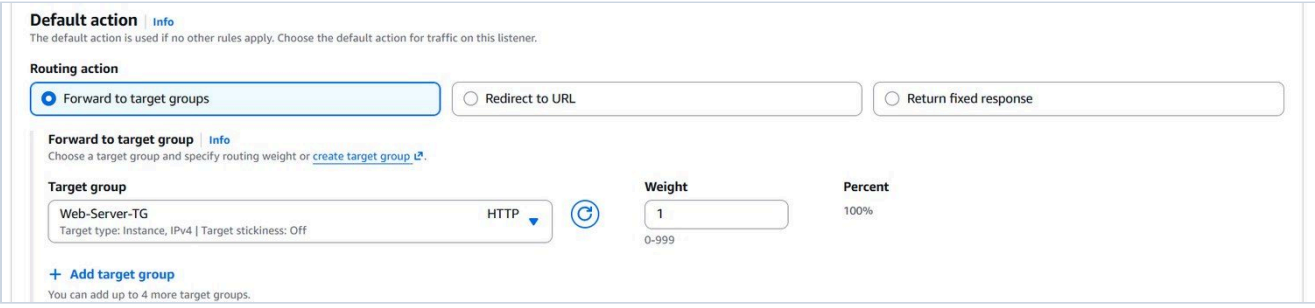


Figure 171. Listener default action: forward to Web-Server-TG.

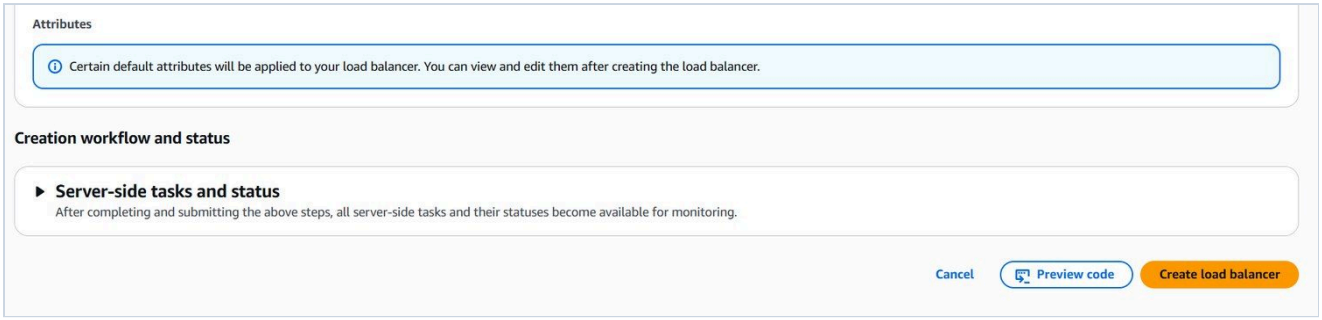


Figure 172. Review, then Create load balancer.

CHECKPOINT

Both target groups and both load balancers exist; the external load balancer is Active and internet-facing.

STEP 13 OF 15

SCALE AND LAUNCH

Create the Auto Scaling Groups

Let AWS keep the servers alive

WHY THIS STEP?

An Auto Scaling group launches servers from the launch template, spreads them across subnets, registers them with the target group and replaces any that fail health checks.

A. Auto Scaling group for the app tier

SETTING	VALUE
Name	`App-Server-ASG`
Launch template	App-Server-Launch-Template
VPC / Subnets	Three-Tier-Project-VPC / App-Tier-Subnet-1-Private and App-Tier-Subnet-2-Private
Load balancing	Attach to an existing load balancer: target group App-Server-TG
Health checks	Turn on Elastic Load Balancing health checks
Group size	Desired 2, minimum 2, maximum 2 (or higher for scaling)

1. Go to **EC2 > Auto Scaling groups > Create Auto Scaling group** and use the settings above. Keep the other options at their defaults and click **Create Auto Scaling group**.

SCREENSHOT TO ADD

Auto Scaling group for the app tier: launch template, subnets, target group and group size.

B. Auto Scaling group for the web tier

SETTING	VALUE
Name	`Web-Server-ASG`
Launch template	Web-Server-Launch-Template
VPC / Subnets	Three-Tier-Project-VPC / Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public
Load balancing	Attach to an existing load balancer: target group Web-Server-TG
Health checks	Turn on Elastic Load Balancing health checks
Group size	Desired 2, minimum 2, maximum 2 (or higher for scaling)

SCREENSHOT TO ADD

Auto Scaling group for the web tier: launch template, subnets, target group and group size.

When both groups are running, the instance list shows four new servers (two web, two app) in addition to the two test servers. Terminate the two test instances, as they are no longer needed.

PRO TIP

Use a desired size of at least 2 per tier. With one server in each zone, a zone failure still leaves one healthy server serving traffic.

CHECKPOINT

Each Auto Scaling group launches its servers across both subnets and both target groups show the registered targets as healthy.

STEP 14 OF 15 SCALE AND LAUNCH

Access the Application

Send a request through every tier

WHY THIS STEP?

This is the moment of truth: a request from your browser must cross the external load balancer, the web servers, the internal load balancer, the app servers and finally the database.

1. Open **EC2 > Load Balancers** and select the internet-facing load balancer.
2. Copy its **DNS name**.
3. Paste it into your browser as `http://<external-load-balancer-dns-name>` (use a private window if it does not load).
4. Open the **DB DEMO** page, add a transaction and confirm it is listed.
5. Confirm the same row exists in MySQL using the client from Step 7.

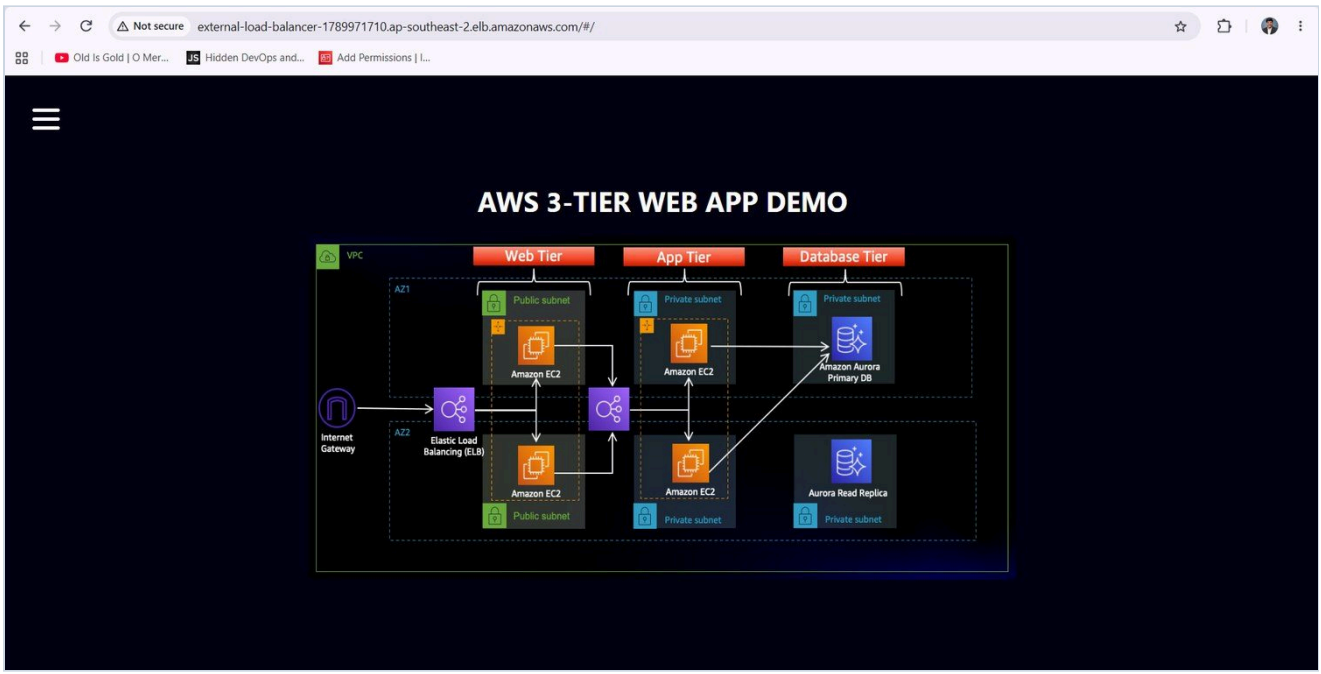


Figure 173. The application opened in the browser through the external load balancer DNS name.

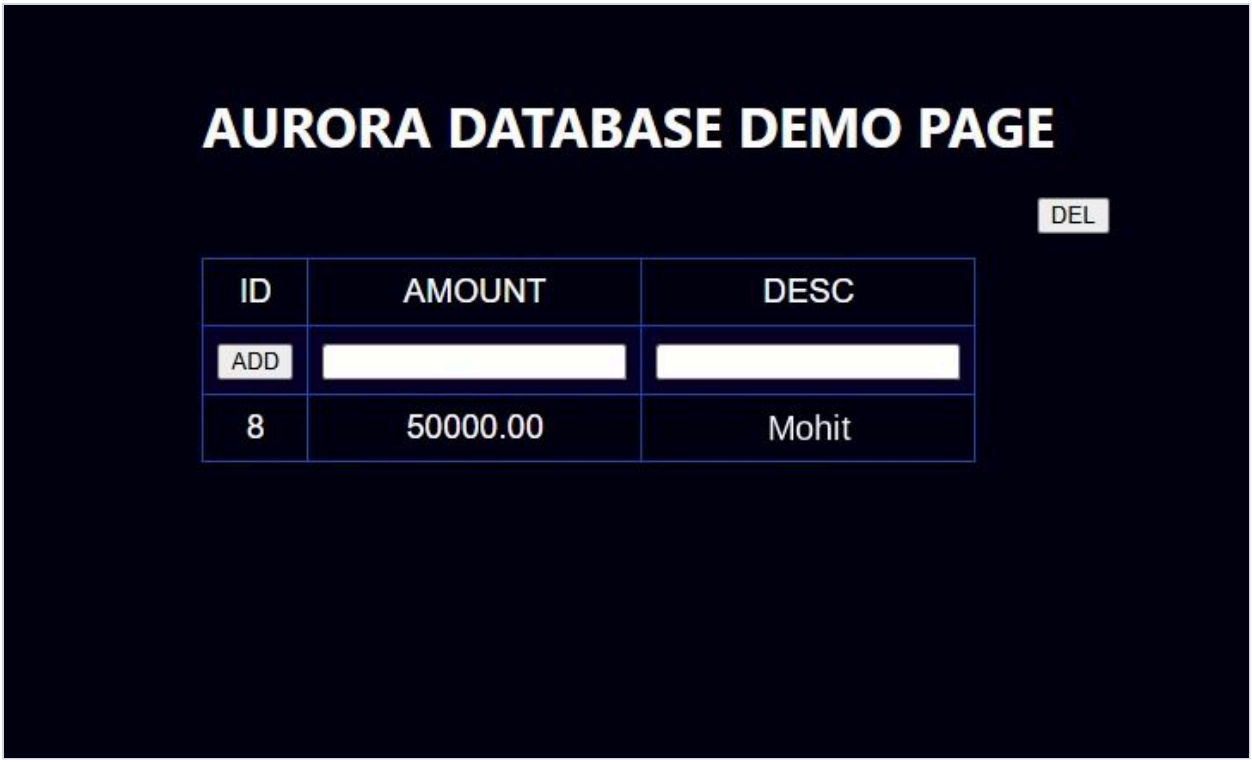


Figure 174. Illustrative screenshot based on the source guide: the Database Demo page listing a transaction (description "Mohit") added through the web tier.

Prove the fault tolerance

- Terminate one web or app instance in the EC2 console. The load balancer stops sending it traffic and the Auto Scaling group launches a replacement from the launch template.
- Refresh the application during this time. It should keep working because the second server is still healthy.

PRO TIP

For a friendly URL, register a domain and create a Route 53 alias record that points to the external load balancer, then add an HTTPS listener with an ACM certificate.

CHECKPOINT

The application loads through the external DNS name, a new transaction is stored in MySQL, and the app survives the loss of one server.

STEP 15 OF 15 SCALE AND LAUNCH

Clean Up the Infrastructure

Switch off the meter

WHY THIS STEP?

Several services bill by the hour whether or not anyone is using the application. Delete everything once your screenshots are captured.

COST WARNING

NAT gateways, load balancers, the database and the instances all bill by the hour. Delete everything as soon as you have your screenshots.

Delete in this order so no resource is blocked by something that still depends on it:

1. **Auto Scaling groups** for the web and app tiers (this terminates their instances).
2. **Load balancers** (delete the listeners first), then **target groups**.
3. **Launch templates**.
4. **AMIs**: deregister both images, then delete their snapshots.
5. Any remaining **test instances**.
6. **RDS database** (disable deletion protection if enabled; skip the final snapshot and tick the acknowledgement), then the DB subnet group.
7. **NAT gateways**, then release both Elastic IPs.
8. **Route tables**: remove all subnet associations first.
9. **Internet gateway**: detach it from the VPC and delete it.
10. **Security groups** and **subnets**, then the **VPC**.
11. **S3 bucket**: empty it first, then delete it.
12. **IAM role** `IAM-role-three-tier-app`.

WARNING

Elastic IPs that are allocated but not attached keep billing. Check **VPC > Elastic IPs** at the end to make sure the list is empty.

CHECKPOINT

The Billing console shows no running resources from this project and the VPC list contains only the default VPC.

Troubleshooting Quick Guide

SYMPTOM	LIKELY CAUSE	WHAT TO CHECK
Private server cannot download packages	No route to a NAT gateway, or the NAT gateway sits in a private subnet	App-Tier route tables have 0.0.0.0/0 to a NAT gateway; NAT gateways are in the public subnets
Instance is not listed under Session Manager	IAM role missing, or the instance cannot reach the SSM endpoints	Role IAM-role-three-tier-app is attached; NAT gateway is Available; wait a few minutes after launch
MySQL connection times out	Security group or endpoint mismatch	DB-Tier-SG allows 3306 from App-Tier-SG; use the RDS endpoint name, not an IP; public access is No on purpose
Target group shows unhealthy targets	Wrong port or health check path, or PM2 or nginx not running	App-Server-TG uses port 4000 and /health (run `pm2 status`); Web-Server-TG uses port 80 (run `systemctl status nginx`)
Browser shows 502 or 504	nginx.conf points at the wrong load balancer	proxy_pass uses the internal load balancer DNS name on port 80; Internal-Load-Balancer-SG allows Web-Tier-SG
Target group missing when creating a load balancer	The VPC selector is still on the default VPC	Change VPC to Three-Tier-Project-VPC before choosing subnets
Auto Scaling launches but servers never register	Launch template has the wrong AMI, group or profile	Template uses the right AMI, tier security group and IAM instance profile

Appendix A: Resource Inventory

RESOURCE	NAME / VALUE
Region and zones	Asia Pacific (Sydney) ap-southeast-2: ap-southeast-2a, 2b and 2c
VPC	Three-Tier-Project-VPC, 10.0.0.0/16 (vpc-064710f4efeca59eb)
Public subnets	Web-Tier-Subnet-1-Public 10.0.1.0/24; Web-Tier-Subnet-2-Public 10.0.4.0/24
Private app subnets	App-Tier-Subnet-1-Private 10.0.2.0/24; App-Tier-Subnet-2-Private 10.0.5.0/24
Private DB subnets	DB-Tier-Subnet-1-Private 10.0.3.0/24; DB-Tier-Subnet-2-Private 10.0.6.0/24
Gateways	Internet-GW-Three-Tier-App; NAT-GW-1 and NAT-GW-2 (each with an Elastic IP)
Route tables	Web-tier route table (Web-Tier-RT-2), App-Tier-RT-1, App-Tier-RT-2
Security groups (5)	External-Load-Balancer-SG, Web-Tier-SG, Internal-Load-Balancer-SG, App-Tier-SG, DB-Tier-SG
Database	RDS for MySQL 8.4 (Free tier, Single-AZ, db.t4g.micro), identifier database-1, user admin, database webappdb, table transactions
DB subnet group	DB-Subnet-Group-Three-Tier-Application
IAM role	IAM-role-three-tier-app (AmazonS3ReadOnlyAccess, AmazonSSMManagedInstanceCore)
S3 bucket	three-tier-app-bucket-5-11-26 (folder application-code)
Test servers	Test-App-Server; Test-Web-Server
AMIs	App-server-image; Web-Server-Image
Launch templates	App-Server-Launch-Template; Web-Server-Launch-Template
Target groups	App-Server-TG (HTTP 4000, /health); Web-Server-TG (HTTP 80, /health)
Load balancers	Application-Load-Balancer (internal); External-Load-Balancer (internet-facing)

Appendix B: Review Notes and Hardening

Reviewing the deployment screenshots produced the following findings. None stops the project from working, but each should be addressed before the design is reused beyond a demo.

#	FINDING	RECOMMENDATION
1	The S3 bucket was created with Block all public access turned off (Step 6).	Keep Block all public access on. The servers read the bucket through the IAM role, so public access is unnecessary and risks exposing the source code and DB settings.
2	DbConfig.js holds the database password in plain text and is stored in S3.	Store credentials in AWS Secrets Manager or SSM Parameter Store and read them at runtime. Never commit them to Git.
3	The internal ALB was mapped to web and database subnets instead of the app-tier subnets (Step 9).	Map the internal ALB to App-Tier-Subnet-1 and 2, as in the reference architecture.
4	The App-Tier-SG form was captured with type HTTP (80) while the backend uses port 4000 (Step 3).	Use Custom TCP 4000 from Internal-Load-Balancer-SG so health checks and traffic reach the Node.js app.
5	Names contain slips: Web-Tier-Subner-2Public, App-Tier-Subner-2-Private, Web-Tier-RT-2 for the web table.	Correct the Name tags so they match the documented naming scheme.
6	Auto-assign public IP was enabled for the test app server in a private subnet (Step 7).	Disable auto-assign public IP on private subnets; they do not need a public address.

Ideas to harden for production

- **HTTPS everywhere.** Add an ACM certificate and an HTTPS listener on the external load balancer, then redirect HTTP to HTTPS.
- **Multi-AZ database.** Switch the RDS instance to a Multi-AZ deployment (or Aurora) so a standby takes over automatically.
- **Secrets Manager.** Let RDS manage the master password in AWS Secrets Manager instead of a hand-typed password in DbConfig.js.
- **Tighter outbound rules.** Replace the allow-all outbound rule on each security group with only the ports that tier needs.
- **VPC endpoints.** Add endpoints for S3 and Systems Manager so private servers need less NAT traffic.
- **Web Application Firewall.** Attach AWS WAF to the external load balancer to block common attacks.
- **Monitoring and alarms.** Create CloudWatch alarms for unhealthy hosts, CPU and database storage.
- **Infrastructure as code.** Rebuild the same stack with CloudFormation or Terraform so the environment is repeatable and reviewable.

References and Credits

ITEM	DETAILS
Created by	Mohit Gadilohar
Application source code	Public three-tier sample project by Subodh Kumar: github.com/SubodhK143/three-tier-app-on-aws
AWS documentation	Amazon VPC, Elastic Load Balancing, EC2 Auto Scaling, Amazon RDS, Systems Manager (docs.aws.amazon.com)
Evidence	Screenshots from the author's own AWS account, deployed on 5 November 2026. Two figures captioned "Reference screenshot" and the "Illustrative" Database Demo figure are based on the source guide by Darshan Patil (Canada region) because they were not captured in this deployment; the demo figure has the sample name changed to Mohit.

ABOUT THE AUTHOR



Mohit Gadilohar

Author of this Cloud and DevOps implementation project. Questions, feedback and collaboration are welcome through the links below.

in LinkedIn </> GitHub

https://www.linkedin.com/in/mohit-gadilohar-devops
https://github.com/mohitlohar1555-oss